Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH

Instytut Automatyki i Informatyki Stosowanej

Praca dyplomowa magisterska

na kierunku Automatyka i Robotyka

Projekt, wykonanie oraz nauka chodu autorskiej maszyny kroczącej z wykorzystaniem cyfrowego bliźniaka

Witold Pietroń
Numer albumu 310245

promotor
dr inż. Patryk Chaber

WARSZAWA 2026

Projekt, wykonanie oraz nauka chodu autorskiej maszyny kroczącej z wykorzystaniem cyfrowego bliźniaka

Streszczenie.

Przedmiotem pracy jest projekt, wykonanie oraz nauka stabilnego chodu autorskiej maszyny kroczącej z wykorzystaniem metod uczenia przez wzmacnianie oraz koncepcji cyfrowego bliźniaka. Rozpatrywany problem ograniczono do ruchu w jednym kierunku. W ramach pracy opracowano niskokosztową platformę czworonożną o dwóch stopniach swobody na każdą kończynę, wykonaną w technologii druku 3D. Układ napędowy oparto na modelarskich serwomechanizmach bez możliwości odczytu pozycji oraz regulacji prędkości obrotowej, natomiast jako jednostkę centralną zastosowano mikrokontroler Raspberry Pi Pico.

W kolejnym etapie stworzono cyfrowego bliźniaka robota w środowisku CoppeliaSim, kładąc szczególny nacisk na odwzorowanie parametrów dynamicznych i konstrukcyjnych platformy. Równolegle opracowano wieloskładnikową metrykę oceniającą jakość chodu, uwzględniającą prędkość oraz stabilność ruchu. Następnie przeprowadzono proces uczenia przez wzmacnianie z wykorzystaniem algorytmu Proximal Policy Optimization, architektury asymetrycznego aktora-krytyka oraz techniki *curriculum learning*. Wytrenowana polityka osiągnęła w symulacji ponad trzykrotnie wyższą prędkość niż prędkość uzyskiwana dla trajektorii referencyjnej opracowanej metodą białej skrzynki.

W celu przeniesienia wytrenowanej polityki na platformę fizyczną zaimplementowano autorski mechanizm inferencji na mikrokontrolerze w środowisku MicroPython, zoptymalizowany z wykorzystaniem modułu ulab. Problem rozbieżności pomiędzy symulacją a rzeczywistością (ang. *reality gap*) ograniczono poprzez wprowadzenie do funkcji nagrody kary za gwałtowne zmiany akcji.

Badania eksperymentalne przeprowadzone na fizycznym robocie potwierdziły prawidłowe działanie transferu polityki oraz możliwość uzyskania stabilnego i efektywnego chodu na platformie fizycznej.

Słowa kluczowe: Uczenie przez wzmacnianie, robot czworonożny, Proximal Policy Optimization, cyfrowy bliźniak, transfer sim-to-real

Design, construction, and reinforcement learning of a custom quadruped robot using a digital twin

Abstract.

The subject of this thesis is the design, construction and learning of stable locomotion for a custom-built legged machine using reinforcement learning methods and the digital twin. The considered problem is limited to motion in a single direction. As part of the thesis, a low-cost quadrupedal platform with two degrees of freedom per leg was developed using 3D printing technology. The drive system is based on hobby-grade servomechanisms without position feedback or rotational speed control, while a Raspberry Pi Pico microcontroller was used as the central control unit.

In the next stage, a digital twin of the robot was developed in the CoppeliaSim environment, with particular focus placed on accurately reproducing the dynamic and structural parameters of the platform. Simultaneously, a multi-component gait quality metric was developed, taking into account both locomotion, speed and stability. Reinforcement learning process was then performed using the Proximal Policy Optimization algorithm, an asymmetric actor-critic architecture, and curriculum learning. Trained policy achieved velocity more than three times higher than that obtained using the reference trajectory developed with a white-box approach.

To transfer the trained policy to the physical platform, a custom inference mechanism was implemented on the microcontroller using the MicroPython environment and optimized with the ulab module. The reality gap between the simulation and the physical system was reduced by introducing a penalty for abrupt changes in actions into the reward function.

Experimental tests conducted on the physical robot confirmed the successful transfer of the policy and demonstrated the ability to achieve stable and efficient locomotion on the physical platform.

Keywords: Reinforcement Learning, Quadrupedal Robot, Proximal Policy Optimization, Digital Twin, Sim-to-Real Transfer

Spis treści

1. Wstęp 7
2. Przegląd literatury 12
3. Istniejące konstrukcje robotów czworonożnych 14
3.1. Spot 14
3.2. Unitree 15
3.3. ANYmal X 15
3.4. Stanford Pupper 15
3.5. The Dingo 16
3.6. Mjukost 17
3.7. Podsumowanie 17
4. Uczenie przez wzmacnianie 19
4.1. Deep Q-Learning (DQN) 21
4.2. Trust Region Policy Optimization 22
4.3. Proximal Policy Optimization 23
4.4. Advantage Actor-Critic 26
4.5. Soft Actor-Critic 26
4.6. Podsumowanie 27
5. Konstrukcja robota 28
5.1. Budowa robota 28
5.2. Układ napędowy 34
5.3. Podsumowanie parametrów fizycznych robota 35
5.4. Model kinematyczny 35
6. Nauka chodu w symulacji 40
6.1. Cyfrowy bliźniak 41
6.2. Identyfikacja parametrów dynamicznych konstrukcji 42
6.3. Środowisko symulacyjne 44
6.4. Integracja środowiska z biblioteką Stable-Baselines3 45
6.5. Przestrzeń akcji i obserwacji 46
6.6. Metryka skuteczności chodu 49
6.7. Referencyjna trajektoria chodu 53
6.8. Realizacja sieci polityki 54
6.9. Eksperymenty 55
6.10. Porównanie nauczonej trajektorii chodu z trajektorią referencyjną 60
7. Przeniesienie modelu z symulacji do fizycznego robota 63
7.1. Inferencja sieci polityki na Raspberry Pi Pico 63
7.2. Cykl działania Raspberry Pi Pico 65
7.3. Problemy z przeniesieniem polityki 66

7.4. Wyniki 70
8. Podsumowanie 74
Bibliografia 77
Wykaz symboli i skrótów 80
Spis rysunków 81
Spis tabel 82
Spis załączników 82

1. Wstęp

Mimo szybkiego rozwoju algorytmów sztucznej inteligencji oraz robotyki, można odnieść wrażenie, że maszyny kroczące rozwijają się nieco na uboczu w stosunku do głównego nurtu tych dziedzin. Współczesne trendy koncentrują się przede wszystkim na integracji systemów robotycznych z generatywnymi modelami sztucznej inteligencji, pojazdach autonomicznych oraz rozwoju manipulatorów i robotów humanoidalnych. Jedną z przyczyn mniejszego zainteresowania maszynami kroczącymi jest fakt, że w wielu zastosowaniach praktycznych lepszym wyborem okazują się prostsze i bardziej efektywne platformy kołowe, stanowiące alternatywę dla złożonych w konstrukcji i sterowaniu robotów kroczących.

Roboty mobilne wyposażone w napęd kołowy charakteryzują się większą prostotą mechaniczną, łatwiejszym sterowaniem oraz mniej złożonym procesem planowania ruchu. Z tego względu dominują one w wielu zastosowaniach przemysłowych i usługowych, gdzie są już powszechnie wykorzystywane między innymi do transportu ładunków w magazynach i centrach logistycznych. Coraz częściej znajdują również zastosowanie poza zamkniętymi obiektami, czego przykładem jest rozwój autonomicznych robotów przeznaczonych do dostarczania przesyłek bezpośrednio do odbiorców. Jednakże roboty kroczące oferują istotną przewagę w środowiskach nieustrukturyzowanych, takich jak teren nierówny, obszary o ograniczonej dostępności czy środowiska wymagające pokonywania różnego rodzaju przeszkód. Praktycznym atutem robotów kroczących nad robotami kołowymi jest też możliwość poruszania się po schodach, stanowiących powszechny element przestrzeni zurbanizowanej.

Przełomowym momentem dla rozwoju robotów kroczących było zaprezentowanie w 2019 roku przez firmę Boston Dynamics czworonożnego robota Spot [1], który zyskał szerokie uznanie dzięki swoim, praktycznie bezkonkurencyjnym w dniu premiery, zdolnościom lokomocyjnym oraz wszechstronnym możliwościom zastosowań. Spot stał się pierwszą szeroko wykorzystywaną w zastosowaniach przemysłowych platformą czworonożną i zapoczątkował rozwój podobnych konstrukcji.

Jednym z kluczowych wyzwań w dziedzinie robotów kroczących jest problem generowania stabilnego i efektywnego chodu. Tradycyjne podejścia oparte na modelach kinematycznych i dynamicznych wymagają dokładnego odwzorowania parametrów fizycznych robota oraz często prowadzą do skomplikowanych układów sterowania. Klasyczne, niewykorzystujące uczenia maszynowego, sposoby generowania trajektorii chodu mogą też okazać się mało uniwersalne i sprawdzać się przy ruchu w prostych środowiskach, ale wykazywać ograniczoną odporność na zmienne warunki środowiskowe, takie jak nieregularna geometria podłoża lub zadziałanie zewnętrznej siły. Alternatywą dla tych metod są techniki uczenia przez wzmacnianie, które umożliwiają automatyczne wykształcenie strategii lokomocji poprzez interakcję ze środowiskiem.

Celem niniejszej pracy jest zaprojektowanie, wykonanie oraz przetestowanie czworo-

nożnego robota kroczącego wraz z układem sterowania opartym na metodach uczenia przez wzmacnianie. W ramach pracy zakłada się samodzielne opracowanie konstrukcji mechanicznej robota, przy wykorzystaniu możliwie tanich elementów narzucających pewne ograniczenia dynamiczne. Wobec tego istotnym celem cząstkowym jest opracowanie i implementacja strategii lokomocji, która umożliwi stabilny ruch robota w warunkach narzuconych ograniczeń. Podejście to zostanie początkowo zweryfikowane w środowisku symulacyjnym, a docelowo wdrożone na platformie fizycznej.

Praca obejmuje trzy główne etapy. Pierwszym było zaprojektowanie oraz zbudowanie czworonożnego robota. Drugim krokiem, przeprowadzonym w całości na cyfrowym bliźniaku robota, było wytrenowanie polityki lokomocji przy użyciu algorytmów uczenia przez wzmacnianie. Trzecim i finalnym etapem była implementacja wyuczonej strategii sterowania ruchem na rzeczywistą platformę.

W trakcie pracy przyjęto szereg świadomych uproszczeń, mających na celu ułatwienie projektowania i budowy konstrukcji, a także implementacji opracowanych pod nią algorytmów. Jednym z kluczowych założeń było przyjęcie, że robot porusza się wyłącznie po płaskim i nieśliskim terenie, co znacznie uprościło proces tworzenia środowiska symulacyjnego i sam proces nauki. Ponadto ograniczono liczbę stopni swobody układu kinematycznego. Dzięki tym uproszczeniom autor mógł skoncentrować się na głównym celu badań, jakim było opracowanie i optymalizacja podstawowej trajektorii chodu robota, zapewniającej stabilność i efektywność ruchu wzdłuż jednej osi.

Należy zauważyć, że w warunkach idealnie płaskiej powierzchni zastosowanie tradycyjnej platformy kołowej byłoby rozwiązaniem znacznie prostszym, tańszym i bardziej efektywnym energetycznie. Wybór układu kroczącego nie podyktowano jednak dążeniem do optymalizacji transportu na gładkim podłożu, lecz chęcią zbadania granic możliwości sterowania w warunkach silnych ograniczeń sprzętowych. Główną motywacją pracy było wykazanie, że wykorzystanie metod uczenia przez wzmacnianie pozwala na uzyskanie stabilnej lokomocji wielonożnej nawet przy użyciu skrajnie niskobudżetowych komponentów o ograniczonej precyzji i dynamice.

W trakcie pracy porównano działanie różnych algorytmów uczenia przez wzmacnianie takich jak Proximal Policy Optimization, Advantage Actor-Critic oraz Soft Actor-Critic. Dodatkowo, eksperymentalnie zbadano wpływ kluczowych parametrów sterowania robotem w symulacji na jakość jego chodu oraz dostrojono formułę metryki, stosowanej w trakcie procesu uczenia, oceniając jej wpływ na uzyskiwane wyniki. Dodatkowo, zweryfikowano wybrane, obecne w literaturze techniki usprawnienia procesu uczenia.

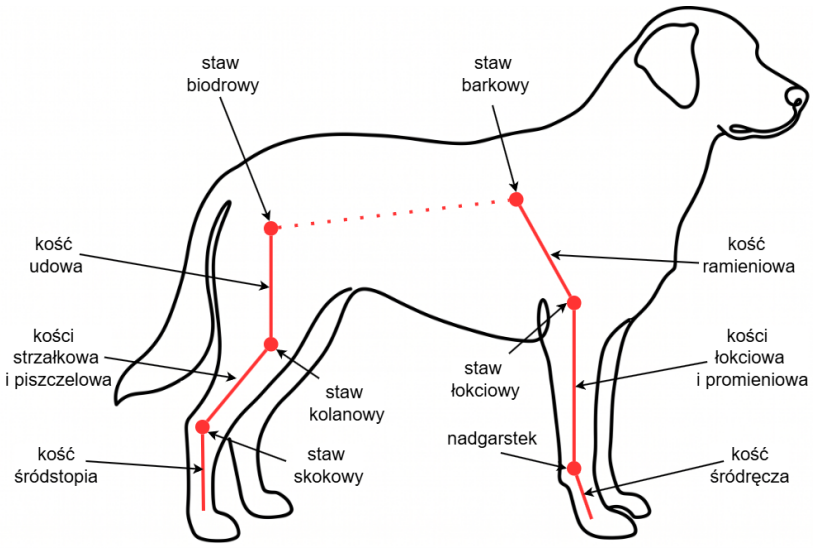
W celu oceny skuteczności zaproponowanego rozwiązania, stworzono bazową trajektorię referencyjną, opartą na klasycznym modelu kinematycznym robota, uwzględniającym parametry takie jak długości przegubów oraz prędkości kątowe poszczególnych stawów. Następnie wyuczoną trajektorię ruchu porównano z trajektorią referencyjną,

aby porównać obydwa podejścia pod względem prędkości, efektywności oraz stabilności generowanego ruchu.

Praca rozpoczyna się od krótkiej analizy chodu psa – biologicznej inspiracji robota czworonożnego. W rozdziale drugim przedstawiono przegląd literatury dotyczącej robotów kroczących oraz algorytmów uczenia przez wzmacnianie. Kolejny, trzeci, rozdział poświęcono krótkiemu opisowi popularnych platform czworonożnych, zarówno komercyjnych jak i tych otwartoźródłowych. Następnie, w rozdziale czwartym, opisano historię rozwoju oraz zasadę działania popularnych dziś i wykorzystywanych w pracy algorytmów uczenia przez wzmacnianie. Rozdział piąty poświęcono krótkiemu opisowi stworzonej konstrukcji. Po nim, w rozdziale szóstym, następuje fragment dedykowany faktycznemu procesowi nauki oraz eksperymentom przeprowadzonym w symulacji. Następnie, w rozdziale siódmym, przedstawiono wyniki implementacji algorytmu na rzeczywistym robocie i opisano trudności napotkane w jej trakcie. Całość pracy kończy podsumowanie oraz wskazanie możliwych kierunków dalszego rozwoju.

Lokomocja psa

Z uwagi, że stworzona konstrukcja maszyny kroczącej jest w dużym stopniu inspirowana wzorcami zwierząt czworonożnych, warto poświęcić chwilę na bliższe przyjrzenie się wzorcom chodu występującym w przyrodzie. Poniżej opisano pokrótce sposób przemieszczania się psa, którego budowa anatomiczna i proporcje kończyn posłużyły jako wzorzec podczas projektowania robota.



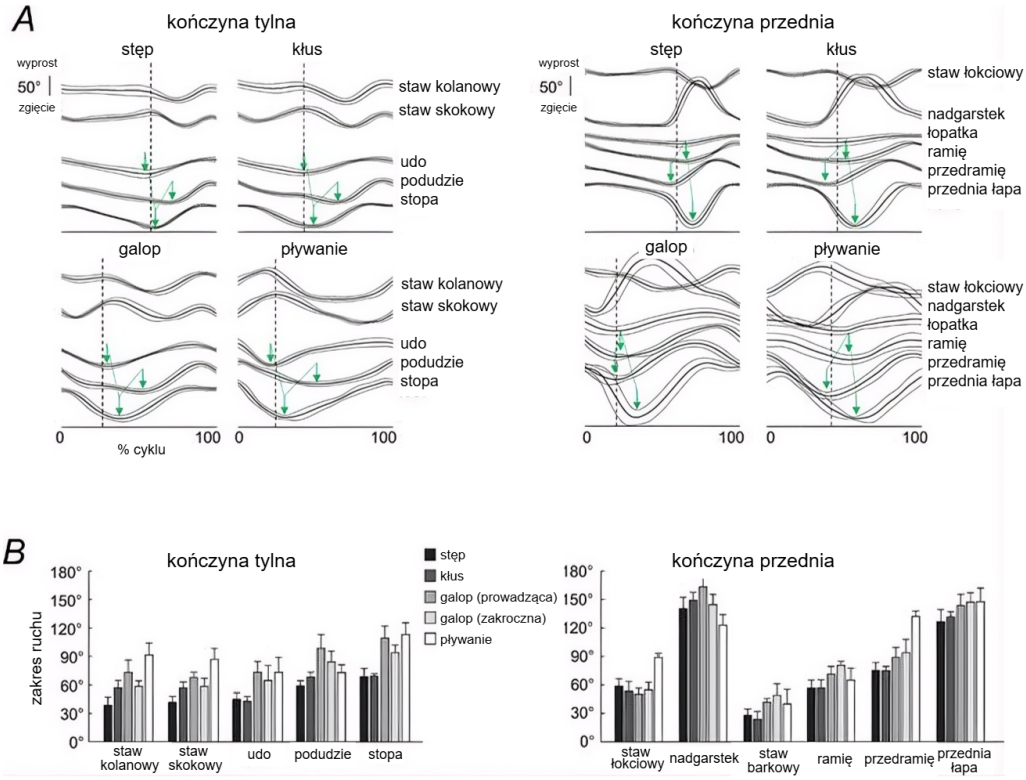
Rysunek 1.1. Rysunek ilustrujący schematycznie ułożenie głównych kości i stawów składających się na kończyny tylne i przednie psa

Zważając na fakt, że pies, tak samo jak człowiek, należy do rodziny ssaków, jego ciało składa się z kości i stawów zbliżonych do tych występujących u człowieka. W trakcie procesu ewolucji zmieniła się długość, jak i rola poszczególnych kości, jednak zasadnicza

hierarchia budowy kończyn pozostała taka sama. Kończyny przednie psa zbudowane są z kości: ramieniowej, promieniowej i łokciowej, a także śródręcza zakończonego palcami. Kończyny tylne natomiast składają się z kości: udowej, piszczelowej, strzałkowej oraz śródstopia i palców. Rysunek 1.1 przedstawia schematyczną budowę kończyn psa z zaznaczonymi głównymi kośćmi oraz stawami.

Znaczącą różnicą w budowie kończyny psa, w porównaniu do nogi człowieka, jest wyższe położenie stawu skokowego względem podłoża. W konsekwencji, o ile kończynę człowieka można w uproszczeniu modelować za pomocą dwóch segmentów, to nawet uproszczony model kończyny psa musi składać się z trzech segmentów.

Anatomia psa umożliwia mu wiele różnych form przemieszczania się, z których trzy najważniejsze to stęp, kłus i galop. W literaturze naukowej [2] dostępne są szczegółowe informacje na temat zakresów ruchu w poszczególnych stawach podczas każdego z wymienionych chodów. Rysunek 1.2 przedstawia trajektorie oraz zakresy zmian kątów w stawach kończyny przedniej i tylnej psa dla wymienionych wcześniej form lokomocji.



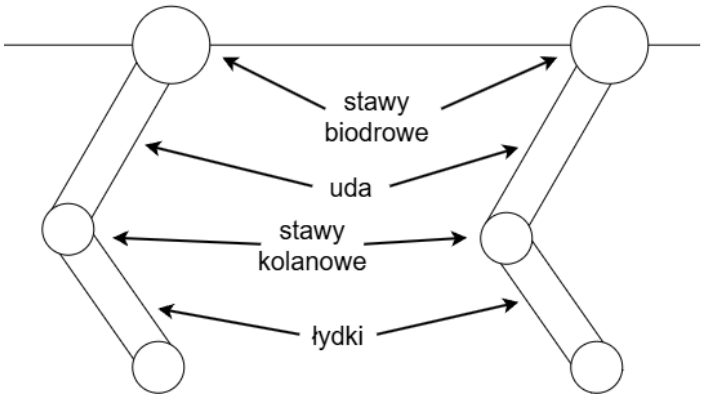
Rysunek 1.2. Przebiegi zmian kątów w stawach kończyn psa podczas różnych form lokomocji. Źródło: tłumaczenie własne na podstawie [2].

Analizując wykres B rysunku 1.2 możemy zauważyć, że zakres ruchu stawów kolanowego i łokciowego wynosi w trakcie kłusu około 60°. Zakresy ruchu stawów biodrowego i barkowego wynoszą wówczas około 45° i 30°. Natomiast w pracy[3] oszacowano te wartości na odpowiednio $52,9° \pm 7,0°$ dla stawu łokciowego i $30,1° \pm 3,7°$ dla barku w trakcie ruchu

po płaskim podłożu. W tym artykule, oszacowano również maksymalne prędkości kątowe w tych stawach jako $4{,}56 \pm 0{,}5$ rad/s i $1{,}51 \pm 0{,}4$ [3].

W trakcie projektowania robota założono jednak, że maksymalny zakres ruchu odpowiadających im stawów będzie większy, aby uwzględnić fakt, że robot będzie posiadał mniejszą liczbę stopni swobody w porównaniu do swojego biologicznego odpowiednika. W związku z tym zwiększenie dopuszczalnych zakresów ruchu stanowi kompensację uproszczonej kinematyki układu oraz ma na celu zapewnienie wystarczającej swobody do realizacji podobnych wzorców lokomocji.

Podobnie jak w przypadku robota Spot, noga maszyny tworzonej w ramach tego projektu została uproszczona do dwóch segmentów, w konfiguracji najbardziej przypominającej – jeśli szukać odpowiednika w anatomii psa – połączenie kości promieniowej i łokciowej z kośćmi śródręcza, poruszanymi przez stawy odpowiadające stawowi łokciowemu i nadgarstkowi, które przedstawione są na rysunku 1.1. Mimo tego uproszczenia, zarówno stawy, jak i segmenty nóg robota będą konsekwentnie nazywane: udem i stawem biodrowym oraz łydką i stawem kolanowym. Schemat konstrukcji wraz z nazwami poszczególnych jej segmentów przedstawiony jest na rysunku 1.3. Choć z biologicznego punktu widzenia nie są to określenia precyzyjne (np. staw kolanowy w rzeczywistości zgina się w przeciwną stronę), wydają się autorowi dużo bardziej intuicyjne.



Rysunek 1.3. Rysunek przedstawiający nazwy nadane poszczególnych segmentom kończyn robota

Na koniec, w formie dygresji można dodać, że analiza prac traktujących o sposobie lokomocji psów prowadzi do wniosku, że jest to zagadnienie wysoce złożone, co wynika zarówno z dużej liczby stopni swobody układu, jak i silnych zależności pomiędzy ruchami poszczególnych kończyn. W praktyce pełne odtworzenie wzorca chodu wymaga jednoczesnego uwzględnienia trajektorii wszystkich czterech kończyn. Programowanie robotów naśladujących ten ruch metodą modeli „białych skrzynek” wydaje się być bardzo trudne do wykonania – tym bardziej uzasadnione zdaje się wykorzystanie metod uczenia maszynowego.

2. Przegląd literatury

Jak wspomniano we wstępie, dopiero od 2019 roku można zaobserwować wyraźne przyspieszenie w rozwoju maszyn kroczących oraz rosnące zainteresowanie nimi w środowisku badawczym. Dostrzeżono przewagę tego typu konstrukcji nad kołowymi robotami mobilnymi w pewnych zastosowaniach wymagających szczególnej odporności robota na warunki terenowe, takie jak nierówności, pokonywanie przeszkód i efektywne wspinanie się na schody [4]. Umiejętności te umożliwiają maszynie kroczącej docieranie do trudno dostępnych miejsc – coraz częściej więc roboty takie znajdują zastosowanie w branżach, takich jak przemysł motoryzacyjny [5], przemysł spożywczy [6], branża budowlana [7] czy militarna [8]. Roboty kroczące są wykorzystywane przeważnie w roli automatycznych inspektorów i kontrolerów procesu produkcyjnego i jego utrzymania.

Roboty kroczące znajdują również zastosowania w pracy w środowiskach niebezpiecznych dla człowieka, czego przykładem może być ich wykorzystanie do gospodarowania odpadami z elektrowni jądrowych [9]. Pierwsze roboty kroczące pokonują już również drabiny otwierając przed nimi pole do zastąpienia człowieka w roli inspektora kolejnych, potencjalnie niebezpiecznych, miejsc [10].

W gronie badaczy panuje również zgoda, że wśród maszyn kroczących to konstrukcje czworonożne wyróżniają się najlepszym połączeniem stabilności ruchu, łatwości sterowania i prostoty projektowania [11]. Podkreślana jest również wszechstronność zastosowań czworonożnej konstrukcji [4]. Roboty czworonożne zdają się zatem być dobrym kompromisem, łącząc stabilność charakterystyczną dla konstrukcji wielonożnych z prostotą sterowania.

W literaturze dostępny jest także zbiór cennych wskazówek i wytycznych dotyczących projektowania oraz budowy tego typu maszyn. Różni autorzy prezentują swoje własne konstrukcje, takie jak Xdog [12] czy Mjukost [13]. Ponadto, niektórzy badacze zwracają uwagę na problem wysokich kosztów tych konstrukcji, proponując alternatywne, bardziej przystępne cenowo rozwiązania [14].

Ponieważ kluczowym elementem pracy będzie wykorzystanie algorytmów uczenia maszynowego do nauki chodu maszyny kroczącej, warto przyjrzeć się również literaturze w tej dziedzinie. Główny nacisk zostanie położony na metody uczenia przez wzmacnianie (ang. *reinforcement learning*), w szczególności na Advantage Actor-Critic (A2C) stanowiący uproszczoną i bardziej praktyczną wersję algorytmu Asynchronous Advantage Actor Critic (A3C) [15], Proximal Policy Optimization [16] oraz Soft Actor-Critic[17].

Powyższe algorytmy są już z powodzeniem wykorzystywane do nauki poruszania się różnych robotów takich jak Minitaur [18]. Autorzy pracy zwracają uwagę na problemy techniczne wiążące się z nauką rzeczywistego robota, takich jak konieczność długotrwałych iteracji, które są czasochłonne i kosztowne. W odpowiedzi na te wyzwania proponują oni własny algorytm ograniczający liczbę niezbędnych iteracji uczących do minimum.

Istotnym problemem metod uczenia przez wzmacnianie jest również duża wrażli-

wość na dobór hiperparametrów oraz ograniczona uniwersalność pomiędzy różnymi zadaniami. W praktyce większość algorytmów wymaga odpowiedniego dostrojenia do konkretnego problemu. Dodatkowo często konieczne jest zastosowanie dodatkowych technik wspomagających stabilność i efektywność procesu uczenia. W niniejszej pracy wykorzystano między innymi opisywane w literaturze podejście uczenia programowego (ang. *curriculum learning*) [19] oraz technikę asymetrycznego rozdzielenia przestrzeni obserwacji pomiędzy aktora i krytyka [20].

Kolejnym istotnym zagadnieniem jest problem przenoszenia wiedzy zdobytej podczas nauki w symulacjach na rzeczywiste maszyny, co na dzień dzisiejszy pozostaje dużym wyzwaniem. Autorzy prac takich jak [21] i [22] proponują różne techniki, na przykład randomizację środowiska symulacyjnego, które umożliwiają skuteczniejsze przenoszenie wyuczonej wiedzy na fizyczne maszyny.

Pomimo dynamicznego rozwoju metod uczenia przez wzmacnianie i ich implementacji w robotyce, skuteczne przenoszenie polityk wyuczonych w symulacji na rzeczywiste roboty nadal stanowi istotne wyzwanie. Problem ten jest szczególnie widoczny w przypadku niskokosztowych konstrukcji, wyposażonych w ograniczoną liczbę czujników oraz uproszczone elementy wykonawcze. Z tego względu istotnym zagadnieniem pozostaje opracowywanie metod umożliwiających efektywne uczenie stabilnego chodu przy wykorzystaniu możliwie prostych i tanich platform sprzętowych.

3. Istniejące konstrukcje robotów czworonożnych

Roboty kroczące stanowią wyspecjalizowaną gałąź robotyki, której rozwój wiąże się z licznymi wyzwaniami mechanicznymi oraz problemami związanymi ze sterowaniem. Mimo to, w ostatnich latach pojawiło się wiele interesujących konstrukcji, zarówno komercyjnych, jak i otwartoźródłowych, które przyczyniły się do rozwoju tej dziedziny. W niniejszym rozdziale przedstawiono wybrane roboty stanowiące punkt odniesienia dla projektowanego systemu.

3.1. Spot

Robot Spot (rys. 3.1a) opracowany przez firmę Boston Dynamics jest jedną z najbardziej rozpoznawalnych współczesnych konstrukcji czworonożnych. Platforma została zaprojektowana do autonomicznego poruszania się w trudnym terenie i może być wykorzystywana do wykonywania różnego rodzaju zadań inspekcyjnych i pomiarowych.

Konstrukcja posiada dwanaście stopni swobody, po trzy dla każdej nogi. Dzięki zaawansowanym algorytmom stabilizacji ruchu robot jest w stanie płynnie się poruszać, utrzymywać równowagę po zaburzeniu ruchu, pokonywać przeszkody, wchodzić po schodach, a także odwrócić się i podnieść po przewróceniu na plecy. Dodatkowo możliwe jest rozszerzenie konstrukcji o manipulator o sześciu stopniach swobody, który umożliwia robotowi wykonywanie całej gamy dodatkowych zadań.

(a) Spot **(b)** Unitree Go2

Rysunek 3.1. Porównanie wybranych komercyjnych konstrukcji robotów czworonożnych: (a) Spot, (b) Unitree Go2

Robot Spot odegrał istotną rolę w popularyzacji komercyjnego wykorzystania robotów czworonożnych i, mimo rozwoju konkurencji, prawdopodobnie wciąż jest najpowszechniejszą konstrukcją tego typu mającą praktyczne zastosowania przemysłowe. Z tego

względu stanowi istotny punkt odniesienia dla wszystkich konstruktorów współczesnych robotów kroczących.

3.2. Unitree

W ostatnim czasie dużą popularność zyskały roboty chińskiego producenta Unitree, który w ostatnich latach dokonał dynamicznego rozwoju w zakresie budowy robotów kroczących. W swojej gamie, oprócz robotów humanoidalnych, posiada on też trzy konstrukcje czworonożne: edukacyjną Unitree Go2 (rys. 3.1b), większą Unitree A2 oraz typowo przemysłową Unitree B2. Konstrukcje te swoimi możliwościami są zbliżone do robota Spot i oferowane są przy niższych kosztach niż porównywalne konkurencyjne rozwiązania.

3.3. ANYmal X

Robot ANYmal X (rys. 3.2a), opracowany przez szwajcarską firmę ANYbotics, stanowi unikalną konstrukcję na tle konkurencji ze względu na swoją wysoką specjalizację przemysłową. W przeciwieństwie do robotów Spot czy Unitree, ANYmal X został zaprojektowany z myślą o pracy w skrajnie niebezpiecznych środowiskach, co potwierdza certyfikat ATEX/IECEx (Zone 1).

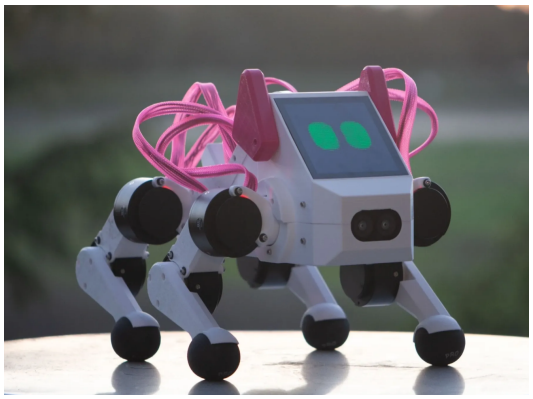
Konstrukcja ta charakteryzuje się wysokim stopniem ochrony przed pyłem i wodą (IP67) oraz osłoniętymi komponentami, które eliminują ryzyko wywołania zapłonu w atmosferze wybuchowej. Pod względem mechanicznym robot wykorzystuje zaawansowane siłowniki o wysokim momencie obrotowym, co pozwala mu na precyzyjne poruszanie się po wysokich schodach i wąskich przejściach typowych dla platform wiertniczych, zakładów chemicznych oraz kopalni.

3.4. Stanford Pupper

Równolegle do rozwoju platform komercyjnych prowadzony był rozwój maszyn kroczących budowanych na licencji otwartoźródłowej. Wśród nich warto wymienić platformę Stanford Pupper – robota czworonożnego opracowanego na Uniwersytecie Stanforda w celach edukacyjnych oraz badawczych. Konstrukcja została zaprojektowana jako stosunkowo tania i łatwa do odtworzenia platforma badawcza [23].

Od momentu publikacji projektu w 2021 roku powstały na jego bazie kolejne konstrukcje: otwartoźródłowy Pupper v3 (rys.3.2b) oraz komercyjnie sprzedawany Mini Pupper. Oba projekty kontynuują idee rozwoju robotów kroczących niewielkich rozmiarów przeznaczonych do działań rozwojowych, badawczych lub nawet służąc jako inteligentne zabawki. Oba projekty rozbudowały konstrukcję o ekran umieszczony w przedniej części robota, pełniący między innymi rolę twarzy, nadając swoim robotom bardziej przyjazny dla człowieka wygląd. Obie koncepcje dobrze nadają się do implementacji algorytmów sterowania ruchem oraz eksperymentów związanych z autonomią i uczeniem maszynowym.

(a) ANYmal X



(b) Pupper v3

Rysunek 3.2. Porównanie wybranych konstrukcji robotów czworonożnych:
(a) ANYmal X, (b) Pupper v3

3.5. The Dingo

Inną ciekawą konstrukcją robota czworonożnego jest otwartoźródłowa platforma The Dingo, przedstawiona na rysunku 3.3a. Autorzy projektu dążyli do stworzenia całkowicie otwartoźródłowej, a przy tym zaawansowanej maszyny o dużych możliwościach, która umożliwiłaby rozwój maszyn czworonożnych przez studentów i entuzjastów z całego świata.

Tym, co wyróżnia tę konstrukcję pod względem mechanicznym, jest interesujący sposób realizacji napędu stawów kolanowych. W robocie tym zdecydowano się na przeniesienie wszystkich elementów napędowych, w tym odpowiedzialnych za ruch stawu kolanowego, do wnętrza korpusu oraz przenoszenie ruchu do stawu kolanowego za pomocą dedykowanego mechanizmu. Umożliwiło to zmniejszenie masy kończyn oraz wykorzystanie w roli napędu trzech takich samych serwomechanizmów o standardowych rozmiarach i niewielkiej cenie. Rozwiązanie to pozwoliło uniknąć konieczności stosowania niewielkich serwomechanizmów o dużej mocy, które byłyby znacznie droższe. Warto również dodać, że w tej konstrukcji elementy napędowe stawu kolanowego i biodrowego znajdują się na tej samej osi, co znacząco upraszcza konstrukcję robota.

Podsumowując, The Dingo jest konstrukcją łączącą prostotę projektową i duże możliwości platformy, będąc jedną z najbardziej zaawansowanych platform otwartoźródłowych zbliżonych możliwościami do konstrukcji komercyjnych. Rozwiązania zastosowane w robocie The Dingo, jak również cały projekt, mogą stanowić bardzo wartościowe źródło doświadczeń dla projektanta robota czworonożnego.

3.6. Mjukost

Innym ciekawym przykładem konstrukcji otwartoźródłowej jest robot Mjukost, przedstawiony na rysunku 3.3b. Jest on eksperymentalną konstrukcją robota czworonożnego opracowaną w celach badawczych i edukacyjnych. W projekcie tym szczególny nacisk położono na uproszczenie konstrukcji, wykorzystaniu łatwo dostępnych komponentów i minimalizacji kosztu produkcji.

Konstrukcja Mjukost pokazuje możliwość budowy funkcjonalnego robota czworonożnego przy stosunkowo niewielkich kosztach. Istotną cechą projektu jest możliwość wykonania wszystkich elementów konstrukcyjnych za pomocą druku 3D, co znacząco upraszcza proces budowy. Całkowity szacowany koszt wykonania robota wynosi około 900 USD. Znaczną część tej kwoty stanowi koszt napędów – konstrukcja wykorzystuje 12 inteligentnych serwomechanizmów Dynamixel AX-12A, których cena jednostkowa wynosi około 50 USD. Serwomechanizmy te zapewniają sprzężenie zwrotne, co umożliwia realizację zaawansowanych algorytmów sterowania.

W konstrukcji Mjukost zastosowano rozwiązania sprzętowe zbliżone do planowanych w robocie opisywanym w niniejszej pracy, w szczególności wykorzystując serwomechanizmy oraz lekką konstrukcję opartą na elementach drukowanych. Z tego względu projekt Mjukost stanowi wartościowy punkt odniesienia podczas projektowania własnej platformy robotycznej.

(a) The Dingo **(b)** Mjukost

Rysunek 3.3. Porównanie wybranych otwartoźródłowych konstrukcji robotów czworonożnych: (a) The Dingo X, (b) Mjukost

3.7. Podsumowanie

Przedstawione powyżej platformy nie opisują oczywiście wszystkich rozwijanych na świecie koncepcji maszyn czworonożnych. Wśród niewymienionych wcześniej platform warto wspomnieć również o grupie robotów czworonożnych nastawionych na możliwie wierne naśladowanie psów pod względem wyglądu oraz zachowania. Do konstrukcji tego typu należą między innymi CyberDog 2 firmy Xiaomi oraz Bittle firmy Petoi. Prze-

analizowane konstrukcje pokazują jednak różne podejścia do projektowania robotów czworonożnych – od zaawansowanych platform komercyjnych po uproszczone, otwarto-źródłowe konstrukcje edukacyjne. Wnioski wynikające z analizy istniejących rozwiązań zostały wykorzystane podczas projektowania własnego robota, szczególnie w zakresie uproszczenia mechaniki, proporcji konstrukcji oraz rozmieszczenia napędów.

4. Uczenie przez wzmacnianie

W przypadku robota opracowywanego w niniejszej pracy zastosowanie klasycznych metod planowania ruchu jest utrudnione. Konstrukcja robota jest nietypowa ze względu na wykorzystanie prostych serwomechanizmów modelarskich, które nie umożliwiają bezpośredniego sterowania prędkością i posiadają stosunkowo niewielki moment obrotowy. Sterowanie konstrukcją ogranicza się do zadawania kolejnych pozycji kątowych stawów z określoną częstotliwością. Na rzeczywisty ruch robota wpływają dodatkowo wydajnościowe ograniczenia serwomechanizmów oraz trudne do precyzyjnego modelowania interakcje układu z otoczeniem, co znacząco utrudnia budowę dokładnego modelu dynamiki.

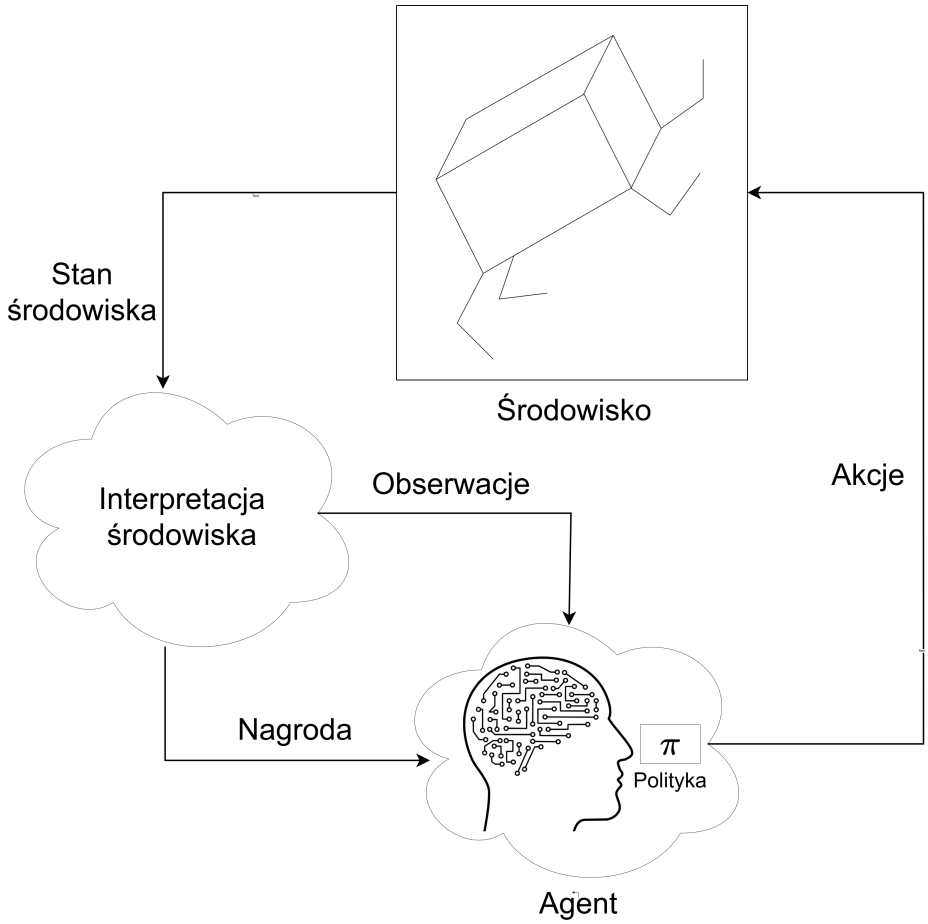
Z tego względu metody oparte na ręcznym projektowaniu trajektorii ruchu nóg lub sterowaniu wykorzystującym model matematyczny robota byłyby w praktyce trudne do zaimplementowania. Wymagałyby one znacznego uproszczenia dynamiki, co ograniczałoby jakość uzyskanego chodu. Zasadne wydaje się więc wykorzystanie technik uczenia maszynowego, wśród których można wyróżnić trzy podstawowe podejścia:

- **Uczenie nadzorowane** – algorytm uczy się na podstawie danych treningowych zawierających przykładowe wejścia oraz odpowiadające im poprawne wyjścia (etykiety). Wymaga ono dostępności zbioru danych referencyjnych, np. trajektorii eksperckich.
- **Uczenie nienadzorowane** – algorytm analizuje dane bez etykiet, koncentrując się na wykrywaniu różnego rodzaju zależności w danych.
- **Uczenie przez wzmacnianie** – algorytm nie dysponuje gotowym zbiorem danych uczących, lecz generuje doświadczenie poprzez interakcję ze środowiskiem i uczy się na podstawie otrzymywanej informacji zwrotnej.

W przypadku rozpatrywanego problemu podejścia oparte na uczeniu nadzorowanym nie są możliwe do bezpośredniego zastosowania ze względu na brak dostępnych danych referencyjnych w postaci poprawnych trajektorii ruchu robota. Z kolei uczenie nienadzorowane, mimo że pozwala na analizę i wykrycie pewnych zależności w danych, nie dostarcza mechanizmu dążenia do realizacji konkretnego celu, jakim jest efektywny chód robota.

Odpowiednim podejściem dla danego problemu jest uczenie przez wzmacnianie, które umożliwia bezpośrednią optymalizację polityki sterowania w oparciu o zdefiniowaną funkcję nagrody, bez konieczności dostarczania danych uczących. Dzięki temu możliwe jest wykształcenie strategii lokomocji w warunkach trudnych do modelowania analitycznego.

Typowy schemat działania algorytmu uczenia przez wzmacnianie przedstawiono na rysunku 4.1. W algorytmie tym można wyróżnić dwa główne elementy. Pierwszym z nich jest agent – uczący się twór, który podejmuje działania na podstawie dostępnych informacji o stanie środowiska, zgodnie z określoną polityką, oznaczaną zwykle symbolem π. Drugim elementem jest środowisko, które może przyjmować różnorodne formy i reprezentuje cały



Rysunek 4.1. Ogólny schemat działania algorytmów uczenia przez wzmacnianie

świat istotny dla agenta i zmieniający się w odpowiedzi na jego akcje. Dodatkowo często wyodrębnia się komponent zwany interpreterem, który analizuje zachowanie agenta i ocenia je według zdefiniowanej metryki. Interpreter może być również potrzebny do przetworzenia informacji o środowisku do przestrzeni stanów przyjmowanej przez agenta.

Celem agenta jest maksymalizacja nagrody skumulowanej w czasie. Inteligentny agent jest więc w stanie zrezygnować z pewnej nagrody natychmiastowej by móc otrzymać większą nagrodę w przyszłości. W trakcie procesu uczenia agent początkowo podejmuje akcje w sposób całkowicie losowy. Z czasem zaczyna kierować się wypracowaną polityką, cały czas zbierając informacje o wpływie swoich działań na środowisko – w postaci obserwacji – oraz o ich skuteczności, ocenianej za pomocą nagrody związanej z realizowanym zadaniem. Zebrane dane służą następnie do stopniowego udoskonalania polityki. Końcowym efektem pracy algorytmu powinna być polityka (zrealizowana w zdecydowanej większości przypadków, jako sieć neuronowa), która na podstawie dostępnych informacji o bieżącym stanie środowiska podejmuje odpowiednie akcje.

Istotnym ograniczeniem tego podejścia jest wpływ decyzji podejmowanych przez niewytrenowanego agenta na proces zbierania danych. W praktyce często prowadzi to do szybkiego ugrzęźnięcia algorytmu w okolicy suboptymalnej polityki, która zapewnia

jedynie umiarkowane nagrody. Wyjście z takiego minimum lokalnego może okazać się w wielu sytuacjach nawet niemożliwe.

Rozwiązaniem pozwalającym na złagodzenie tych problemów i stabilizację procesu uczenia jest zastosowanie architektury typu aktor-krytyk (ang. *actor-critic*). W podejściu tym agent składa się z dwóch oddzielnych, lecz współpracujących ze sobą modeli (najczęściej głębokich sieci neuronowych). Pierwszy z nich – aktor, na podstawie obserwacji stanu podejmuje decyzje o wyborze konkretnej akcji. Drugi element – krytyk, uczy się estymować sumę przyszłych nagród (od danej chwili do końca epizodu) będących konsekwencją podjęcia danej akcji w danym stanie. Na podstawie tej estymacji dokonuje on oceny działań podjętych przez aktora. Zamiast opierać się wyłącznie na surowej, często opóźnionej nagrodzie ze środowiska, aktor jest aktualizowany na podstawie dokładniejszej informacji zwrotnej od krytyka. Jego obecność jest niezbędna, ponieważ zdefiniowana funkcja nagrody ocenia jedynie natychmiastowy, chwilowy efekt ruchu, nie dostarczając informacji o jego długoterminowych konsekwencjach (np. o tym, czy pozornie dobry krok nie doprowadzi ostatecznie do utraty równowagi). Rozdzielenie tych ról znacząco zmniejsza wariancję podczas aktualizacji wag sieci, co przyspiesza zbieżność algorytmu i – w połączeniu z odpowiednimi mechanizmami eksploracji – ułatwia unikanie wspomnianych wcześniej minimów lokalnych.

W dalszej części tego rozdziału opisano po krótce ewolucję kolejnych algorytmów uczenia przez wzmacnianie prowadzącą do jednego z najpopularniejszych dziś algorytmów – Proximal Policy Optimization (PPO). Dodatkowo przedstawiono dwa inne algorytmy, które są powszechnie stosowane zamiennie z tym algorytmem.

4.1. Deep Q-Learning (DQN)

Deep Q-Learning jest rozwinięciem klasycznego algorytmu Q-Learning, opracowanego w 1989 roku, którego rdzeniem jest tzw. Q-tablica. Przechowuje ona zgromadzone w trakcie nauki wartości funkcji Q, które określają oczekiwany zysk z wykonania danej akcji w konkretnym stanie (oznaczane jako $Q(s,a)$). Chociaż podejście to sprawdza się w prostych środowiskach, to jego zastosowanie w bardziej złożonych zadaniach jest problematyczne. Liczba komórek tej tablicy, które należy uzupełnić zgromadzoną wiedzą będąca iloczynem rozmiaru przestrzeni akcji i przestrzeni stanów, staje się zbyt duża, czyniąc metodę niepraktyczną [24].

Deep Q-Learning rozwiązuje ten problem, zastępując wiedzę zgromadzoną w Q-tablicy siecią neuronową, która aproksymuje funkcję $Q(s,a)$, gdzie s oznacza stan środowiska, a a podjętą akcję. Dzięki temu możliwe stało się zastosowanie tego algorytmu w dużo bardziej złożonych środowiskach. Autorzy metody, testując ją na siedmiu grach z platformy Atari, wykazali, że agent uczony przy użyciu DQN przewyższał wydajnością wcześniejsze algorytmy w sześciu przypadkach, a w trzech przypadkach przewyższał także wynik ludzkiego eksperta [25].

Pomimo tego sukcesu, DQN ma istotne ograniczenia. Jednym z głównych problemów jest brak mechanizmu zapobiegającego nadmiernemu eksploatowaniu jednej strategii. Przypadkowe znalezienie jednej lub sekwencji kilku akcji zapewniających umiarkowaną, ale wyróżniającą się nagrodę może skutkować przeuczeniem sieci neuronowej i utknięciem w suboptymalnej polityce[26]. Kolejne algorytmy uczenia przez wzmacnianie koncentrowały się w dużej mierze na rozwiązaniu tego problemu. Warto jednak podkreślić, że DQN wciąż znajduje zastosowanie, szczególnie tam, gdzie istotna jest wysoka efektywność wykorzystania danych, co pozostaje jego istotną zaletą [27].

4.2. Trust Region Policy Optimization

Jednym z algorytmów stanowiących odpowiedź na problem niestabilności procesu uczenia występujący, między innymi, w algorytmie DQN jest Trust Region Policy Optimization (TRPO), zaproponowany w 2015 roku. Jego główną zaletą jest bardziej stabilne i kontrolowane podejście do optymalizacji polityki decyzyjnej [28].

Główna idea TRPO polega na ograniczaniu maksymalnej wielkości pojedynczej aktualizacji polityki. By uniknąć zbyt dużych „skoków" polityki TRPO narzuca warunek na maksymalne dopuszczalne przesunięcie nowej polityki względem starej, mierzone za pomocą średniej dywergencji Kullbacka-Leiblera (ang. *Kullback-Leibler divergence*) pomiędzy tymi politykami. Rysunek 4.2 przedstawia poglądową różnicę pomiędzy podejściem do optymalizacji polityki w algorytmach DQN i TRPO. W algorytmie DQN aktualizacje mogą prowadzić do dużych zmian strategii, prowadzących do potencjalnie złych polityk, podczas gdy TRPO ogranicza ich wielkość poprzez wprowadzenie obszaru zaufania, co skutkuje bardziej stabilnym procesem uczenia.

Rysunek 4.2. Poglądowe porównanie sposobu aktualizacji polityki w algorytmach DQN (lewa strona) i TRPO (prawa strona)

Dzięki wprowadzeniu ograniczenia obszaru zaufania (ang. *trust region*), TRPO umożliwia wolniejsze, ale znacznie bardziej stabilne uczenie się optymalnej polityki. Badania empiryczne wykazały, że algorytm ten osiąga bardzo dobre rezultaty w zadaniach sterowania ruchem robotów zarówno dwu-, jak i czworonożnych [29]. Choć wyniki te były bardzo

konkurencyjne w momencie publikacji, nowsze algorytmy, często osiągają porównywalne lub lepsze rezultaty przy niższym koszcie obliczeniowym.

Mimo swoich zalet, TRPO ma również istotne wady – najpoważniejszą z nich jest wysoka złożoność obliczeniowa. W trakcie treningu konieczne jest rozwiązywanie zadania optymalizacji z ograniczeniami, co wiąże się m.in. z wykorzystaniem aproksymacji macierzy pochodnych drugiego rzędu (np. macierzy Fishera) oraz rozwiązywaniem problemu optymalizacji metodą gradientów sprzężonych. Dodatkowymi trudnościami są złożoność implementacyjna algorytmu oraz jego wrażliwość na dobór hiperparametrów. Z tych powodów TRPO zostało w dużej mierze wyparte przez nowszy algorytm – Proximal Policy Optimization – który również trzyma się zasady ograniczania aktualizacji polityki, jednak robi to w sposób prostszy i znacznie bardziej efektywny obliczeniowo.

4.3. Proximal Policy Optimization

Algorytm Proximal Policy Optimization, zaproponowany przez firmę OpenAI w 2017 roku, stanowi uproszczoną i bardziej praktyczną alternatywę dla algorytmu TRPO[16]. Algorytm ten bardzo szybko zdobył popularność i do dziś pozostaje jedną z najczęściej stosowanych i najbardziej uniwersalnych metod dla wielu klas problemów uczenia przez wzmacnianie, szczególnie w zastosowaniach umożliwiających równoległe zbieranie danych przez wiele instancji środowiska. Doczekał się również licznych udanych implementacji, z których najbardziej medialnym przykładem było pokonanie przez agenta uczonego tym algorytmem drużyny mistrzów świata w grze komputerowej *Dota 2* [30]. PPO można interpretować jako uproszczoną, heurystyczną aproksymację podejścia TRPO, która rezygnuje z jawnego ograniczenia dywergencji Kullbacka-Leiblera na rzecz łatwiejszej w implementacji funkcji celu.

By lepiej zrozumieć algorytm PPO warto przyjrzeć się bliżej jego funkcji straty. Całkowita strata minimalizowana w algorytmie PPO $\mathscr{L}$ wygląda następująco:

$$\mathscr{L}(\theta) = -L^{\text{CLIP}}(\theta) + c_1 \cdot VF_{\text{loss}} - c_2 \cdot H(\pi_\theta) \tag{1}$$

gdzie θ oznacza zbiór parametrów sieci neuronowej reprezentującej politykę π_θ. Składnik $L^{\text{CLIP}}(\theta)$ jest funkcją celu PPO odpowiedzialną za aktualizację polityki przy jednoczesnym ograniczeniu zbyt dużych zmian jej parametrów. Człon VF_{loss} (Value function loss) określa błąd funkcji wartości, mierzący różnicę pomiędzy przewidywaną a rzeczywistą wartością nagrody oczekiwanej w danym stanie. Współczynnik c_1 kontroluje wpływ błędu funkcji wartości na całkowitą stratę. Entropia polityki $H(\pi_\theta)$ stanowi miarę losowości podejmowanych decyzji i wspiera eksplorację środowiska. Współczynnik c_2 określa znaczenie składnika entropii podczas uczenia. Minimalizacja funkcji $\mathscr{L}(\theta)$ prowadzi więc do jednoczesnej poprawy polityki, dokładności funkcji wartości oraz utrzymania odpowiedniego poziomu eksploracji.

Należy zaznaczyć, że funkcja straty nie jest obliczana po wykonaniu każdego kroku w środowisku. Algorytm PPO gromadzi dane z określonej liczby epizodów lub kroków, a dopiero następnie wykonuje aktualizację parametrów. Oznacza to, że w trakcie liczenia straty znane są już nagrody będące konsekwencją wykonania poszczególnych akcji.

Najważniejszym elementem straty jest $L^{\text{CLIP}}(\theta)$, pomnożony przez -1 co oznacza, że celem optymalizatora powinno być maksymalizowanie jego wartości. Dokładny wzór opisujący tę część straty przedstawiony jest poniżej:

$$L^{\text{CLIP}}(\theta) = \min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right) \tag{2}$$

gdzie clip$(\cdot)$ oznacza funkcję przycinania ograniczającą wartość $r_t(\theta)$ do przedziału $[1-\epsilon, 1+\epsilon]$, dla wartości stałej ϵ przyjmowanej standardowo jako $\epsilon = 0{,}2$. By zrozumieć dlaczego strata ta wygląda w taki a nie inny sposób należy najpierw wyjaśnić czym są poszczególne jej składniki:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$
$$\hat{A}_t = R_t + \gamma V(s_{t+1}) - V(s_t)$$

gdzie:

- $r_t(\theta)$ – stosunek prawdopodobieństwa akcji a_t w chwili t znajdując się w stanie s_t dla nowej (π_θ) i starej $(\pi_{\theta_{\text{old}}})$ polityki. Wartość większa od 1 oznacza, że akcja ta stała się bardziej prawdopodobna, natomiast wartość mniejsza od 1 oznacza, że prawdopodobieństwo podjęcia tej akcji się zmniejszyło.
- $\hat{A}_t$ – estymacja przewagi (ang. *advantage estimation*) określa, jak korzystne było wykonanie danej akcji w stanie s_t względem polityki bazowej (funkcji wartości). W praktyce stanowi ona różnicę pomiędzy skumulowaną nagrodą uwzględniającą przyszłe zwroty a wartością oczekiwaną estymowaną przez funkcję wartości. $R_t + \gamma V(s_{t+1})$ jest sumą nagrody uzyskanej w chwili t oraz estymacji funkcji wartości dla kolejnego stanu s_{t+1} i stanowi estymację zwrotu od kroku t. Odejmowana od niej wartość $V(s_t)$ jest estymacją funkcji wartości w chwili t. Wartość γ we wzorze jest współczynnikiem dyskontującym, określającym znaczenie przyszłych nagród względem nagród natychmiastowych. Dodatnia wartość $\hat{A}_t$ oznacza, że wykonana akcja doprowadziła do wyniku lepszego niż funkcja wartości oczekiwała w stanie s_t, natomiast wartość ujemna oznacza wynik gorszy od przewidywanego.

Należy jednak pamiętać, że $V(s)$ jest jedynie estymacją nagrody realizowaną przez sieć neuronową trenowaną dla klasycznego zadania uczenia nadzorowanego, więc nie zawsze, a w szczególności na początku procesu uczenia, wartości przez nią zwracane są prawidłowe. Odpowiednie nauczenie się przez sieć krytyka przewidywania oczekiwanej

wartości nagrody jest jednak warunkiem koniecznym nauczenia się przez agenta sprawnej polityki.

Zakres wartości zarówno $r_t(\theta)$ jak i $\hat{A}_t$ dzieli się więc na dwa przedziały, co pozwala wyodrębnić cztery przypadki zachowania się straty przedstawionej w równaniu (2). Interpretacje poszczególnych przypadków przedstawione są w tabeli 4.1.

$r_t(\theta)$	$\hat{A}_t$	Interpretacja
$r_t(\theta) > 1$	$\hat{A}_t > 0$	Nowa polityka zwiększa prawdopodobieństwo korzystnej akcji - sytuacja pożądana, jednak by uniknąć nadmiernej aktualizacji polityki maksymalna strata jest ograniczona do $r_t(\theta) \cdot (1+\epsilon) \cdot \hat{A}_t$.
$r_t(\theta) < 1$	$\hat{A}_t > 0$	Nowa polityka zmniejsza prawdopodobieństwo korzystnej akcji - strata nie jest ograniczana.
$r_t(\theta) > 1$	$\hat{A}_t < 0$	Nowa polityka zwiększa prawdopodobieństwo niekorzystnej akcji - strata nie jest ograniczana.
$r_t(\theta) < 1$	$\hat{A}_t < 0$	Nowa polityka zmniejsza prawdopodobieństwo niekorzystnej akcji - sytuacja poządana, jednak by uniknąć nadmiernej aktualizacji polityki maksymalna strata jest ograniczona do $r_t(\theta) \cdot (1-\epsilon) \cdot \hat{A}_t$.

Tabela 4.1. Interpretacja działania algorytmu PPO dla różnych relacji pomiędzy $r_t(\theta)$ a $\hat{A}_t$. Analizując tabelę należy pamiętać, że w metodach optymalizacji gradientowe zmiana wag sieci jest proporcjonalna do gradientu funkcji straty. Nasycenie się wartości straty blokuje niekontrolowaną zmianę wartości wag.

Podsumowując, algorytm PPO – podobnie jak jego poprzednik, algorytm TRPO – dąży do ograniczenia ryzyka niestabilnych zmian polityki poprzez kontrolowanie rozmiaru jej aktualizacji. W przeciwieństwie do TRPO, zamiast rozwiązywać złożone zadanie optymalizacji z ograniczeniami, PPO osiąga ten cel poprzez optymalizację sprytnie sformułowanej funkcji celu, co czyni go znacznie prostszym i bardziej praktycznym w użyciu. Istotną cechą PPO jest również wykonywanie wielu iteracji optymalizacji na tych samych danych, co zwiększa efektywność uczenia.

Główną wadą algorytmu PPO jest jednak stosunkowo niska efektywność wykorzystania danych zbieranych w trakcie uczenia. Choć algorytm wykonuje kilka iteracji optymalizacji na tej samej partii doświadczeń, po ich zakończeniu dane są odrzucane, a agent musi zebrać nową próbkę danych zebranych w trakcie interakcji ze środowiskiem. Dodatkowo mechanizm przycinania ogranicza skalę aktualizacji polityki, przez co uzyskiwane zmiany względem kosztu zebrania danych są relatywnie niewielkie. Z tego względu PPO najlepiej sprawdza się w zastosowaniach, w których pozyskiwanie danych jest szybkie, tanie oraz możliwe do zrównoleglenia z wykorzystaniem wielu agentów.

4.4. Advantage Actor-Critic

Algorytm Advantage Actor-Critic (A2C) stanowi uproszczoną, synchroniczną wersję algorytmu Asynchronous Advantage Actor Critic (A3C) [15]. Podobnie jak algorytm PPO wykorzystuje on dwa współpracujące modele: aktora, odpowiedzialnego za dobór akcji, oraz krytyka, który ocenia jakość podejmowanych decyzji poprzez estymację funkcji wartości. W porównaniu do PPO, A2C nie stosuje jednak mechanizmów ograniczających rozmiar aktualizacji polityki. Z tego względu jest on metodą prostszą obliczeniowo i łatwiejszą w implementacji, lecz jednocześnie bardziej podatną na niestabilność uczenia. Pomimo swojej prostoty, A2C jest obecnie rzadziej wykorzystywany w bardziej wymagających zadaniach, gdzie często ustępuje nowszym algorytmom, takim jak PPO, które oferują lepszą kontrolę nad procesem aktualizacji polityki. W pewnych środowiskach możliwe jest jednak uzyskanie przy użyciu A2C wyników porównywalnych, a czasem nawet lepszych niż w przypadku PPO, przy jednocześnie niższym koszcie obliczeniowym.

4.5. Soft Actor-Critic

Algorytm Soft Actor-Critic (SAC) jest nowoczesną metodą uczenia przez wzmacnianie należącą do klasy algorytmów aktor-krytyk. Algorytm różni się jednak w sposób istotny od przedstawionych wcześniej algorytmów TRPO, PPO oraz A2C, które należą do metod on-policy, wymagających generowania nowych danych treningowych dla każdej aktualizacji polityki. SAC natomiast jest algorytmem off-policy, wykorzystującym bufor doświadczeń (ang. *replay buffer*), co pozwala na wielokrotne wykorzystanie tych samych próbek i znacząco zwiększa efektywność uczenia.

Kluczową cechą SAC jest bezpośrednie uwzględnienie czynnika odpowiedzialnego za maksymalizację entropii polityki w optymalizowanej funkcji straty. Oznacza to, że oprócz maksymalizacji oczekiwanej nagrody, algorytm dąży również do zwiększania entropii polityki, a więc utrzymywania odpowiedniego poziomu losowości podejmowanych decyzji. W praktyce prowadzi to do lepszej eksploracji przestrzeni stanów oraz zmniejszenia ryzyka utknięcia w minimum lokalnym polityki.

Eksperymenty przeprowadzone przez autorów algorytmu wykazały wysoką stabilność działania, dużą efektywność próbkowania oraz bardzo dobre wyniki w zadaniach sterowania ciągłego, takich jak *HalfCheetah-v1*, *Walker2d-v1* czy *Ant-v1*[17], będących standardowymi środowiskami testowymi, wykorzystywanymi w literaturze do porównywania algorytmów uczenia przez wzmacnianie. Wyniki te wskazują, że SAC może przewyższać pod względem skuteczności klasyczne metody on-policy, takie jak PPO, szczególnie w środowiskach, w których pozyskiwanie danych jest kosztowne lub czasochłonne.

4.6. Podsumowanie

Warto podkreślić istotną różnicę pomiędzy omówionym wcześniej algorytmem DQN a algorytmami TRPO, PPO, A2C i SAC. W przypadku DQN sieć neuronowa wykorzystywana jest do estymacji funkcji $Q(s,a)$, natomiast polityka wyznaczana jest pośrednio poprzez wybór akcji maksymalizującej wartość $Q(s,a)$ w stanie s. Z kolei w algorytmach TRPO, PPO, A2C i SAC sieć neuronowa reprezentuje bezpośrednio politykę agenta. Oznacza to, że optymalizacja funkcji celu prowadzi bezpośrednio do aktualizacji sposobu podejmowania decyzji przez agenta. Algorytmy tego typu określane są mianem metod *policy gradient*, ponieważ uczą się polityki bezpośrednio na podstawie gradientu oczekiwanej nagrody względem parametrów sieci. Tabela 4.2 przedstawia zestawienie najważniejszych różnic pomiędzy algorytmami PPO, A2C oraz SAC testowanymi w ramach niniejszej pracy.

	A2C	PPO	SAC
Typ uczenia	On-policy	On-policy	Off-policy
Efektywność próbkowania	Niska	Średnia	Wysoka
Stabilność uczenia	Średnia	Wysoka	Średnia/Wysoka
Mechanizm eksploracji	Entropia w funkcji straty	Entropia w funkcji straty	Maksymalizacja entropii
Mechanizm stabilizacji nauki	Brak	Przycinanie	Brak (stabilność wynikająca z maksymalizacji entropii)
Złożoność obliczeniowa	Niska	Umiarkowana	Wysoka
Rok publikacji	2016	2017	2018

Tabela 4.2. Porównanie kluczowych cech algorytmów A2C, PPO oraz SAC

5. Konstrukcja robota

Celem tego rozdziału jest szczegółowe przedstawienie architektury robota kroczącego opracowanego w ramach niniejszej pracy. Opisywana konstrukcja stanowi czworonożną platformę badawczą, zaprojektowaną z myślą o implementacji wyników uzyskanych w procesie uczenia przez wzmacnianie, przeprowadzonego w środowisku symulacyjnym.

W rozdziale przedstawiono również uzasadnienie przyjętych założeń projektowych oraz pokrótce omówiono ewolucję projektu, która doprowadziła do ostatecznego kształtu konstrukcji. Szczególną uwagę poświęcono kompromisom pomiędzy złożonością mechaniczną robota a wymaganiami stawianymi przez algorytmy uczenia. W kolejnych podrozdziałach szczegółowo opisano architekturę mechaniczną, zastosowany układ napędowy, autorską płytkę sterującą oraz model kinematyczny robota.

5.1. Budowa robota

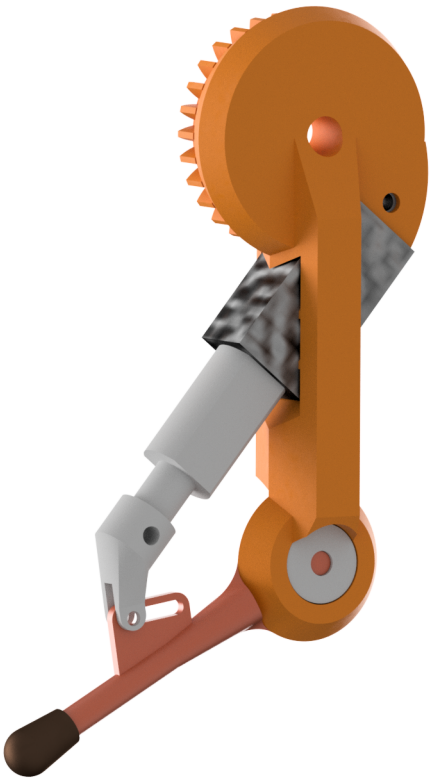
Konstrukcja robota została zaprojektowana jako układ symetryczny, wyposażony w cztery identyczne kończyny. Każda z nóg posiada dwa stopnie swobody, realizowane za pomocą przegubów odpowiadających ruchowi w stawie biodrowym oraz kolanowym. Przyjęta liczba stopni swobody stanowi kompromis pomiędzy zdolnością do generowania złożonych trajektorii ruchu a stopniem skomplikowania konstrukcji mechanicznej oraz modelu sterowania. Ograniczenie liczby stopni swobody upraszcza zarówno proces projektowania i budowy robota, jak i zmniejsza wymiar przestrzeni stanów i akcji, co ma istotne znaczenie w kontekście efektywności uczenia przez wzmacnianie.

Główna struktura nośna robota została wykonana z elementów zaprojektowanych przez autora i wytworzonych przy użyciu technologii druku 3D. W konstrukcji wykorzystano materiały PLA oraz PET-G, które zapewniają odpowiednią wytrzymałość mechaniczną elementów przy niewielkiej masie i są powszechnie stosowane w tego typu konstrukcjach.

5.1.1. Noga

Proces projektowania nogi robota obejmował analizę wielu wariantów konstrukcyjnych różniących się zarówno sposobem realizacji napędu, jak i rozwiązaniami mechanicznymi zastosowanymi w obrębie stawu kolanowego. Poszczególne koncepcje były kolejno weryfikowane pod kątem przewidywanego wpływu na zdolność robota do wyuczenia skutecznej strategii chodu.

Pierwszym rozważanym rozwiązaniem była konstrukcja wykorzystująca siłownik liniowy do realizacji ruchu w stawie kolanowym. Wizualizacja tej koncepcji przedstawiona jest na rysunku 5.1. W takim układzie staw mógł przyjmować jedynie dwa skrajne położenia: odpowiadające wysunięciu lub wsunięciu siłownika. Główną zaletą tego podejścia była możliwość generowania dużego momentu obrotowego w stawie kolanowym. Innymi zaletami były prostota mechaniczna oraz łatwość sterowania upraszczająca zadanie z



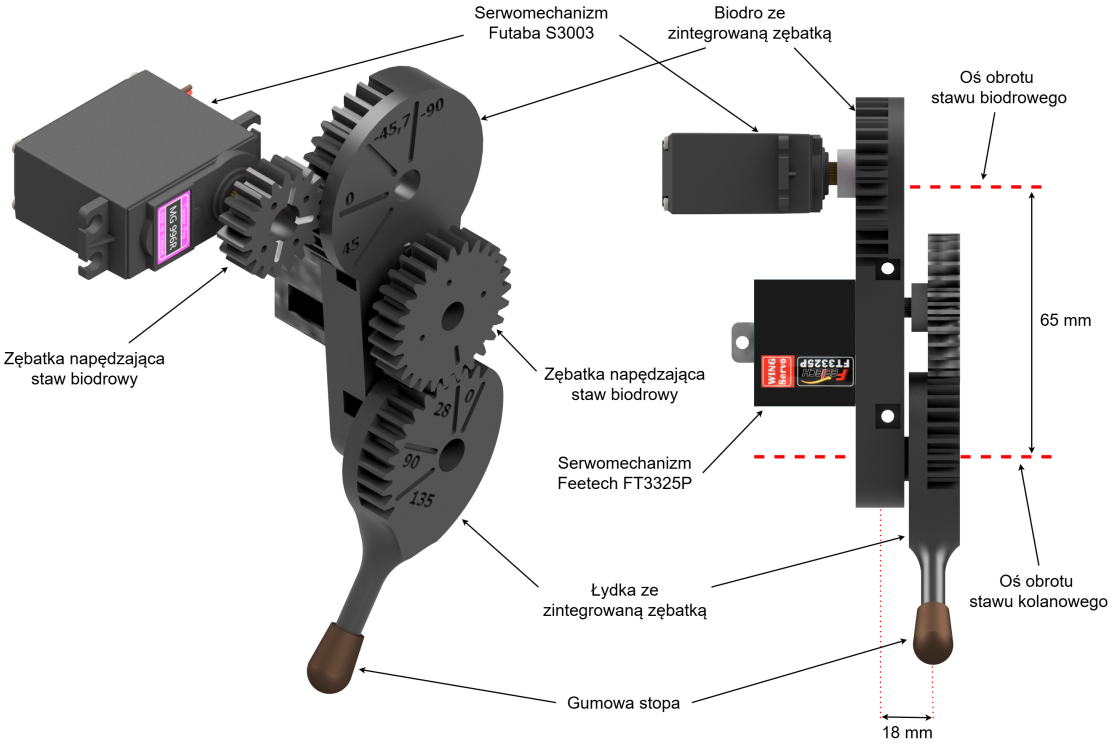
Rysunek 5.1. Wizualizacja jednej z wcześniejszych koncepcji napędu nogi robota, wykorzystująca siłownik liniowy do poruszania stawem kolanowym

problemu wyznaczenia 8 liczb ciągłych do problemu wyznaczenia 4 liczb ciągłych i 4 wartości binarnych. Przeprowadzone testy symulacyjne wykazały jednak, że binarny charakter pozycji stawu kolanowego znacząco utrudniał uzyskanie płynnego i efektywnego chodu. Dodatkowym problemem okazała się zbyt mała prędkość ruchu siłownika i w konsekwencji – niewystarczająca prędkość kątowa stawu kolanowego. Uzyskiwane strategie lokomocji nie spełniały założeń projektu, dlatego z rozwiązania tego zrezygnowano.

Kolejna analizowana koncepcja opierała się na wykorzystaniu mechanizmu dźwigniowego do przeniesienia napędu z serwomechanizmu na staw kolanowy. Zastosowanie odpowiednio dobranych ramion dźwigni umożliwiało zwiększenie efektywnego momentu obrotowego działającego na łydkę, jednak odbywało się kosztem większej liczby elementów oraz wyższej złożoności konstrukcji. Po przeprowadzeniu analizy stwierdzono, że podobny efekt można uzyskać przy użyciu prostszego i bardziej zwartego mechanizmu przekładni zębatej.

Ostatecznie zdecydowano się więc na rozwiązanie wykorzystujące dwa niezależnie sterowane serwomechanizmy oraz przekładnie zębate zintegrowane z segmentami nogi. Na rysunku 5.2 przedstawiono wizualizację tego elementu konstrukcji. Pierwszy staw,

zlokalizowany w miejscu połączenia nogi z korpusem, pełni funkcję stawu biodrowego. Napęd tego stawu realizowany jest za pomocą serwomechanizmu umieszczonego w korpusie robota, który przenosi moment obrotowy poprzez zintegrowany z segmentem nogi profil koła zębatego.



Rysunek 5.2. Wizualizacja nogi robota w ostatecznej formie

Drugi staw odpowiada za podnoszenie i opuszczanie łydki, pełniąc funkcję stawu kolanowego. Jego napęd realizowany jest przez serwomechanizm zamocowany w pierwszym segmencie nogi, również z wykorzystaniem mechanizmu przekładni zębatej. Takie rozwiązanie ma tę zaletę, że położenie stawu kolanowego jest niezależne od aktualnego kąta ustawienia stawu biodrowego, co upraszcza sterowanie oraz analizę ruchu.

Odległość pomiędzy osiami obrotu obydwu stawów wynosi 65 mm. Dodatkowo płaszczyzny w których obracają się stawy są od siebie odsunięte o 18 mm. Rozwiązanie to wynika z uproszczeń konstrukcyjnych i zostało uwzględnione w modelu symulacyjnym robota. Przyjęte przesunięcie nie ma istotnego wpływu na dynamikę robota.

W celu zapewnienia płynności ruchu oraz poprawnego przenoszenia obciążeń, w przegubach zastosowano łożyska kulkowe. W stawach kolanowych wykorzystano łożyska o średnicy zewnętrznej 13 mm, średnicy wewnętrznej 5 mm, grubości 5 mm i masie 2,8 g, które dobrano ze względu na niewielkie gabaryty, małą masę oraz niski koszt. W stawach biodrowych, w celu zapewnienia lepszej stabilności nogi, zdecydowano się na grubsze łożyska o grubości 7 mm, średnicy zewnętrznej 22 mm, średnicy wewnętrznej 6 mm i masie 13 g. Zastosowanie łożysk o większych gabarytach pozwoliło praktycznie całkowicie

wyeliminować, obserwowany wcześniej, problem odchylania się nóg od korpusu w trakcie chodu.

Segmenty nóg zostały zaprojektowane z uwzględnieniem minimalizacji masy przy jednoczesnym zachowaniu odpowiedniej sztywności konstrukcji. Geometria elementów została dobrana w sposób umożliwiający uzyskanie wymaganego zakresu ruchu oraz uniknięcie kolizji pomiędzy elementami podczas pracy robota.

5.1.2. Korpus

Korpus robota stanowi główny element nośny całej konstrukcji, do którego mocowane są kończyny oraz komponenty układu napędowego i sterowania. Na rysunku 5.3, przedstawiającym wizualizację robota, przedstawiony jest korpus wraz z rozmieszczeniem poszczególnych elementów robota.

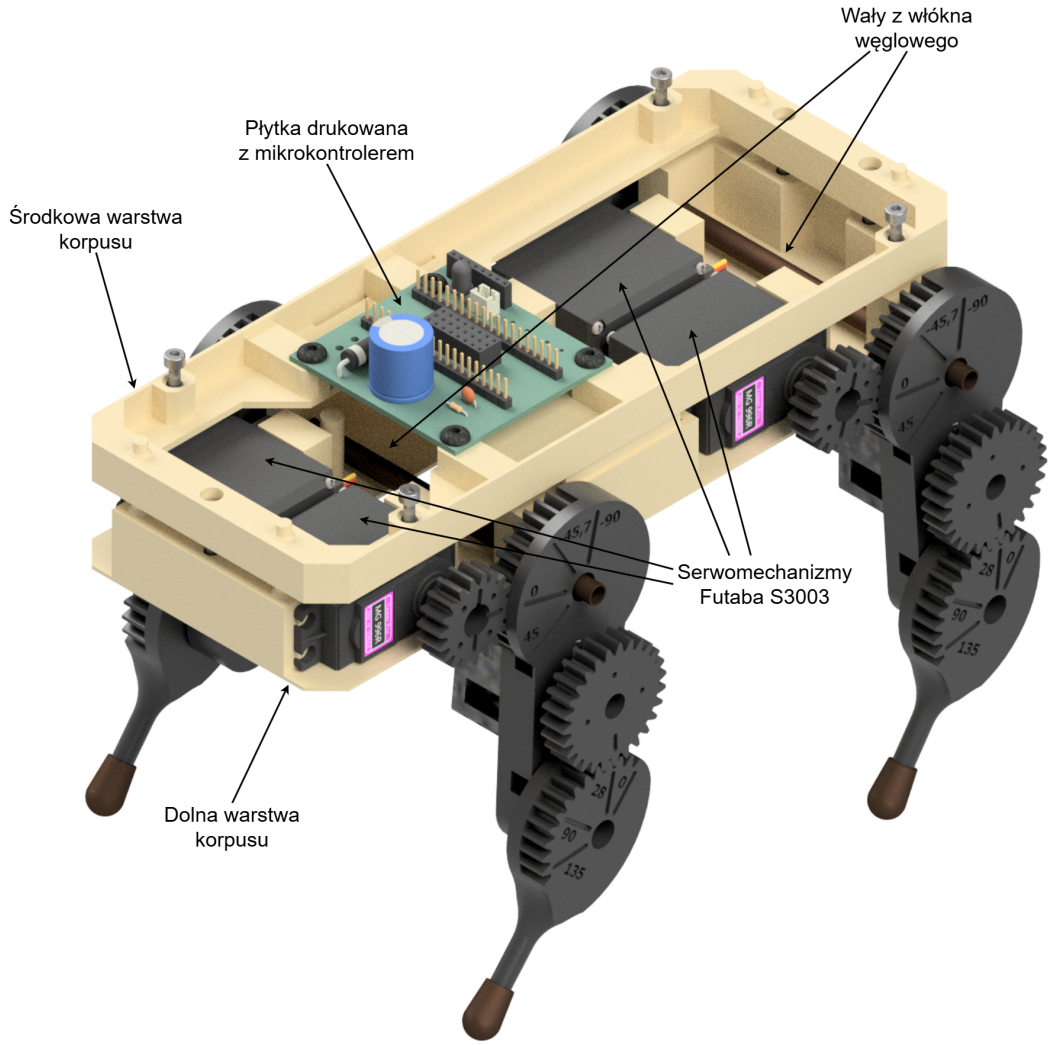
Konstrukcja korpusu została zaprojektowana jako bryła składająca się z trzech oddzielnych części. Pierwszą z nich jest podstawa, do której mocowane są serwomechanizmy. Drugą stanowi środkowy segment przeznaczony do montażu płytki sterującej robota. Elementy te przedstawiono na rysunku 5.3 – zostały one celowo rozsunięte, aby uwidocznić ich wzajemne położenie oraz sposób podziału konstrukcji. Trzecim elementem jest pokrywa, nieprzedstawiona na rysunku 5.3, wyposażona w dwa otwory umożliwiające montaż przycisków służących odpowiednio do włączania zasilania oraz zmiany trybu pracy robota.

Korpus został wykonany z elementów zaprojektowanych przez autora i wytworzonych przy użyciu technologii druku 3D, z wykorzystaniem filamentu PLA. Poszczególne elementy połączono za pomocą śrub mocowanych do insertów umieszczonych w specjalnie przygotowanych otworach, co umożliwia wielokrotny montaż i demontaż konstrukcji. Przez całą szerokość korpusu przepuszczono dodatkowo dwa przymocowane do niego wały z włókna węglowego które stanowią osie obrotu nóg robota. Zdecydowano się na wykorzystanie pustych w środku wałów wykonanych z włókna węglowego, ponieważ zapewniają one bardzo dobrą wytrzymałość i sztywność przy znikomej masie nieprzekraczającej 2 g.

Wymiary i rozmieszczenie komponentów wewnątrz korpusu zostało dobrane na podstawie wymagań konstrukcyjnych. Szerokość robota wynika przede wszystkim z konieczności umieszczenia wewnątrz dwóch serwomechanizmów, natomiast długość została dobrana na podstawie optymalnej odległości pomiędzy przednią a tylną parą nóg, zapewniającej stabilny chód robota. Serwomechanizmy, stanowiące najcięższy element konstrukcji, zostały umieszczone możliwie najniżej, co pozytywnie wpływa na stabilność chodu robota.

5.1.3. Układ sterowania i płytka PCB

Układ sterowania robota, przedstawiony na rysunku 5.4 stanowi zintegrowany system, obejmujący mikrokontroler oraz zaprojektowaną na jego potrzeby płytkę drukowaną

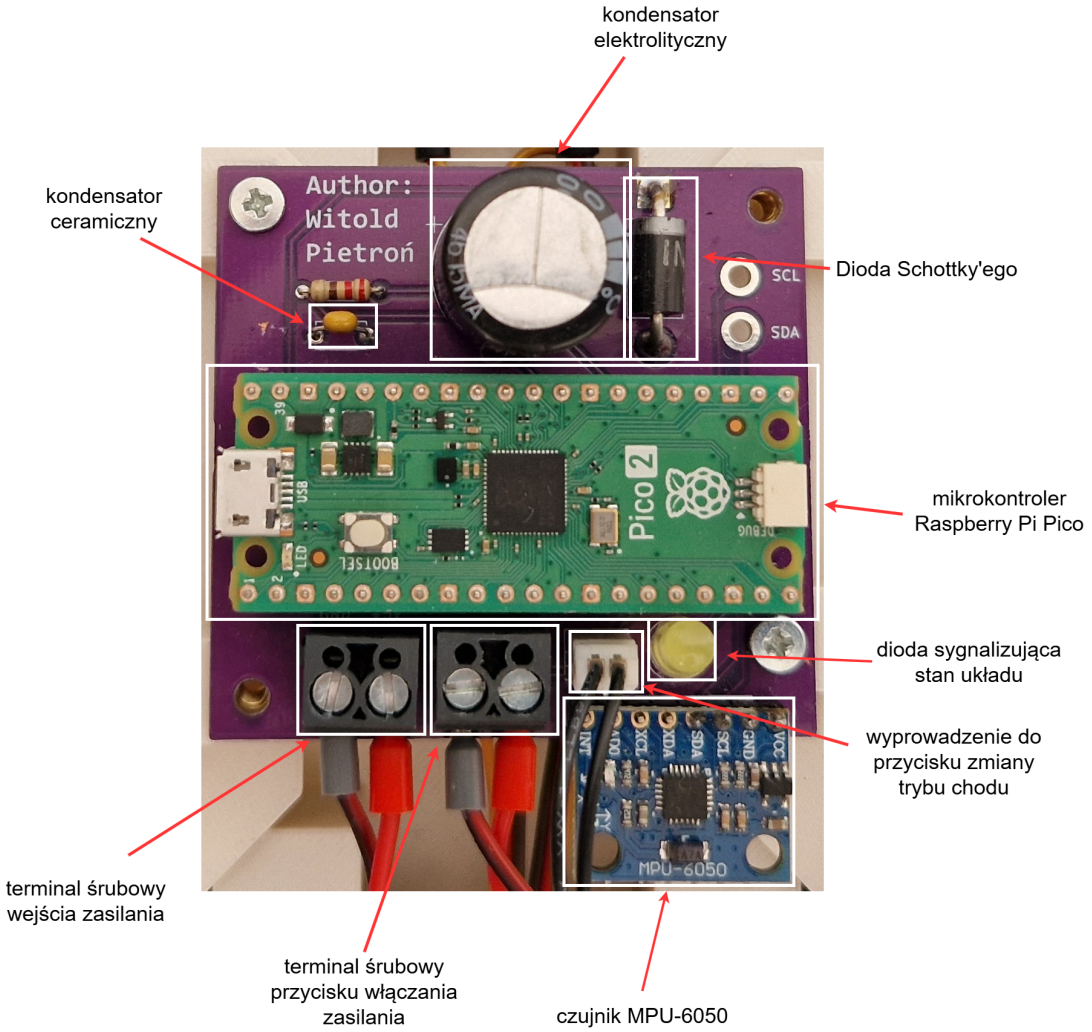


Rysunek 5.3. Wizualizacja modelu robota bez górnej pokrywy

(PCB). Odpowiada on za generowanie trajektorii ruchu chodu, wysyłanie sygnałów sterujących dla serwomechanizmów oraz obsługę czujnika inercyjnego. Centralnym elementem układu jest mikrokontroler typu Raspberry Pi Pico, który został wybrany ze względu na niewielkie wymiary, łatwość obsługi (w tym zasilania), prostotę programowania, niską cenę oraz prostotę obsługi peryferiów w postaci serwomechanizmów.

W celu integracji wszystkich komponentów zaprojektowano dedykowaną płytkę PCB, która pełni rolę układu pośredniczącego pomiędzy mikrokontrolerem, napędami, czujnikiem inercyjnym oraz elementami zasilania. Płytka umożliwia uporządkowane prowadzenie połączeń pomiędzy Raspberry Pi Pico a serwomechanizmami, a także uporządkowane rozprowadzenie napięcia zasilającego do wszystkich elementów układu.

Zasilanie układu realizowane jest poprzez terminal śrubowy, do którego doprowadzane jest napięcie z zewnętrznego źródła. Takie rozwiązanie zapewnia pewne i trwałe połączenie, a jednocześnie umożliwia łatwy montaż oraz demontaż układu. W torze zasilania mikrokontrolera, przed odpowiednim pinem wejściowym, zastosowano diodę Schot-



Rysunek 5.4. Zdjęcie układu sterowania złożonego z mikrokontrolera i dedykowanej płytki PCB

tky'ego zgodnie z zaleceniami dokumentacji. Jej zadaniem jest zapobieganie przepływowi prądu wstecznego do źródła.

W celu stabilizacji napięcia oraz ograniczenia zakłóceń generowanych przez pracujące serwomechanizmy zastosowano kondensator elektrolityczny o pojemności 4700 µF oraz kondensator ceramiczny o pojemności 100 nF. Kondensator elektrolityczny tłumi spadki napięcia niskiej częstotliwości, natomiast kondensator ceramiczny skutecznie redukuje zakłócenia wysokoczęstotliwościowe, co zapewnia stabilną zasilanie układu.

Sygnały sterujące przekazywane są do serwomechanizmów za pomocą złącza kołowego w konfiguracji 3×8, umożliwiającego jednoczesne podłączenie ośmiu napędów. Każde ze złączy zapewnia dostęp do linii zasilania, masy oraz sygnału sterującego PWM. Takie rozwiązanie upraszcza prowadzenie przewodów i montaż całego układu, jednocześnie zwiększając niezawodność połączeń.

Na płytce sterującej umieszczono również czujnik inercyjny MPU-6050, umożliwiający

pomiar przyspieszeń oraz prędkości kątowych. Dane z czujnika wykorzystywane są do estymacji orientacji robota w osi przechyłu i pochyłu.

Płytka została wyposażona w elementy interfejsu użytkownika: przycisk zasilania, umożliwiający włączanie i wyłączanie robota oraz przełącznik suwakowy, pozwalający na zmianę trybu chodu. Oba elementy zamontowano w tylnej części pokrywy robota. Dodatkowo na płytce znajduje się dioda LED, służąca do sygnalizacji aktualnego stanu robota.

W trakcie testów układu zaobserwowano problem nieprawidłowej detekcji stanu logicznego na pinie GPIO21 mikrokontrolera. Analiza wykazała, że przyczyną była niewystarczająca stabilizacja stanu niskiego na wejściu GPIO – wewnętrzny rezystor pull-down okazał się zbyt słaby, co powodowało niejednoznaczną interpretację stanu wejścia. Problem rozwiązano poprzez zastosowanie zewnętrznego rezystora pull-down o wartości 10 kΩ, podłączonego między GPIO21 a masę układu, co zapewniło poprawne i stabilne odczyty stanu przycisku.

5.2. Układ napędowy

Układ napędowy robota został oparty na serwomechanizmach dobranych indywidualnie do wymagań poszczególnych stawów. Głównymi kryteriami doboru były: generowany moment obrotowy, masa napędu, wymiary oraz koszt zastosowanego rozwiązania. Zestawienie wybranych parametrów serwomechanizmów wykorzystanych w projekcie przedstawione jest w tabeli 5.1.

Serwomechanizm	Feetech FT3325P	Futaba S3003
Napędzany staw	kolanowy	biodrowy
Moment obrotowy (6 V)	0,71 Nm	0,40 Nm
Wymiary	30,0 × 10,0 × 35,5 mm	40 × 20 × 36 mm
Masa	44 g	24 g
Cena jednostkowa	56,90 zł	17,89 zł

Tabela 5.1. Zestawienie wybranych parametrów serwomechanizmów Feetech FT3325P oraz Futaba S3003.

W stawie kolanowym zastosowano serwomechanizm typu mikro Feetech FT3325P. Wybór tego modelu wynikał z konieczności zainstalowania serwomechanizmu o wysokim momencie obrotowym w bardzo ograniczonej przestrzeni montażowej. Zastosowany serwomechanizm o deklarowanym momencie obrotowym 0,71 Nm przy napięciu zasilania 6V i niewielkich gabarytach bardzo dobrze spełnił te założenia.

Do napędu stawu biodrowego wykorzystano serwomechanizm Futaba S3003 o deklarowanym momencie obrotowym 0,40 Nm przy napięciu zasilania 6V. W tym przypadku dostępna przestrzeń montażowa była większa, dlatego możliwe było zastosowanie większego i tańszego napędu. Rozważano również użycie tego samego modelu mikroserwa co

w stawie kolanowym, jednak jego koszt był znacząco wyższy, a większe gabaryty w obrębie korpusu robota nie stanowiły istotnego ograniczenia konstrukcyjnego.

Rysunek 5.5. Porównanie serwomechanizmów Futaba S3003 oraz Feetech FT3325P

Na etapie projektowania testowano także inne serwomechanizmy o zbliżonych wymiarach do zastosowanego w biodrze, lecz charakteryzujące się większą masą i momentem obrotowym wynoszącym około 0,80 Nm. Ostatecznie zdecydowano się na lżejszy model, którego moment obrotowy okazał się wystarczający do realizacji założonych ruchów robota. Pozwoliło to ograniczyć całkowitą masę konstrukcji oraz zmniejszyć pobór prądu podczas pracy.

5.3. Podsumowanie parametrów fizycznych robota

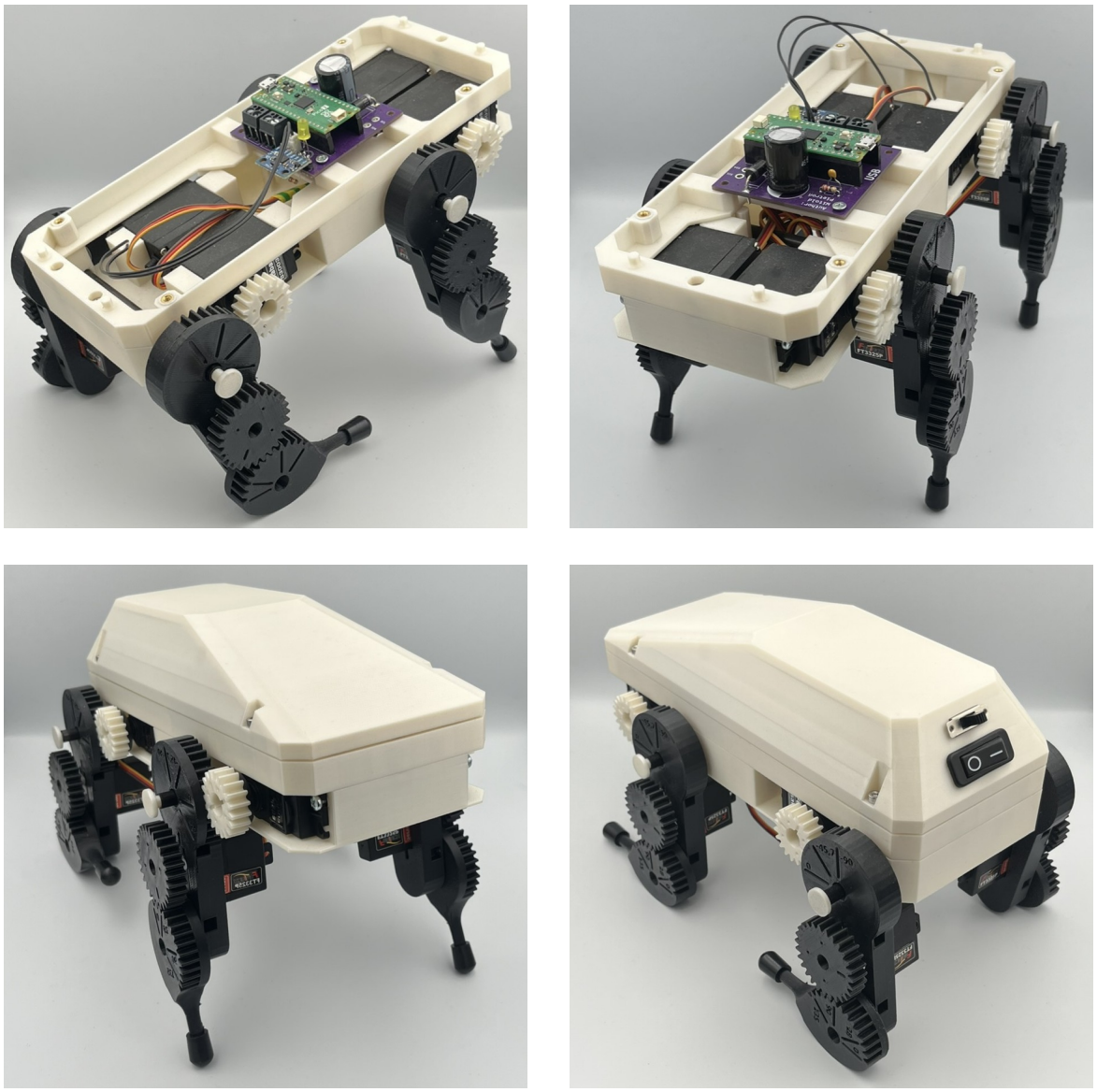
Zaprojektowany oraz wykonany robot kroczący charakteryzuje się kompaktową konstrukcją oraz niewielką masą własną, co pozwala na jego sprawne poruszanie się przy jednoczesnym niewielkim obciążeniu napędów. Podstawowe parametry fizyczne konstrukcji robota przedstawiono poniżej:

- masa robota: 773 g,
- szerokość konstrukcji: 15 cm,
- długość konstrukcji: 25 cm,
- wysokość robota na ugiętych nogach: 14 cm,
- wysokość robota na wyprostowanych nogach: 18,5 cm.

Ponadto przygotowano zestawienie wszystkich zastosowanych komponentów, uwzględniające ich masy oraz koszty, przedstawione w Załączniku 1. Rysunek 5.6 przedstawia zdjęcia gotowej konstrukcji robota z różnych perspektyw.

5.4. Model kinematyczny

Opracowanie modelu kinematycznego jest fundamentalnym krokiem w procesie tworzenia układu sterowania praktycznie każdego robota. Jego celem, w wypadku maszyny kroczącej, jest matematyczne powiązanie kątów obrotu w poszczególnych stawach z



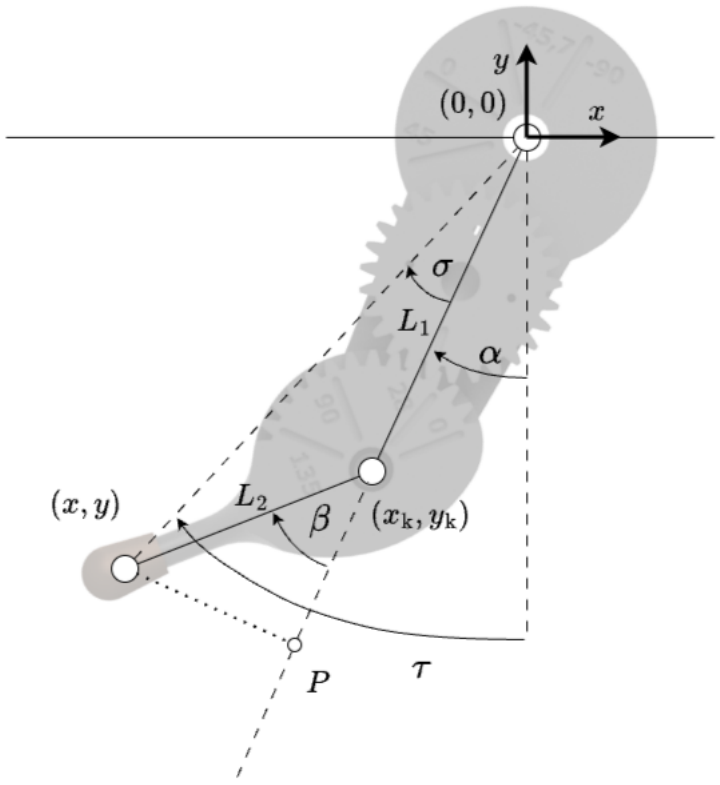
Rysunek 5.6. Zdjęcia złożonej konstrukcji

rzeczywistym położeniem końca kończyny, czyli stopy. Zamiast tworzyć jeden, wysoce złożony model kinematyczny dla całej konstrukcji, w niniejszym podrozdziale analizę ograniczono wyłącznie do pojedynczej nogi. Wynika to z faktu, że konstrukcja robota jest symetryczna, a system sterowania przetwarza ruch każdej z kończyn w sposób niezależny, w jej lokalnym układzie współrzędnych. Wyprowadzone w ten sposób równania można łatwo zastosować do sterowania pozostałymi nóg.

Biorąc pod uwagę fakt, że projekt w głównej mierze opiera się na metodach uczenia maszynowego, krok związany z analitycznym wyprowadzaniem modelu kinematycznego można było teoretycznie pominąć. W podejściu opartym na uczeniu przez wzmacnianie agent powinien bowiem być w stanie samodzielnie nauczyć się interpretować przestrzeń obserwacji i podejmować odpowiednie akcje, bez konieczności ich fizycznej interpretacji. Zdecydowano się jednak na implementację tego modelu z dwóch kluczowych powodów.

Po pierwsze, pozwala to na zachowanie pełnej świadomości bieżącego stanu robota, a więc dokładnej pozycji każdej ze stóp w przestrzeni w dowolnym momencie jego działania. Po drugie, rozwiązanie odwrotnego zadania kinematyki było niezbędne do stworzenia systemu sterowania opartego na manualnie zaplanowanej trajektorii referencyjnej chodu, co zostało szczegółowo opisane w rozdziale 6.

Rysunek 5.7 przedstawia schemat nogi robota wraz ze zmiennymi opisującymi jej konfigurację: α odpowiadającą położeniu stawu biodrowego, oraz β odpowiadającą położeniu stawu kolanowego. Długości uda i łydki oznaczono odpowiednio przez L_1 i L_2. Ponieważ ruch nogi odbywa się w jednej płaszczyźnie, analizę ograniczono do przypadku dwuwymiarowego. W dalszej części rozdziału przedstawiono wyprowadzenie prostego i odwrotnego zadania kinematyki dla rozważanego modelu.



Rysunek 5.7. Schemat nogi robota

5.4.1. Proste zadanie kinematyki

Wyprowadzenie równań opisujących położenie końcówki nogi dla układu z dwoma przegubami jest stosunkowo proste. Współrzędne stawu kolanowego w układzie odniesie-

nia, którego początek znajduje się w punkcie mocowania nogi, oznaczone jako (x_k, y_k), można zapisać w postaci:

$$x_k = -L_1 \cdot \sin(\alpha) \tag{3}$$

$$y_k = -L_1 \cdot \cos(\alpha) \tag{4}$$

Łydka robota jest odchylona od pionu o kąt równy sumie kątów $\alpha + \beta$. W rezultacie współrzędne końca efektora (x, y) opisane są zależnościami:

$$x = -L_1 \cdot \sin(\alpha) - L_2 \cdot \sin(\alpha + \beta) \tag{5}$$

$$y = -L_1 \cdot \cos(\alpha) - L_2 \cdot \cos(\alpha + \beta) \tag{6}$$

5.4.2. Odwrotne zadanie kinematyki

Odwrotne zadanie kinematyki dla tego przypadku można łatwo rozwiązać korzystając z twierdzenia cosinusów dla trójkąta utworzonego przez punkty wyznaczane odpowiednio przez: staw biodrowy, staw kolanowy oraz końcówkę nogi. Równanie, po wstawieniu zależności wynikającej z twierdzenia Pitagorasa przyjmuje postać:

$$x^2 + y^2 = L_1^2 + L_2^2 + 2L_1L_2\cos(\beta) \tag{7}$$

Stąd wyznaczamy wartość $\cos(\beta)$:

$$\cos(\beta) = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2} \tag{8}$$

W celu jednoznacznego wyznaczenia kąta, obliczamy wartość $\sin(\beta)$:

$$\sin(\beta) = \pm\sqrt{1 - \cos^2(\beta)} \tag{9}$$

Ostateczną wartość kąta β wyznaczamy za pomocą funkcji atan2:

$$\beta = \text{atan2}\left(\sin(\beta), \cos(\beta)\right) \tag{10}$$

Oznaczmy przez τ kąt określający kierunek od początku układu współrzędnych do zadanego punktu docelowego (x, y) względem pionowej osi. Wyznaczenie kąta obrotu stawu biodrowego α realizujemy poprzez odjęcie od kąta τ kąta korekcyjnego wynikającego z rozwarcia ramion wewnątrz trójkąta wyznaczanego przez dwa przeguby σ. Kąt τ łatwo wyznaczyć jako:

$$\tau = \text{atan2}(-x, -y) \tag{11}$$

Kąt korekcyjny σ wyznaczamy analizując trójkąt rozpięty przez człony L_1, L_2 oraz wektor pozycji docelowej. Wykorzystując rzut geometryczny punktu (x, y) na oś pierwszego

ramienia P, definiujemy odpowiednio składową prostopadłą k_1 oraz równoległą k_2:

$$k_1 = L_2 \cdot \sin(\beta) \tag{12}$$

$$k_2 = L_1 + L_2 \cdot \cos(\beta) \tag{13}$$

Poszukiwany kąt wewnętrzny trójkąta σ wynosi:

$$\sigma = \text{atan2}(k_1, k_2) \tag{14}$$

Ostateczną otrzymujemy:

$$\alpha = \tau - \sigma \tag{15}$$

6. Nauka chodu w symulacji

Proces uczenia robota kroczącego w środowisku symulacyjnym miał charakter iteracyjny i ewolucyjny. Stanowił on wieloetapowy cykl eksperymentów, w ramach którego stopniowo modyfikowano zarówno środowisko symulacyjne, algorytm uczenia, jak i parametry fizyczne konstrukcji robota – w przypadkach, gdy okazywało się to konieczne dla uzyskania poprawy wyników.

W kolejnych iteracjach analizowano uzyskiwane zachowanie robota, identyfikowano problemy (takie jak brak stabilności, bardzo krótki czas trwania epizodów, nienaturalny lub zdegradowany chód oraz niepożądane wzorce ruchu), a następnie podejmowano próbę określenia ich przyczyn. Na tej podstawie wprowadzano modyfikacje mające na celu eliminację zaobserwowanych nieprawidłowości. Po każdej zmianie proces uczenia był powtarzany, a jego rezultaty poddawane ponownej analizie. Na przestrzeni kolejnych iteracji modyfikacjom podlegały między innymi: konstrukcja oraz parametry fizyczne robota (np. masa i wymiary poszczególnych elementów), charakterystyka napędów (w przypadkach gdy okazywały się zbyt ciężkie lub niewystarczająco mocne), hiperparametry procesu uczenia (takie jak współczynnik uczenia, liczba epok czy architektura sieci aktora i krytyka), definicja funkcji nagrody wraz z wagami jej składowych, a także wybrane parametry środowiska symulacyjnego.

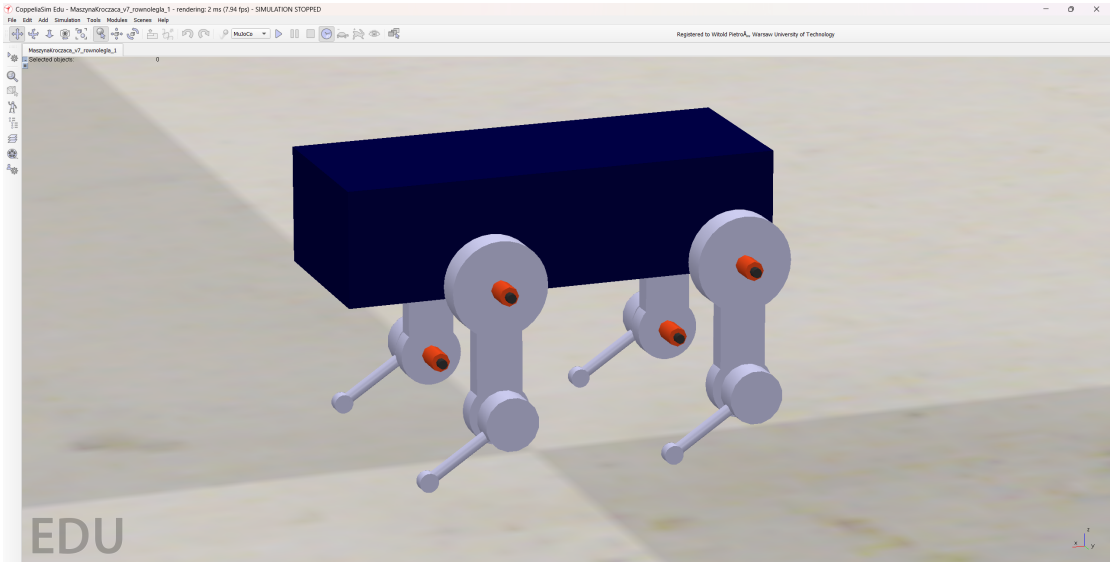
Wielokrotne powtarzanie tego procesu umożliwiło wypracowanie konstrukcji robota odpowiedniej do realizacji chodu oraz stopniowe udoskonalenie zarówno samego procesu uczenia, jak i sposobu oceny jego rezultatów. Takie podejście pozwoliło również na lepsze zrozumienie wpływu poszczególnych komponentów systemu na jakość uzyskiwanego chodu i stanowiło podstawę do przeprowadzenia bardziej sformalizowanych eksperymentów przedstawionych w dalszej części pracy.

Na tym etapie jedynym elementem procesu uczenia, który nie był modyfikowany, był sam algorytm – Proximal Policy Optimization. Wybór ten wynikał z jego wysokiej skuteczności w zadaniach o ciągłej przestrzeni stanów i akcji, stabilności procesu uczenia, braku konieczności bardzo precyzyjnego strojenia hiperparametrów oraz możliwości prowadzenia równoległej eksploracji środowiska przez wielu agentów. Z tego względu zdecydowano się wykorzystać ten algorytm jako podstawę do opracowania metryki jakości chodu, konstrukcji modelu oraz pozostałych elementów środowiska. Po ustaleniu tych parametrów przeprowadzono porównanie działania algorytmu z konkurencyjnymi podejściami: Advantage Actor-Critic oraz Soft Actor-Critic.

Niniejszy rozdział rozpoczyna się od skróconego opisu procesu tworzenia cyfrowego bliźniaka oraz wykorzystanego środowiska symulacyjnego. Następnie przedstawiono przestrzeń akcji oraz obserwacji, definiując sposób reprezentacji stanu środowiska dostępnego dla robota. Kolejno omówiono metrykę stosowaną do oceny jakości chodu wraz z krótkim opisem jej ewolucji. Ostatnia część rozdziału poświęcona jest eksperymentom, które doprowadziły do uzyskania końcowej, działającej najlepiej polityki sterowania robotem.

6.1. Cyfrowy bliźniak

W celu przeprowadzenia procesu uczenia przez wzmacnianie stworzono cyfrowego bliźniaka robota w środowisku symulacyjnym CoppeliaSim[1]. Jest on przedstawiony na rysunku 6.1. Model symulacyjny miał za zadanie możliwie wiernie odwzorować właściwości rzeczywistej platformy, tak aby wytrenowana polityka sterowania mogła zostać przeniesiona na fizycznego robota.



Rysunek 6.1. Cyfrowy bliźniak robota w programie CoppeliaSim

Podczas tworzenia cyfrowego bliźniaka skupiono się przede wszystkim na odwzorowaniu kluczowych parametrów wpływających na dynamikę ruchu robota. Uwzględniono wymiary oraz proporcje konstrukcji, a także masy poszczególnych elementów. Istotnym aspektem było również odwzorowanie właściwości napędów, takich jak maksymalne prędkości i momenty generowane w stawach robota.

Stworzony model nie stanowił jednak dokładnej repliki geometrycznej rzeczywistej konstrukcji i składał się z dziewięciu głównych elementów: czterech ud, czterech łydek oraz korpusu. Zamiast szczegółowego odwzorowania wszystkich podzespołów zastosowano uproszczone bryły geometryczne, takie jak prostopadłościany oraz walce. Takie podejście pozwoliło ograniczyć złożoność obliczeń fizycznych potrzebnych do przeprowadzania symulacji i okazało się w zupełności wystarczające. Upraszczanie ilości obliczeń niezbędnych do przeprowadzenia symulacji miało szczególne znaczenie ze względu na chęć przeprowadzania długotrwałego procesu uczenia przez wzmacnianie na wielu równoległych instancjach symulowanego środowiska.

Podczas budowy modelu zadbano również o poprawne odwzorowanie rozkładu masy oraz położenia środków ciężkości poszczególnych segmentów robota. Parametry te zostały wyznaczone na dwa sposoby. W przypadku elementów kończyn zrobiono to analitycz-

[1] Oficjalna strona programu CoppeliaSim: `https://www.coppeliarobotics.com/`

nie, bezpośrednio z oprogramowania CAD, w którym je zamodelowano. Zdecydowanie bardziej złożonym przypadkiem był korpus platformy – ze względu na asymetryczne rozmieszczenie komponentów wewnętrznych. Z tego powodu położenie środka ciężkości korpusu wyznaczono w sposób empiryczny, wykorzystując metodę równoważenia na krawędzi. Poprzez doświadczalne poszukiwanie osi, względem których momenty sił ciężkości się równoważyły, precyzyjnie zlokalizowano środek masy bryły głównej robota. Zastosowanie takiego hybrydowego, analityczno-pomiarowego podejścia miało na celu uzyskanie parametrów dynamicznych cyfrowego bliźniaka możliwie zbliżonych do rzeczywistych.

6.2. Identyfikacja parametrów dynamicznych konstrukcji

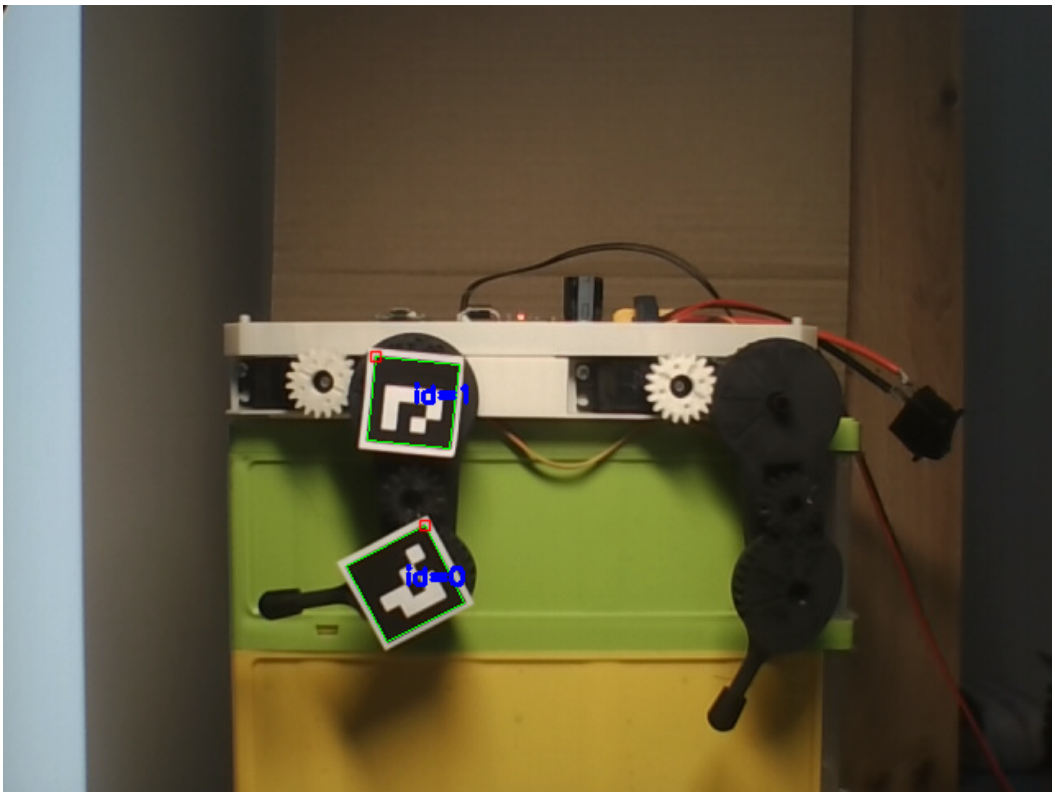
Jedną z głównych obaw podczas realizacji projektu było ryzyko wystąpienia znacznych różnic pomiędzy dynamiką modelu symulacyjnego a rzeczywistą konstrukcją robota. Oznaczałoby to, że polityka sterowania wytrenowana w symulacji będzie działała poprawnie wyłącznie w środowisku wirtualnym, natomiast po przeniesieniu na rzeczywistego robota okaże się nieskuteczna z powodu innych prędkości ruchu kończyn oraz ograniczeń napędów.

W celu ograniczenia tego problemu zdecydowano się na identyfikację parametrów dynamicznych konstrukcji, w szczególności maksymalnych prędkości ruchu stawów biodrowych oraz kolanowych. Podstawą do wyznaczenia wartości tych parametrów były dane zebrane w czasie eksperymentów przeprowadzanych na rzeczywistym robocie. Pomiary przeprowadzono bez dodatkowego obciążenia, a uzyskane wartości zostały następnie uwzględnione jako ograniczenie dynamiczne cyfrowego bliźniaka.

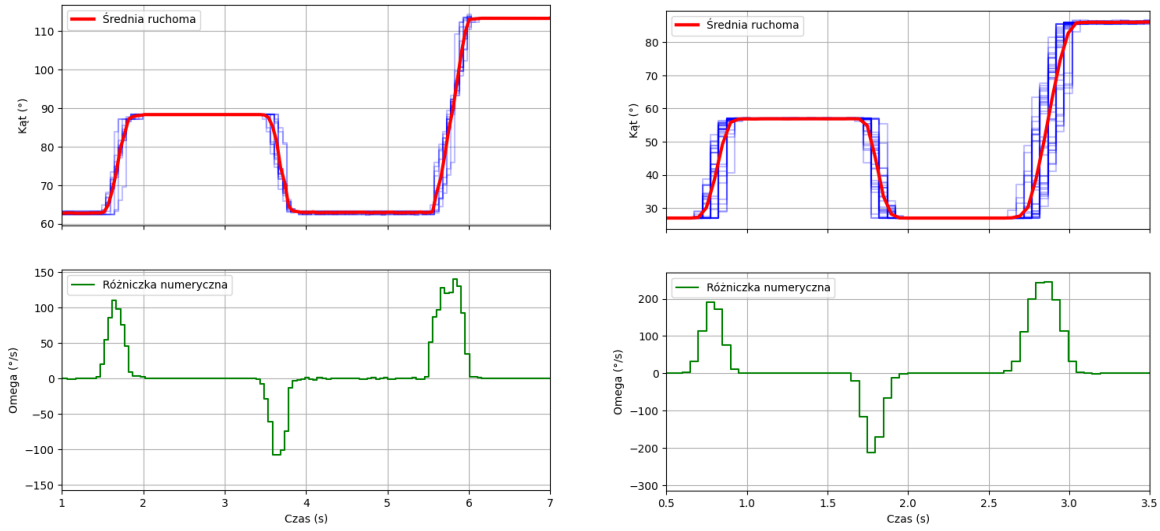
Do realizacji pomiarów wykorzystano znaczniki ArUco umieszczone na kończynach robota (rys. 6.2). Próbowano również zastosować w tym celu kody QR ale ich wykrycie w trakcie ruchu okazało się bardzo trudne. Markery ArUco zapewniały zdecydowanie bardziej niezawodną detekcję położenia. W trakcie eksperymentu robot został ustawiony tak, by jego noga znajdowała się dokładnie naprzeciw kamery rejestrującej obraz z częstotliwością 20 Hz. Do rejestracji wykorzystano standardową kamerę wbudowaną w laptop, skonfigurowaną na możliwie krótki czas naświetlania w celu ograniczenia rozmycia ruchu.

Na podstawie zarejestrowanych obrazów wyznaczano położenia kątowe stawów biodrowych oraz kolanowych w kolejnych chwilach czasu. Następnie, przy użyciu różniczki numerycznej, oszacowano odpowiadające im prędkości kątowe. W celu ograniczenia wpływu zakłóceń oraz błędów pomiarowych wyniki uśredniono na podstawie kilkudziesięciu cykli ruchu. Otrzymane charakterystyki przedstawiono na rysunkach 6.3a oraz 6.3b.

W wyniku analizy wyników identyfikacji przedstawionych na rysunku 6.3 zdecydowano się na ograniczenie maksymalnych prędkości kątowych stawów w symulacji do 100 rad/s dla stawu biodrowego oraz do 180 rad/s dla stawu kolanowego. Wyniki wskazują, że w ruchu bez obciążenia stawy mogą chwilowo osiągać wyższe prędkości kątowe. Przepro-



Rysunek 6.2. Zdjęcie robota w trakcie identyfikacji parametrów dynamicznych ruchu jego kończyn wraz z wyróżnionymi znacznikami ArUco



(a) Staw biodrowy

(b) Staw kolanowy

Rysunek 6.3. Wyniki identyfikacji parametrów dynamicznych konstrukcji. Górne wykresy przedstawiają zmiany pozycji, natomiast dolne wykresy przedstawiają prędkości stawów biodrowego (a) oraz kolanowego (b).

wadzone testy wykazały jednak, że w praktyce ruch odbywa się głównie na krótszych zakresach, co uzasadnia przyjęte ograniczenia.

6.3. Środowisko symulacyjne

Do przeprowadzenia eksperymentów wykorzystano środowisko symulacyjne CoppeliaSim. Narzędzie to jest szeroko stosowane w zastosowaniach robotycznych i dobrze nadaje się do realizacji tego typu badań. Istotną zaletą CoppeliaSim jest intuicyjny proces tworzenia modeli – konstrukcja oraz późniejsza modyfikacja robota mogą być realizowane w sposób interaktywny, z wykorzystaniem graficznego interfejsu użytkownika, co umożliwia wprowadzanie niezbędnych zmian w krótkim czasie.

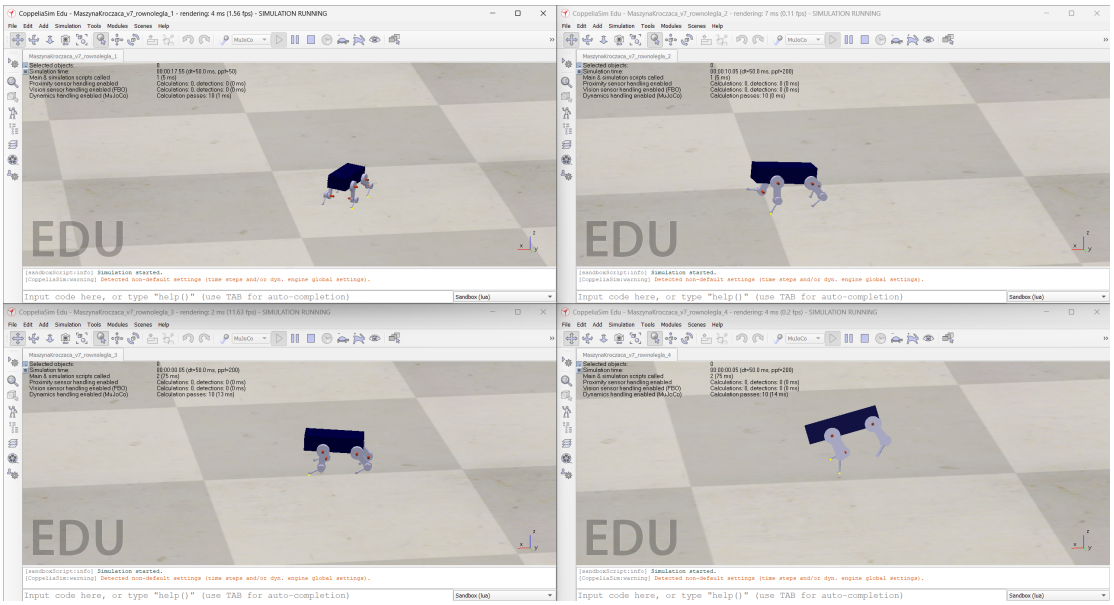
Środowisko to zapewnia również wygodny dostęp do interfejsu programistycznego (API), umożliwiającego sterowanie symulacją oraz integrację z zewnętrznymi narzędziami, w tym implementacjami algorytmów uczenia maszynowego. Dzięki temu możliwe było bezpośrednie połączenie środowiska symulacyjnego z kodem w języku Python, odpowiedzialnym za proces uczenia.

W trakcie prac konieczna okazała się zmiana silnika fizycznego wykorzystywanego przez środowisko. Początkowo stosowany silnik Bullet[2] nie zapewniał wystarczająco realistycznego modelowania kontaktu pomiędzy końcówką robota a podłożem, w szczególności w zakresie tarcia. Miało to istotny wpływ na jakość uzyskiwanego chodu oraz stabilność robota. Ostatecznie zdecydowano się na wykorzystanie silnika MuJoCo[3], który charakteryzuje się bardziej wiarygodnym odwzorowaniem zjawiska tarcia i pozwala na uzyskanie bardziej naturalnego zachowania robota podczas interakcji z podłożem. Co istotne, zmiana ta nie wpłynęła negatywnie na czas trwania obliczeń, a proces symulacji w nowym środowisku pozostał płynny. Kosztem przejścia na silnik MuJoCo była jedynie konieczność bardziej rygorystycznego zdefiniowania modelu – środowisko to cechuje się znacznie mniejszą tolerancją na błędy, takie jak na przykład przenikanie się brył, w stosunku do pierwotnie używanego silnika.

Informacje dostępne w literaturze potwierdzają zasadność wykorzystania silnika MuJoCo w tego typu aplikacjach. Silniki fizyczne, takie jak Bullet, wywodzą się głównie z branży rozrywkowej i gier wideo, gdzie priorytetem jest wydajność obliczeniowa przy dużej liczbie zderzających się obiektów swobodnych, co często odbywa się kosztem dokładności fizycznej kontaktu. Z kolei MuJoCo to środowisko stworzone specjalnie na potrzeby robotyki [31]. Podsumowując, przeprowadzona zmiana silnika miała wymierny, pozytywny wpływ na końcową jakość symulowanego chodu.

W celu przyspieszenia procesu uczenia zastosowano równoległe uruchamianie wielu instancji środowiska symulacyjnego. Eksperymenty były prowadzone jednocześnie na czterech niezależnych środowiskach (przykładowa wizualizacja jednoczesnej nauki wielu agentów przedstawiona jest na rys. 6.4), co pozwoliło na efektywniejsze zbieranie doświadczeń i skrócenie czasu treningu modeli.

[2] Oficjalna strona projektu Bullet Physics: `https://github.com/bulletphysics/bullet3`
[3] Oficjalna strona projektu MuJoCo: `https://mujoco.org`



Rysunek 6.4. Cztery instancje środowiska symulacyjnego CoppeliaSim, na których równolegle prowadzony jest proces uczenia

6.4. Integracja środowiska z biblioteką Stable-Baselines3

W celu realizacji procesu uczenia wykorzystano bibliotekę Stable-Baselines3[4], stanowiącą jedną z najpopularniejszych implementacji algorytmów uczenia przez wzmacnianie w języku Python. Biblioteka ta udostępnia gotowe implementacje algorytmów takich jak PPO, A2C czy SAC oraz zapewnia obsługę równoległego uczenia w wielu środowiskach jednocześnie.

Ponieważ środowisko CoppeliaSim nie jest natywnie zgodne z interfejsem biblioteki Stable-Baselines3, konieczne było stworzenie własnej klasy środowiska zgodnej ze standardem Gymnasium. Gymnasium to powszechnie przyjęty w środowisku naukowym standard implementacji algorytmu uczenia przez wzmacnianie, definiujący uniwersalny sposób interakcji agenta ze środowiskiem. Spełnienie tego standardu jest architektonicznym wymogem biblioteki Stable-Baselines3, której algorytmy operują wyłącznie na obiektach dziedziczących po klasie `gymnasium.Env`. Standard ten wymaga implementacji podstawowych metod, takich jak `step()` oraz `reset()`, a także zdefiniowania przestrzeni akcji i obserwacji.

Klasa ta pełniła rolę warstwy pośredniczącej pomiędzy interfejsem programistycznym CoppeliaSim a biblioteką uczenia przez wzmacnianie. Zastosowanie ustandaryzowanego interfejsu miało kluczowy wpływ na architekturę projektu – zapewniło wysoką modularność rozwiązania. Dzięki temu możliwe było testowanie różnych algorytmów uczenia bez konieczności jakiejkolwiek ingerencji w kod samego środowiska symulacyjnego.

[4] Oficjalna dokumentacja biblioteki Stable-Baselines3: `https://stable-baselines3.readthedocs.io/`

Zaimplementowane środowisko odpowiadało za reset i inicjalizację symulacji, wykonywanie akcji generowanych przez agenta, odczyt obserwacji ze środowiska oraz wyznaczanie wartości funkcji nagrody. Dodatkowo środowisko realizowało synchronizację pomiędzy przebiegiem symulacji a procesem uczenia prowadzonym przez algorytmy biblioteki Stable-Baselines3. Przygotowanie środowiska zgodnego z interfejsem Gymnasium umożliwiło efektywne wykorzystanie standardowych implementacji algorytmów uczenia przez wzmacnianie.

6.5. Przestrzeń akcji i obserwacji

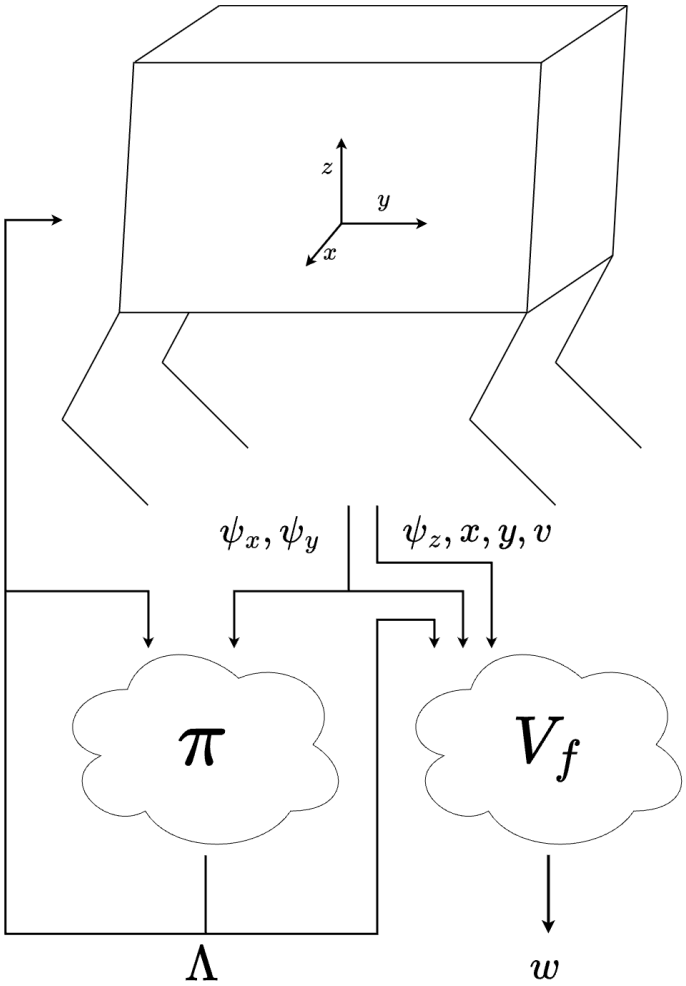
Charakter problemu sterowania robotem narzucił zdefiniowanie przestrzeni akcji jako zadawanych pozycji ośmiu stawów robota. Zdefiniowanie przestrzeni obserwacji było bardziej złożone, ponieważ możliwe było włączenie w jej skład różnego rodzaju informacji opisujących stan układu. Ze względu na zastosowanie serwomechanizmów bez sprzężenia zwrotnego nie było możliwe bezpośrednie odczytanie rzeczywistych położeń stawów podczas pracy robota. W celu częściowego odtworzenia informacji o stanie układu zdecydowano się na zastosowanie architektury autorekurencyjnej, w której w roli obserwacji wejściowej wykorzystywane były akcje wygenerowane w poprzednich krokach czasowych. Opisane zależności są częścią szerszej architektury algorytmu uczącego. Dokładny schemat przepływu informacji pomiędzy agentem działającym w środowisku symulacyjnym, siecią aktora oraz siecią krytyka przedstawiono na rysunku 6.5.

Dodatkowo obserwacje rozszerzono o estymowane za pomocą czujnika MPU-6050 wartości przechyłu $\psi_{\mathrm{y}}(k)$ oraz pochyłu $\psi_{\mathrm{x}}(k)$ dla chwili k. Uwzględnienie informacji o orientacji umożliwiło uzyskanie stabilniejszego chodu oraz zapewniło podstawową formę rzeczywistego sprzężenia zwrotnego. Ostateczna obserwacja wejściowa obejmowała historię stanów z ostatnich n kroków czasowych. Wartość n została dobrana w trakcie eksperymentów opisanych w dalszej części pracy. Działanie sieci polityki π można opisać zależnością:

$$\begin{aligned}\Lambda(k) = \pi(&\Lambda(k-1), \psi_{\mathrm{x}}(k), \psi_{\mathrm{y}}(k), \ldots, \\ &\Lambda(k-n), \psi_{\mathrm{x}}(k-n+1), \psi_{\mathrm{x}}(k-n+1))\end{aligned} \tag{16}$$

gdzie $\Lambda(k)$ oznacza wektor pozycji stawów robota w chwili k.

W trakcie procesu uczenia równolegle optymalizowana była również sieć krytyka V_f, odpowiedzialna za aproksymację funkcji wartości stanu, wykorzystywanej następnie do wyznaczania estymaty przewagi w algorytmie PPO. Podczas pierwszych prób nauki agenta stosowano podejście klasyczne, w którym wejście sieci krytyka jest takie samo jak wejście sieci aktora. Najpopularniejsza biblioteka języka Python implementująca algorytmy i środowisko do uczenia maszynowego Stable-Baselines3 nie przewiduje standardowo



Rysunek 6.5. Schamat przedstawiający przepływ informacji w środowisku w trakcie procesu nauki przez wzmacnianie z wyodrębnioną siecią polityki π oraz siecią krytyka (funkcją wartości) V_f

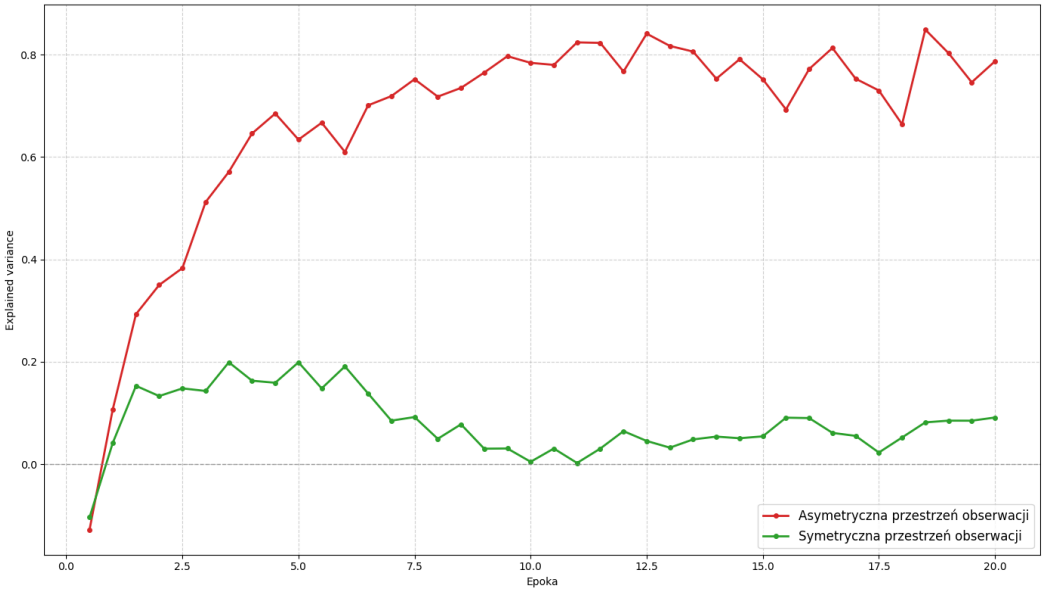
innego podejścia. Konfiguracja taka nie pozwalała jednak na zadowalające nauczenie się sieci krytyka.

Okazało się, że **kluczowym czynnikiem sukcesu całego procesu nauki było wykorzystanie asymetrycznej przestrzeni obserwacji aktora i krytyka**. Podczas gdy sieć aktora otrzymywała wyłącznie informacje dostępne na rzeczywistym robocie, sieć krytyka miała dostęp do poszerzonej informacji o stanie środowiska symulacyjnego, obejmującej położenie $x(k)$, $y(k)$, prędkość $v(k)$ oraz pełną orientację robota, a więc informację dostępną dla aktora uzupełnioną o informację o chwilowym odchyleniu $\psi_z(k)$.

Jakość predykcji tej sieci może być na bieżąco monitorowana w trakcie treningu za pomocą metryki wariancji wyjaśnionej (ang. *explained variance*[5]), rejestrowanej domyślnie na każdym etapie procesu uczenia. Parametr ten określa, w jakim stopniu przewidywania krytyka tłumaczą wariancję rzeczywistych nagród zdobytych przez agenta. Przy standar-

[5] Definicja tego parametru w bibliotece Stable-Baselines3 dostępna jest w oficjalnej dokumentacji: `https://stable-baselines3.readthedocs.io/en/v1.0/common/utils.html`

dowym podejściu wartość tej metryki utrzymywała się na bardzo niskim poziomie, co wskazywało na błędne szacowanie funkcji wartości stanu, a w konsekwencji uniemożliwiało agentowi wyuczenie się prawidłowej trajektorii chodu. Zastosowanie asymetrycznej przestrzeni obserwacji aktora i krytyka pozwoliło na znaczący wzrost wartości tego współczynnika. Na rysunku 6.6 przedstawiono zmianę tego parametru w trakcie pierwszych 20 epok dla dwóch badanych wariantów architektury – wykorzystujących symetryczną oraz asymetryczną przestrzeń obserwacji w ramach wejścia sieci aktora i krytyka.



Rysunek 6.6. Przebieg zmian współczynnika wariancji wyjaśnionej dla dwóch wariantów architektury sieci aktora i krytyka w trakcie pierwszych 20 epok treningu

Takie rozwiązanie umożliwiło w końcu osiągnięcie zadowalającej dokładności szacowania wartości stanu przez krytyka, bez wprowadzania do sieci aktora informacji niedostępnych podczas rzeczywistej pracy robota. Dodanie tych danych również do obserwacji aktora mogłoby przyspieszyć proces uczenia, prowadziłoby jednak do zjawiska określanego jako wyciek informacji (ang. *information leakage*), polegającego na wykorzystaniu podczas uczenia informacji niedostępnych na etapie inferencji. Podejście to jest zgodne z koncepcją asymetrycznego aktora-krytyka opisaną w literaturze [20]. Oznaczmy estymację funkcji wartości w chwili k przez $w(k)$ – wówczas działanie sieci krytyka można opisać formułą:

$$w(k) = V_f(\Lambda(k-1), \psi_x(k), \psi_y(k), \psi_z(k), v(k), x(k), y(k), \ldots, \\ \Lambda(k-n), \psi_x(k-n+1), \psi_y(k-n+1), \psi_z(k-n+1), v(k-n+1), x(k-n+1), y(k-n+1)) \tag{17}$$

6.6. Metryka skuteczności chodu

Jednym z największych wyzwań rozwiązanych w ramach niniejszej pracy, jak również w ogólności w zadaniach uczenia przez wzmacnianie, było opracowanie odpowiedniej metryki oceny jakości działania agenta. Funkcja nagrody pełni zawsze kluczową rolę, ponieważ bezpośrednio kształtuje zachowanie uczonej polityki. Przeprowadzone doświadczenia pokazały, że nawet niewielka modyfikacja tej funkcji może prowadzić do całkowicie odmiennego zachowania agenta. Konieczne więc było uwzględnienie w niej wszystkiego co istotne z punktu widzenia zadania chodu i dobranie odpowiednich wag poszczególnych jej składników.

Ostatecznie zastosowana metryka jakości chodu składa się z pięciu składników, których celem jest promowanie pożądanych oraz ograniczanie niepożądanych wzorców ruchu robota. Postać metryki obliczającej nagrodę $r(k)$ dla k-ego kroku epizodu przedstawiono poniżej:

$$r(k) = \alpha_1 \cdot r_{\text{prędkość}} + \alpha_2 \cdot p_{\text{wysokość}} + \alpha_3 \cdot p_{\text{rotacja}} + \alpha_4 \cdot p_{\text{równoległość nóg}} + r_{\text{życie}}$$

gdzie:

- $r_{\text{prędkość}}$ – składnik nagradzający ruch robota do przodu,
- $p_{\text{wysokość}}$ – kara za utrzymywanie korpusu robota zbyt blisko podłoża,
- p_{rotacja} – kara za rotację robota w trakcie chodu (w osiach przód-tył i lewo-prawo),
- $p_{\text{równoległość nóg}}$ – kara za synchroniczne poruszanie kończynami,
- $r_{\text{życie}}$ – stała nagroda za utrzymanie robota w stanie „aktywnym".

Współczynniki $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ określają względne znaczenie poszczególnych składników funkcji nagrody i były dobierane eksperymentalnie w trakcie procesu uczenia. Wartość tej metryki była obliczana na końcu każdego kroku symulacji i sumowana przez algorytm w trakcie trwania całego epizodu. Poniżej opisano dokładniej każdy z wymienionych składników wraz z motywacją do ich uwzględnienia i konsekwencjami przyznania im zbyt dużej, lub zbyt małej wagi.

6.6.1. Nagroda za ruch

Zdecydowanie najważniejszym składnikiem funkcji oceny ruchu jest ten, nagradzający za ruch robota do przodu. Z uwagi na charakter zadania, tj. ruch wzdłuż jednej osi, nagroda obliczana jest jedynie na podstawie przemieszczenia wzdłuż osi x układu. Wartość $r_{\text{prędkość}}$ obliczana jest jako:

$$r_{\text{prędkość}} = \begin{cases} v(k), & \text{dla } v(k) \geq 0 \\ v(k) \cdot \kappa, & \text{dla } v(k) < 0 \end{cases}$$

Doświadczenia pokazały, że bardzo istotne jest powiązanie nagrody z prędkością, a nie z przemieszczeniem wzdłuż osi x. W przypadku wykorzystania przemieszczenia zamiast prędkości agent może eksploatować strategię polegającą na wykonaniu pojedynczego

gwałtownego ruchu do przodu, a następnie pozostawaniu w bezruchu. Strategia taka pozwala utrzymać dodatnią sumaryczną nagrodę przy jednoczesnym ograniczeniu ryzyka przewrócenia robota.

Innym, istotnym elementem formuły jest zwiększenie wagi składnika podczas ruchu wstecz. Zniechęca to agentów do przyjmowania strategii chodu polegających na naprzemiennym wykonywaniu kroków wprzód i kilku wstecz. Testy pokazały, że wartość $\kappa = 1{,}2$ dobrze spełnia swoją rolę.

Zbyt duża wartość współczynnika α_1 powoduje, że agent skupia się wyłącznie na eksploatacji trajektorii chodu zapewniających szybkie przemieszczanie się, nie zważając na jego stabilność. Zbyt mała wartość powoduje, że agent skupia się na minimalizowaniu kar związanych z pozostałymi składnikami nagrody.

6.6.2. Kara za ruch zbyt blisko podłoża

Kolejnym składnikiem metryki jest składnik odpowiedzialny za utrzymanie korpusu robota na odpowiedniej wysokości, obliczany następująco:

$$p_{\text{wysokość}} = -max(0, h_0 - h)$$

Kara za utrzymywanie korpusu poniżej granicznej wartości h_0 jest więc proporcjonalna do przekroczenia tej wartości. W praktyce składnik ten ma największe znaczenie w początkowej fazie uczenia, gdy działania agenta mają w dużym stopniu charakter losowy. Zapobiega on eksploatowaniu strategii polegających na przemieszczaniu robota poprzez „czołganie się" po podłożu. Wartość współczynnika α_2 dobrano stosunkowo wysoko, przy jednoczesnym zachowaniu konserwatywnej wartości parametru h_0. Powoduje to występowanie wyraźnej kary dla zbyt niskiego położenia korpusu, przy jednoczesnym zaniku wpływu tego składnika po wyuczeniu pożądanej strategii chodu.

6.6.3. Kara za pochył i przechył

Następnym składnikiem metryki jest składnik karzący za nadmierny obrót robota w osi przód-tył i prawo-lewo, tj. pochył i przechył. Jego wartość obliczana jest według wzoru:

$$p_{\text{rotacja}} = -(max(0, |\psi_x| - \delta_1) + max(0, |\psi_y| - \delta_2))$$

gdzie ψ_x odpowiada kątowi pochylenia robota, ψ_y odpowiada kątowi przechyłu robota, natomiast δ jest współczynnikiem tolerancji, czyli wartością kąta, poniżej której pochył i przechył nie są karane. Wartości współczynników δ przyjęto jako $\delta_1 = 5°$ oraz $\delta_2 = 2{,}5°$.Testy wykazały, że o ile niewielkie i stabilne pochylenie robota może mieć pozytywny wpływ na jakość chodu, o tyle jego przechył w osi bocznej zawsze wpływa negatywnie na stabilność i płynność ruchu. Składnik p_{rotacja} odgrywa istotną rolę w kształtowaniu stabilnej strategii chodu. Wartość współczynnika α_3 dobrano w taki sposób, aby kara była wystarczająco duża, by ograniczać nadmierne przechyły robota, lecz jednocześnie na tyle

mała, aby nie dominowała nad pozostałymi składnikami funkcji nagrody. Szczególnie istotne okazało się wprowadzenie współczynnika tolerancji δ, który umożliwia agentowi wykonywanie niewielkich, bezpiecznych odchyleń korpusu niezbędnych podczas ruchu. Dzięki temu agent zachowuje możliwość eksploracji bardziej dynamicznych strategii chodu bez nadmiernego karania za niewielkie wychylenia.

6.6.4. Kara za synchronizację ruchu poszczególnych kończyn

Kolejnym składnikiem funkcji nagrody jest kara za poruszanie kończynami przednimi lub tylnymi w sposób równoległy. Bez uwzględnienia tego składnika agenci często przyjmują strategie chodu przypominające ruch żaby, w których robot wykonuje gwałtowne odepchnięcia obiema tylnymi kończynami jednocześnie. Takie strategie pozwalają uzyskać krótkotrwały wzrost prędkości, jednak prowadzą do niestabilnego i mało naturalnego sposobu poruszania się. Wartość składnika obliczana jest jako:

$$p_{\text{równoległość nóg}} = -\left(|\Delta\lambda_{\text{TL}} + \Delta\lambda_{\text{TP}}| + \kappa \cdot |\Delta\lambda_{\text{PL}} + \Delta\lambda_{\text{PP}}|\right) \cdot c$$

gdzie $\Delta\lambda$ oznacza zmianę kąta w stawie biodrowym, natomiast TL, TP, PL oraz PP odpowiadają kolejno tylnej lewej, tylnej prawej, przedniej lewej oraz przedniej prawej kończynie. W sytuacji gdy kończyny, przednie bądź tylne, poruszają się w przeciwnych kierunkach, znaki $\Delta\lambda$ są przeciwne i wartość kary jest niewielka. W przeciwnym wypadku, gdy kończyny poruszają się w jednym kierunku, wartość kary jest wysoka.

Bez zastosowania tego składnika funkcji nagrody agenci często wpadają w optimum lokalne odpowiadające wspomnianemu „żabiemu" sposobowi poruszania się. Dodanie tej kary wymusza bardziej naprzemienną pracę kończyn, co prowadzi do uzyskania stabilniejszych i wyraźnie efektywniejszych strategii chodu. W praktyce składnik ten promuje ruch przypominający kłus obserwowany u zwierząt czworonożnych[6].

W praktyce większe znaczenie miało ograniczenie równoległego ruchu kończyn tylnych, ponieważ to właśnie one odpowiadały za generowanie najbardziej niepożądanych strategii ruchu. Z tego względu zdecydowano się na zastosowanie dodatkowego współczynnika $\kappa = 0,5$, zmniejszającego wpływ synchronizacji kończyn przednich względem tylnych.

Wartość współczynnika α_4 dobrano stosunkowo zachowawczo, tak aby ten składnik funkcji nagrody nie zdominował pozostałych. Dodatkowo bardzo istotne okazało się stopniowe zwiększanie wpływu tego składnika wraz z czasem, co osiągnięto poprzez zastosowanie dodatkowego współczynnika c, którego wartość rosła stopniowo od 0,1 do 1,0 podczas procesu uczenia, zwiększając się o 0,1 co 300 zakończonych epizodów treningowych. Podejście takie stanowi przykład techniki *curriculum learning*, powszechnie

[6] Warto zauważyć, że w niektórych rodzajach chodu zwierząt czworonożnych, przykładowo w galopie, kończyny tylne przez dużą część cyklu poruszają się w tych samych kierunkach. W stosowanej metryce chód taki byłby jednak silnie karany. Głównym jego celem było uniknięcie wpadnięcia w inną suboptymalną trajektorię i w tej roli składnik ten sprawdził się bardzo dobrze.

stosowanej w uczeniu przez wzmacnianie, w której agent rozpoczyna naukę od prostszych wariantów zadania, ucząc się przede wszystkim utrzymania stabilności i unikania upadków, a następnie stopniowo przechodzi do bardziej wymagających aspektów, takich jak optymalizacja jakości chodu.

6.6.5. Nagroda za trwanie

Ostatnim składnikiem funkcji nagrody jest stała wartość $r_{\text{życie}}$ dodawana w każdym kroku epizodu, której celem jest premiowanie samego utrzymania aktywności agenta przez dłuższy czas. Ponieważ całkowita funkcja nagrody składa się z wielu składników obniżających nagrodę, początkowa polityka, oparta na losowej eksploracji, często generuje bardzo niskie jej wartości.

W trakcie eksperymentów zaobserwowano tendencję do przyjmowania suboptymalnej strategii polegającej na wykonaniu pojedynczego, gwałtownego ruchu do przodu, a następnie zakończeniu epizodu w wyniku celowej utraty równowagi. W sytuacji, gdy kary związane z rotacją i równoległością nóg są stosunkowo wysokie, a kara za zakończenie epizodu niewystarczająco silna, zachowanie takie może stanowić lokalne optimum metryki chodu.

Wprowadzenie niewielkiej, stałej nagrody za każdy krok pozwala przeciwdziałać temu zjawisku, zachęcając agenta do wydłużania epizodów i utrzymywania stabilnego chodu. W połączeniu z zastosowaniem *curriculum learning* mechanizm ten przyczynia się do eliminacji krótkowzrocznych strategii eksploracyjnych.

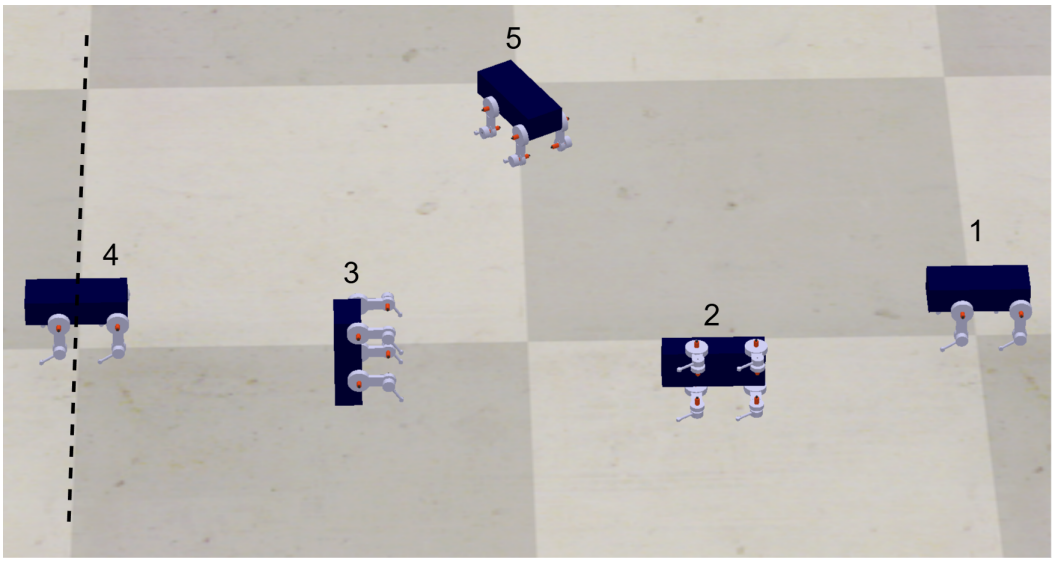
Wartość $r_{\text{życie}}$ musi być jednak odpowiednio mała, aby nie dominowała nad pozostałymi składnikami funkcji nagrody i nie prowadziła do strategii polegających wyłącznie na „przetrwaniu" bez efektywnego ruchu. Zbyt duża wartość tego składnika może znacząco zaburzyć proces uczenia i prowadzić do degeneracji polityki agenta.

6.6.6. Nagrody związane z zakończeniem epizodu

Dodatkową grupę składników funkcji nagrody stanowią nagrody oraz kary przyznawane w momencie zakończenia epizodu. Warunkiem jego końca jest utrata stabilności robota, skręt od trajektorii prostej o kąt większy od granicznego lub osiągnięcie celu, rozumianego jako przemieszczenie robota na określoną odległość wzdłuż osi ruchu. Możliwe stany kończące epizod przedstawione są na rysunku 6.7.

W sytuacji utraty stabilności, definiowanej jako przechył lub pochył o wartość większą od dopuszczalnej wartości granicznej epizod jest przerywany, a agentowi przyznawana jest kara, której celem jest silne zniechęcenie agenta do strategii prowadzących do przewracania się. Wartość kary dobrana jest w taki sposób by być dla agenta dotkliwą i zniechęcić do eksploracji technik mogących do niej prowadzić.

Dodatkowo epizod może zostać zakończony w wyniku nadmiernego odchylenia orientacji robota w osi skrętu, co również interpretowane jest jako stan niepożądany, jednak o mniejszym znaczeniu niż całkowita utrata równowagi. W tym przypadku kara ma charakter



Rysunek 6.7. Ilustracja przedstawiająca stan początkowy i możliwe stany kończące epizod: 1) stan początkowy 2) przerwanie eksperymentu spowodowane zbyt dużym przechyłem 3) przerwanie eksperymentu spowodowane zbyt dużym pochyłem 4) przerwanie eksperymentu spowodowane dotarciem do celu 5) przerwanie eksperymentu spowodowane zbyt dużym odchyleniem.

łagodniejszy. Robot w trakcie chodu nie posiada informacji o swojej orientacji w osi skrętu, a ruch po zboczeniu w lewo lub prawo, w interpretowanej przez niego przestrzeni stanu, może wyglądać tak samo jak ruch prawidłowy. By uniknąć sytuacji w której agent jest karany mimo wykonywania, teoretycznie, prawidłowej sekwencji ruchu kończyn trajektorie prowadzące do nadmiernego skrętu musiały być wcześnie eliminowane.

Ostatni przypadek zakończenia epizodu dotyczy osiągnięcia celu, zlokalizowanego w znacznej odległości od punktu początkowego. W takiej sytuacji agent otrzymuje dodatnią nagrodę końcową, która premiuje skuteczne i stabilne wykonanie zadania. Wartość tej nagrody musi być odpowiednio wysoka, ponieważ w przeciwnym przypadku bardziej opłacalną strategią może stać się powolne przemieszczanie w kierunku celu i akumulowanie mniejszych, ale stałych nagród w dłuższym horyzoncie czasowym.

6.7. Referencyjna trajektoria chodu

Oprócz nauki chodu metodami uczenia przez wzmacnianie zdecydowano się dodatkowo na stworzenie dodatkowej strategii poruszania się metodą „białej skrzynki”. Ze względu na uproszczoną konstrukcję robota, posiadającą jedynie dwa stopnie swobody i fakt, że niemożliwe było bezpośrednie regulowanie prędkości ruchu napędów klasyczne biblioteki generowania chodu nie mogły zostać zastosowane. W związku z tym opracowano własny generator referencyjnej trajektorii chodu robota. Generowana trajektoria została sparametryzowana za pomocą kilku kluczowych parametrów:

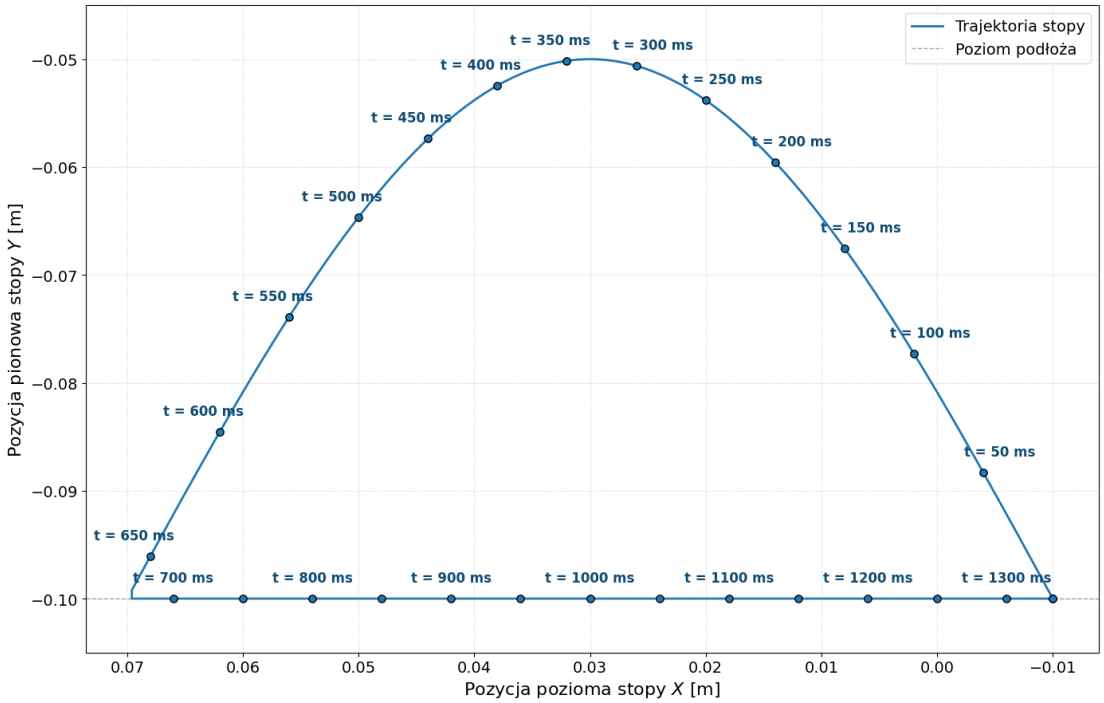
- częstotliwości kroku,
- długości kroku,

- wysokości uniesienia nogi,
- przesunięcia kroku względem pozycji bazowej ($x = 0$).

Dodatkowo przyjęto, że nogi robota pracują w układzie naprzemiennym, co oznacza, że kończyny przeciwległe wykonują ruch w tej samej fazie, natomiast pary nóg lewa–prawa są przesunięte o pół okresu cyklu chodu. Takie założenie jest rozsądnym uproszczeniem pozwalającym na skupieniu się na projektowaniu chodu jednej nogi i zapobiega konieczności ręcznej synchronizacji ruchu wielu kończyn.

Trajektoria końcówki nogi została zdefiniowana jako funkcja czasu zwracająca pozycję x i y w przestrzeni kartezjańskiej. Następnie dla każdej nogi rozwiązywane było odwrotne zadanie kinematyki, które pozwalało przekształcić zadaną pozycję stopy na odpowiadające kąty w stawach biodrowym i kolanowym.

W zaprojektowanym układzie sterowania przyjęto częstotliwość aktualizacji pozycji równą 20 Hz, co zapewnia płynną realizację ruchu oraz odpowiednią stabilność numeryczną w symulacji i na rzeczywistym robocie. Wizualizacja trajektorii ruchu końcówki kończyny robota przedstawiona jest na rysunku 6.8. Wartości poszczególnych parametrów generowanej trajektorii zostały dostrojone eksperymentalnie poprzez obserwowanie wpływu ich zmian na jakość chodu i wartość jego metryki.



Rysunek 6.8. Wizualizacja przebiegu trajektorii referencyjnej kończyny robota

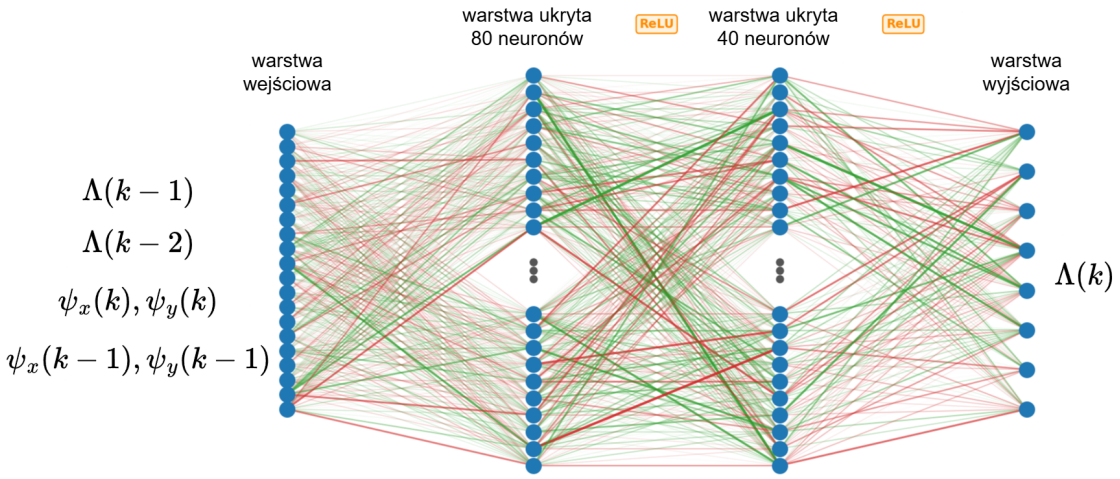
6.8. Realizacja sieci polityki

Większość nowoczesnych metod uczenia przez wzmacnianie realizuję politykę za pomocą modeli uczonych w sposób nadzorowany, najczęściej z użyciem sieci neuronowych.

W niniejszej pracy zdecydowano się na implementację sieci polityki w postaci wielowarstwowego perceptronu (ang. *Multi-Layer Perceptron* – MLP). Wybór tej architektury wynikał z jej prostoty oraz łatwości implementacji na mikrokontrolerze Raspberry Pi Pico.

Struktura sieci obejmuje warstwę wejściową, dwie warstwy ukryte oraz warstwę wyjściową generującą akcję sterującą. Kolejne warstwy mają odpowiednio rozmiary $20 \rightarrow 80 \rightarrow 40 \rightarrow 8$. Zastosowanie sieci o mniejszym rozmiarze miało negatywny wpływ na jakość treningu, natomiast zastosowanie sieci o większym rozmiarze nie poprawiało go w istotny sposób. W warstwach ukrytych zastosowano funkcję aktywacji Rectified Linear Unit (ReLU), wybraną ze względu na niską złożoność obliczeniową w porównaniu z innymi popularnymi funkcjami aktywacji, takimi jak tanh.

Zastosowanie architektury MLP zamiast sieci rekurencyjnych (ang. *recurrent neural network* – RNN) wynikało z faktu, że uwzględnienie historii obserwacji bezpośrednio w wektorze wejściowym okazało się wystarczające do poprawnego odwzorowania dynamiki układu. W konsekwencji nie było konieczne stosowanie bardziej złożonych oraz kosztownych obliczeniowo struktur rekurencyjnych. Ostateczna architektura sieci aktora została przedstawiona na rysunku 6.9.



Rysunek 6.9. Wizualizacja realizacji sieci aktora π dla sekwencji wejściowej o długości $n = 2$

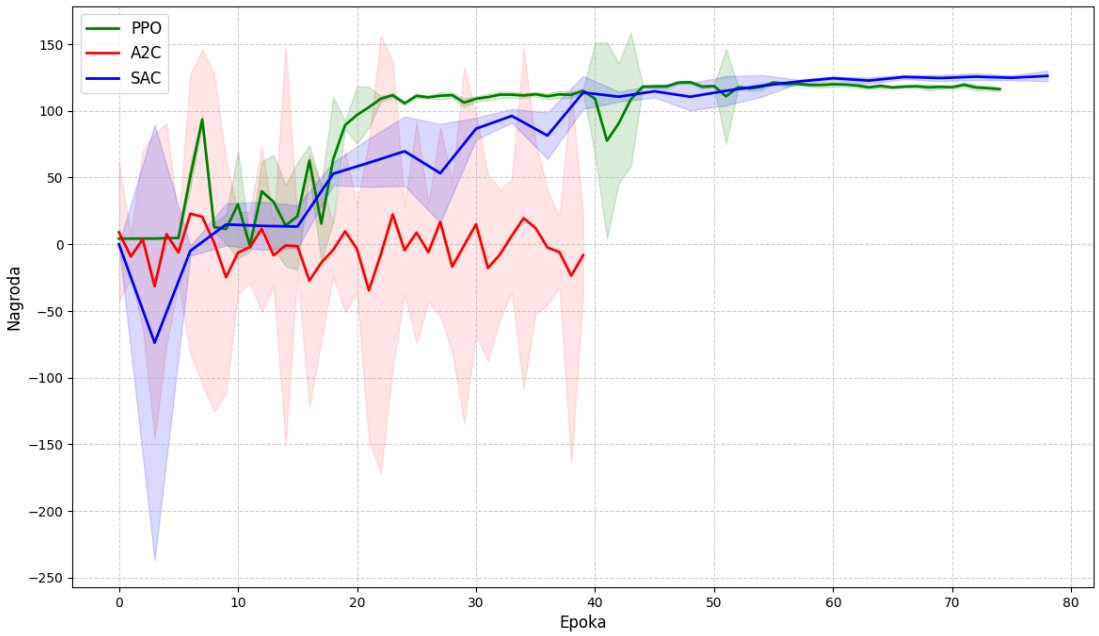
6.9. Eksperymenty

Tak jak wspomniano we wstępie do tego rozdziału cały proces uczenia robota kroczącego w środowisku symulacyjnym miał charakter iteracyjny i ewolucyjny. W jego trakcie wielokrotnie powtórzono cykl analizy zachowania agenta, szukania czynników decydujących o takim, a nie innym zachowaniu i próbie ich poprawy. Mimo takiego podejścia,

zdecydowano się jednak na przeprowadzenie pełnych testów porównawczych mających na celu ocenę wpływu wybranych parametrów i architektury systemu na skuteczność nauki chodu robota. Badania obejmowały porównanie różnych algorytmów uczenia przez wzmacnianie, analizę wpływu częstotliwości sterowania oraz ocenę znaczenia długości sekwencji obserwacji n wykorzystywanej przez sieć polityki.

Wszystkie eksperymenty przeprowadzono przy zachowaniu identycznej funkcji nagrody oraz takich samych warunków środowiska symulacyjnego. Mimo iż przeprowadzenie jednej sesji uczenia trwa wiele godzin, każdy eksperyment został powtórzony trzykrotnie w celu ograniczenia wpływu stochastycznego charakteru procesu uczenia. Dla każdej wyuczonej w ten sposób funkcji polityki przeprowadzono ewaluację polegającą na 10 eksperymentach rozpoczynających się z pozycji startowej, zaburzonej w sposób losowy. Dla 10 eksperymentów wyznaczono następnie średnią wartość metryki, wraz z jej odchyleniem standardowym, mówiącym o stabilności i niezawodności wyuczonej techniki chodu oraz średnią prędkość chodu osiąganą w trakcie eksperymentu.

6.9.1. Wybór algorytmu uczenia przez wzmacnianie



Rysunek 6.10. Średnia nagroda wraz z przedziałem ufności w kolejnych epokach treningu. Wartości metryki przedstawione na wykresie nie są dokładnie porównywalne, przedstawiają jednak poglądowo postępy nauki.

Testy rozpoczęto od zbadania wpływu wykorzystywanego algorytmu uczenia przez wzmacnianie na wyniki nauki. Sprawdzono trzy wybrane wcześniej i powszechnie stosowane algorytmy: Proximal Policy Optimization, Advantage Actor-Critic oraz Soft Actor-Critic. Wyniki eksperymentu polegającego na przeprowadzeniu trzech prób nauki dla każdego z algorytmów przedstawione są w tabeli 6.1. Dodatkowo na rysunku 6.10 przedstawiono

poglądowo przebieg jednego, wybranego, cyklu nauki dla każdego z algorytmów. Wartości metryki przedstawione na tym wykresie nie są dokładnie porównywalne – każdy z algorytmów działał lepiej dla indywidualnie dostrojonej funkcji metryki (za pomocą parametrów $\alpha_1, \alpha_2, \alpha_3, \alpha_4$). Końcowe wyniki ewaluacji, przedstawione w tabeli 6.1 były jednak przeprowadzone w identycznych warunkach.

	PPO		A2C		SAC	
	Metryka ± odch. std	Średnia prędkość (cm/s)	Metryka ± odch. std	Średnia prędkość (cm/s)	Metryka ± odch. std	Średnia prędkość (cm/s)
Nauka 1	112,85 ± 1,44	15,07	−27,98 ± 4,82	1,64	94,49 ± 33,04	11,35
Nauka 2	118,65 ± 1,02	17,05	−12,99 ± 1,90	1,23	104,14 ± 2,42	12,22
Nauka 3	119,15 ± 0,70	17,76	−12,63 ± 5,99	1,34	119,85 ± 1,91	16,89
Średnia	116,88	16,63	−17,87	1,40	106,16	13,49

Tabela 6.1. Wybór algorytmu uczenia przez wzmacnianie

Na podstawie wyników przedstawionych w tabeli 6.1 można zauważyć, że najlepsze rezultaty osiągnął algorytm PPO. Uzyskano zarówno najwyższą średnią wartość metryki, jak i największą średnią prędkość ruchu robota. Dodatkowo wyniki uzyskiwane przez algorytm cechowały się dużą powtarzalnością, a trajektorie wyznaczane na podstawie uzyskanej funkcji polityki charakteryzowała duża stabilność, co potwierdzają niewielkie wartości odchylenia standardowego metryki w trakcie ewaluacji.

W żadnej z prób uwzględnionych w tabeli 6.1, jak również w pozostałych eksperymentach przeprowadzonych przez autora, algorytm A2C nie był w stanie wykształcić skutecznej strategii ruchu robota. Agenci uczeni przy jego pomocy poruszali się w sposób chaotyczny, eksploatując strategie prowadzące do uzyskiwania niewielkich nagród. Skutkiem tego były ujemne wartości metryki podczas ewaluacji oraz bardzo niska średnia prędkość ruchu robota. Wyniki te wskazują na trudności algorytmu A2C w stabilnym uczeniu w rozważanym środowisku.

Wyniki osiągnięte przez algorytm SAC były zbliżone do rezultatów uzyskanych przez PPO, charakteryzowały się jednak znacznie mniejszą powtarzalnością oraz gorszą stabilnością generowanego chodu. Uzyskane rezultaty wskazują, że algorytm SAC, podobnie jak PPO, nadaje się do nauki sterowania robotem w rozpatrywanym środowisku. Ze względu na charakter algorytmu *off-policy*, SAC mógłby potencjalnie stanowić lepszy wybór w sytuacji, w której przeprowadzanie symulacji byłoby kosztowne obliczeniowo lub niemożliwe do efektywnego zrównoleglenia.

Uzyskane wyniki, pokazujące przewagę algorytmu PPO nad dwoma konkurencyjnymi rozwiązaniami, zgadzają się z początkowymi założeniami. Został on wytypowany na głównego kandydata jeszcze przed rozpoczęciem testów, podczas gdy pozostałe metody uwzględniono przede wszystkim w celach porównawczych. Ostatecznie, na podstawie jednoznacznych wyników eksperymentów oraz możliwości równoległego uruchamiania wielu instancji środowiska, zdecydowano się na wykorzystanie algorytmu PPO w dalszej części projektu.

6.9.2. Wybór częstotliwości sterowania

Kolejnym hiperparametrem wymagającym analizy była częstotliwość sterowania robota. Z przyczyn technologicznych jej maksymalna wartość była ograniczona do 10 Hz. W środowisku CoppeliaSim standardowy czas pojedynczego kroku symulacji wynosi 50 ms, co teoretycznie umożliwiałoby sterowanie robotem z częstotliwością 20 Hz. Jednakże, w celu ograniczenia maksymalnego prądu pobieranego przez układ napędowy zdecydowano się na przesunięcie chwili przekazania sterowania na serwomechanizmy odpowiadające za ruch stawów biodrowych względem chwili przekazania sterowania na serwomechanizmy odpowiadające za ruch stawów kolanowych. Miało to znaczący pozytywny wpływ na wygładzenie szczytów chwilowego poboru prądu. W praktyce wymagało to wykonania dodatkowego kroku symulacji odwzorowującego to opóźnienie, co ograniczyło efektywną częstotliwość sterowania do 10 Hz. Dodatkowo przeprowadzono również eksperymenty dla częstotliwości 5 Hz w celu sprawdzenia, czy niższa częstotliwość sterowania okaże się wystarczająca do wyuczenia stabilnego chodu robota.

	Częstotliwość 10 Hz		Częstotliwość 5 Hz	
	Metryka ± odch. std	Średnia prędkość (cm/s)	Metryka ± odch. std	Średnia prędkość (cm/s)
Nauka 1	$112{,}85 \pm 1{,}44$	15,07	$71{,}19 \pm 2{,}64$	8,48
Nauka 2	$118{,}65 \pm 1{,}02$	17,05	$72{,}86 \pm 8{,}29$	5,59
Nauka 3	$119{,}15 \pm 0{,}70$	17,76	$52{,}27 \pm 19{,}52$	5,83
Średnia	116,88	16,63	65,44	6,63

Tabela 6.2. Wybór częstotliwości sterowania algorytmu

Wyniki przedstawione w tabeli 6.2 pokazują, że częstotliwość 10 Hz była wystarczająca do uzyskania płynnego i stabilnego sterowania robotem. Obniżenie częstotliwości do 5 Hz miało natomiast wyraźnie negatywny wpływ zarówno na jakość generowanego chodu, jak i osiąganą prędkość ruchu robota. Częstotliwość 5 Hz umożliwiała agentom wykształcenie podstawowej strategii poruszania się, jednak generowany chód był znacznie mniej efektywny oraz wyraźnie wolniejszy. Prawdopodobną przyczyną tego zjawiska była zbyt mała częstotliwość aktualizacji sygnałów sterujących, ograniczająca możliwość szybkiego

reagowania na zmiany orientacji robota oraz płynnego zadawania kolejnych pozycji poszczególnych stawów. Na podstawie przeprowadzonych eksperymentów zdecydowano się na wykorzystanie częstotliwości sterowania wynoszącej 10 Hz w dalszych etapach pracy.

6.9.3. Wybór długości sekwencji wejściowej n

Ostatnim hiperparametrem systemu sterowania poddanym szczegółowej analizie była długość sekwencji wejściowej n, określająca liczbę poprzednich kroków czasowych uwzględnianych w obserwacji przekazywanej do sieci aktora. Parametr ten odpowiada wartości n występującej we wzorach (16) oraz (17).

Teoretycznie zwiększanie wartości parametru n powinno prowadzić do dostarczenia sieci bardziej kompletnej informacji o stanie dynamicznym układu. Dłuższa historia obserwacji umożliwia bowiem częściowe odtworzenie informacji o ruchu robota oraz zmianach jego orientacji w czasie. Jednocześnie wzrost długości sekwencji wejściowej powoduje istotne zwiększenie rozmiaru przestrzeni obserwacji (dla sieci realizowanych jako perceptron wielowarstwowy), co utrudnia proces uczenia oraz wymaga stosowania większych sieci neuronowych.

W projektowanym systemie problem ten miał szczególne znaczenie ze względu na konieczność wykonywania inferencji docelowej sieci polityki na mikrokomputerze o ograniczonych możliwościach obliczeniowych. Z tego względu konieczne było znalezienie kompromisu pomiędzy ilością informacji przekazywanej do sieci a złożonością obliczeniową całego modelu.

	$n = 1$		$n = 2$		$n = 3$	
	Metryka ± odch. std	Średnia prędkość (cm/s)	Metryka ± odch. std	Średnia prędkość (cm/s)	Metryka ± odch. std	Średnia prędkość (cm/s)
Nauka 1	93,59 ± 9,39	15,31	112,85 ± 1,44	15,07	105,8 ± 6,62	15,43
Nauka 2	104,03 ± 0,59	17,05	118,65 ± 1,02	17,05	111,82 ± 0,56	17,74
Nauka 3	93,80 ± 5,40	12,86	119,15 ± 0,70	17,76	111,22 ± 1,74	16,73
Średnia	97,14	16,18	116,88	16,63	109,61	16,63

Tabela 6.3. Wybór długości sekwencji wejściowej n

Wyniki przedstawione w tabeli 6.3 wskazują, że wykorzystanie wyłącznie pojedynczego kroku historii obserwacji ($n = 1$) miało znaczący, negatywny, wpływ na stabilność i efektywność uzyskanego chodu robota. Wytworzonym w jej trakcie politykom brakowało płynności. Wynik ten sugeruje, że dostarczenie na wejście sieci jedynie jednej, poprzedniej, obserwacji nie zapewniło wystarczającej informacji do podjęcia prawidłowej decyzji co do kolejnej akcji.

Najlepsze rezultaty uzyskano dla konfiguracji wykorzystującej dwa poprzednie kroki czasowe ($n = 2$). W tym przypadku osiągnięto zarówno najwyższą wartość metryki, jak i największą stabilność procesu uczenia. Uzyskane wyniki wskazują, że taka długość sekwencji stanowiła wystarczający kompromis pomiędzy ilością informacji historycznej a złożonością przestrzeni obserwacji.

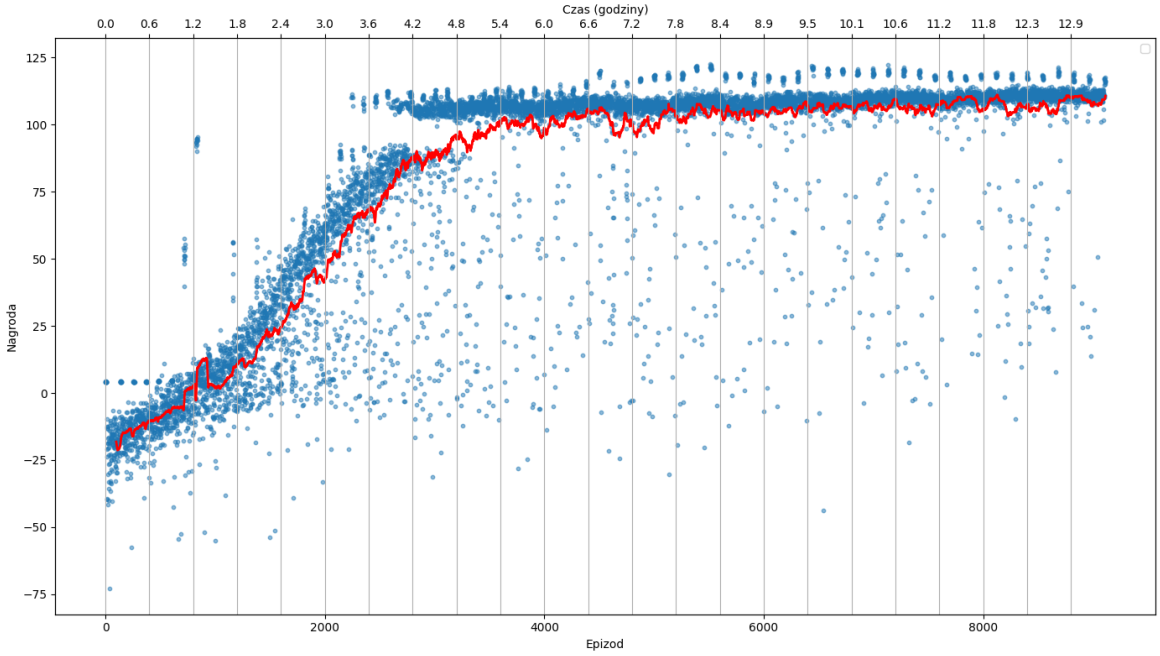
Dalsze zwiększenie długości sekwencji do $n = 3$ nie prowadziło już do poprawy wyników. Zarówno pod względem mierzonej metryki jak i wizualnej oceny chodu, wyniki były porównywalne lub gorsze od tych uzyskiwanych dla długości sekwencji $n = 2$. Prawdopodobną przyczyną tego zjawiska był wzrost wymiarowości przestrzeni wejściowej, utrudniający sieci wyodrębnienie najistotniejszych informacji. W praktyce skuteczne wykorzystanie dłuższych sekwencji mogłoby wymagać zastosowania większych modeli neuronowych lub bardziej zaawansowanych architektur rekurencyjnych.

6.10. Porównanie nauczonej trajektorii chodu z trajektorią referencyjną

Po zbadaniu jakie są optymalne wartości kluczowych hiperparametrów kilkukrotnie powtórzono proces uczenia agenta, a następnie porównano rezultaty najlepszej, uzyskanej w ich trakcie, sieci polityki z wynikami osiąganymi przez opracowaną wcześniej trajektorię referencyjną. Na rysunku 6.11 przedstawiono przebieg procesu uczenia agenta prowadzący do uzyskania najlepszej sieci polityki. Pojedyncze epizody treningowe oznaczono na nim za pomocą niebieskich punktów, natomiast czerwona krzywa obrazuje średnią kroczącą wyznaczoną z okna stu epizodów. Wykres przedstawia stopniowy wzrost wartości nagrody uzyskiwanej w kolejnych epizodach treningowych, co świadczy o skutecznym procesie optymalizacji polityki sterowania. Dodatkowo na wykresie można dostrzec epizody walidacyjne przeprowadzane po każdej epoce. Przyjmują one postać pionowych skupisk punktów, układających się nieznacznie powyżej głównego trendu wyników z epizodów treningowych. Zjawisko to wynika z faktu, że w fazie ewaluacji agent działał w sposób deterministyczny, każdorazowo wybierając optymalną akcję, podczas gdy w fazie treningowej jego decyzje miały charakter stochastyczny w celu wspierania eksploracji środowiska.

W celu porównania wyuczonej w ten sposób strategii chodu ze strategią referencyjną, dla każdego z tych podejść przeprowadzono serię 10 epizodów ewaluacyjnych. Długość każdego epizodu wynosiła 25 sekund, co przy przyjętej częstotliwości 10 Hz odpowiada 250 krokom decyzyjnym. Jako kryteria oceny przyjęto średnią wartość skumulowanej nagrody uzyskiwanej na przestrzeni całego epizodu oraz średnią prędkość ruchu robota. Ta ostatnia była obliczana dla każdej próby jako iloraz całkowitego przemieszczenia robota wzdłuż osi x do czasu trwania epizodu. Uśrednione wyniki ze wszystkich przeprowadzonych prób testowych zaprezentowano w tabeli 6.4.

Uzyskane wyniki wskazują na znaczącą przewagę modelu wytrenowanego z wykorzystaniem uczenia przez wzmacnianie względem ręcznie zaprojektowanej trajektorii



Rysunek 6.11. Wizualizacja procesu nauki agenta przedstawiająca nagrodę zdobytą w kolejnych epizodach treningowych i walidacyjnych. Górna oś przedstawia godzinę liczoną od startu nauki.

Metryka	**Algorytm RL**	**Trajektoria referencyjna**
Średnia wartość metryki	119,09	24,20
Średnia prędkość $\left(\frac{cm}{s}\right)$	19,31	5,46

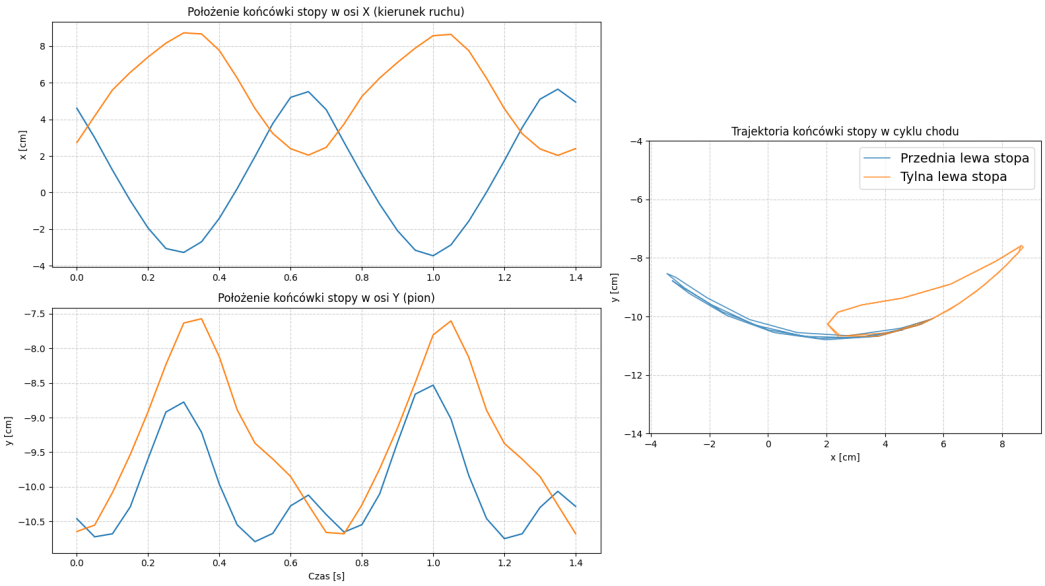
Tabela 6.4. Porównanie wyników algorytmu RL oraz trajektorii referencyjnej

referencyjnej. Agent RL osiągnął zarówno wyższą wartość funkcji nagrody, jak i ponad trzykrotnie większą średnią prędkość poruszania się robota.

W celu dokładniejszej analizy nowej strategii chodu, na rysunku 6.12 zestawiono przebiegi ruchu końcówki stopy dla przedniej lewej (kolor niebieski) oraz tylnej lewej (kolor pomarańczowy) nogi robota. Wykresy umieszczone po lewej stronie przedstawiają zmianę położenia stóp w funkcji czasu odpowiednio dla osi X (zgodnej z kierunkiem ruchu, górny wykres) oraz osi Y (dolny wykres). Z kolei wykres po prawej stronie ilustruje pełną trajektorię ruchu w płaszczyźnie XY. Przedstawione na osiach wartości współrzędnych, zachowujące konwencję znaków zdefiniowaną na rysunku 5.7, wyznaczono poprzez rozwiązanie prostego zadania kinematyki (omówionego w podrozdziale 5.4.1) dla wartości kątów w stawach biodrowym i kolanowym rejestrowanych na bieżąco podczas symulacji.

Przedstawione przebiegi obrazują dwa pełne cykle chodu. Na ich podstawie można wywnioskować, że pojedynczy wyuczony cykl trwa około 0,7 sekundy. Z wykresów położenia względem osi X jednoznacznie wynika, że ruchy przedniej i tylnej stopy realizowane są w przeciwfazie – w momencie, gdy jedna z kończyn przemieszcza się do przodu, druga

porusza się do tyłu. Interesującą anomalią, stanowiącą efekt optymalizacji przeprowadzonej przez algorytm, jest przebieg ruchu przedniej stopy wzdłuż osi Y. Jej cykl ruchu w tej płaszczyźnie został wyraźnie podzielony przez agenta na dwie odrębne części (fazy), co stanowi unikalną cechę wyuczonej polityki.



Rysunek 6.12. Trajektoria końcówki przedniej i tylnej lewej stopy w cyklu chodu, wyznaczona z prostego zadania kinematyki na podstawie kątów w stawach zarejestrowanych w symulacji.

Zestawienie wyuczonej strategii z przebiegami zilustrowanymi na rysunku 6.8 pokazuje istotne różnice względem manualnie zaprojektowanego wzorca. Algorytm uczenia przez wzmacnianie był w stanie samodzielnie odnaleźć bardziej efektywny sposób generowania ruchu, lepiej dostosowany do dynamiki modelu robota oraz przyjętej funkcji nagrody. Podczas tworzenia trajektorii referencyjnej przyjęto założenie, że obie nogi będą realizować identyczną ścieżkę ruchu, przesuniętą w fazie jedynie o pół okresu. Zachowanie wypracowane przez algorytm udowadnia jednak, że było to zbyt duże uproszczenie, a wyższa efektywność wymaga bardziej złożonej, asymetrycznej koordynacji poszczególnych kończyn. Przedstawione wyniki oraz przeprowadzona analiza potwierdzają, że algorytm skutecznie wygenerował stabilną i optymalną strategię chodu w środowisku symulacyjnym. Stanowi to solidny fundament do przeprowadzenia kolejnego, kluczowego etapu prac, jakim jest weryfikacja wyuczonej polityki sterowania na rzeczywistej platformie sprzętowej robota.

7. Przeniesienie modelu z symulacji do fizycznego robota

Przeniesienie polityki sterowania wyuczonej w środowisku symulacyjnym na rzeczywistego robota stanowi istotne wyzwanie w procesie uczenia robotów z wykorzystaniem metod uczenia przez wzmacnianie. W literaturze zagadnienie to określane jest mianem *Sim-to-Real transfer* (*Sim2Real*) i odnosi się do przeniesienia zachowania wyuczonego w symulacji do rzeczywistego układu. Główną przeszkodą w tym procesie jest rozbieżność pomiędzy właściwościami środowiska symulacyjnego a rzeczywistym światem (ang. *reality gap*). Wynika ona między innymi z niedokładności modelu dynamiki, parametrów fizycznych robota, sposobu modelowania kontaktu z podłożem i tarcia, a także z niedokładnego odwzorowania pracy napędów. W konsekwencji polityka, która osiąga stabilny chód w symulacji, nie musi zachowywać się w analogiczny sposób po zaimplementowaniu na fizycznym robocie. Z tego względu skuteczne przeniesienie modelu wymaga odpowiedniego przygotowania zarówno środowiska symulacyjnego, jak i warstwy sterowania rzeczywistego robota. W niniejszym rozdziale przedstawiono proces przeniesienia wyuczonej polityki na skonstruowaną platformę fizyczną oraz wyniki jej działania w rzeczywistych warunkach. Początek rozdziału poświęcono jednak krótkiemu opisowi realizacji systemu sterowania rzeczywistego robota.

7.1. Inferencja sieci polityki na Raspberry Pi Pico

Po zakończeniu procesu uczenia modelu w środowisku symulacyjnym konieczne było przeniesienie wytrenowanej polityki sterowania na rzeczywistą platformę sprzętową. W projekcie wykorzystano mikrokontrolery rodzaju Raspberry Pi Pico. Ze względu na ograniczone zasoby pamięciowe oraz obliczeniowe mikrokontrolera i wynikającą z tego niedostępność bibliotek wspierających inferencje modeli neuronowych na tych kontrolerach zdecydowano się na implementację własnego mechanizmu inferencji sieci neuronowej.

Wagi modelu wytrenowanego przy użyciu biblioteki PyTorch zostały zapisane w pliku formatu JSON, z podziałem na kolejne warstwy sieci. Zastosowane rozwiązanie pozwoliło na łatwe przeniesienie parametrów modelu do środowiska MicroPython. Na potrzeby projektu zaimplementowano własną klasę odpowiedzialną za wykonywanie inferencji sieci neuronowej na mikrokontrolerze. Klasa odpowiada za wczytanie zapisanych wcześniej wag sieci z pliku JSON, przygotowanie obserwacji wejściowej, wykonanie propagacji w przód za pomocą operacji macierzowych i przekształcenie wyjść sieci do przestrzeni sterowań poszczególnych serwomechanizmów.

7.1.1. Biblioteka ulab

W realizacji inferencji bardzo przydatne okazało się wykorzystanie biblioteki ulab[32], będącej lekką implementacją biblioteki NumPy przeznaczoną dla środowiska MicroPython. Biblioteka ta dostarcza podstawowe operacje algebraiczne, takie jak mnożenie macierzy, operacje na wektorach czy funkcje matematyczne, przy zachowaniu niewiel-

kiego zużycia pamięci. Kluczową zaletą biblioteki ulab jest implementacja krytycznych obliczeń w języku C, co znacząco przyspiesza wykonywanie operacji numerycznych względem standardowego interpretera Pythona. W projekcie wykorzystano obraz firmware MicroPython zawierający wbudowaną bibliotekę ulab.

7.1.2. Porównanie wydajności inferencji

W celu oceny wydajności przeprowadzono pomiary średniego czasu pojedynczej inferencji modelu. Porównano działanie implementacji wykorzystującej standardowy interpreter MicroPython oraz wersję korzystającą z biblioteki ulab, zarówno dla mikrokontrolera Raspberry Pi Pico 1 i Raspberry Pi Pico 2.

Aby zapewnić miarodajność i powtarzalność wyników, we wszystkich badanych konfiguracjach wykorzystano identyczną metodykę testową. Skrypt pomiarowy każdorazowo przeprowadzał dziesięciokrotną inferencję modelu dla losowo wygenerowanego wektora wejściowego, a ostateczny wynik stanowił średnią arytmetyczną zmierzonych czasów wykonania. Ponieważ wysokopoziomowa logika programu była taka sama dla każdego przypadku, zaobserwowane rozbieżności wynikają wyłącznie z różnic w architekturze sprzętowej mikrokontrolerów oraz z niskopoziomowego sposobu realizacji obliczeń matematycznych.

	Raspberry Pi Pico 1	**Raspberry Pi Pico 2**
MicroPython bazowy	113,6155 ms	54,5787 ms
MicroPython ulab	10,3577 ms	1,5531 ms

Tabela 7.1. Porównanie czasu inferencji modelu neuronowego

Uzyskane wyniki wskazują na bardzo duży wpływ zastosowania biblioteki ulab na wydajność wykonywania inferencji sieci neuronowej. W bazowej wersji MicroPythona inferencja opierała się na standardowych strukturach danych, takich jak listy, oraz iteracyjnych pętlach, których wykorzystywanie wiąże się z dużym narzutem czasowym. Wykorzystanie biblioteki ulab umożliwiło znaczące przyspieszenie obliczeń – wykonuje ona operacje algebraiczne na tablicach bezpośrednio w zoptymalizowanym, skompilowanym języku C. Tłumaczy to drastyczny, wielokrotny spadek czasu inferencji w stosunku do czystego interpretera. W przypadku płytki Raspberry Pi Pico 2 udało się uzyskać ponad trzydziestokrotne skrócenie czasu inferencji.

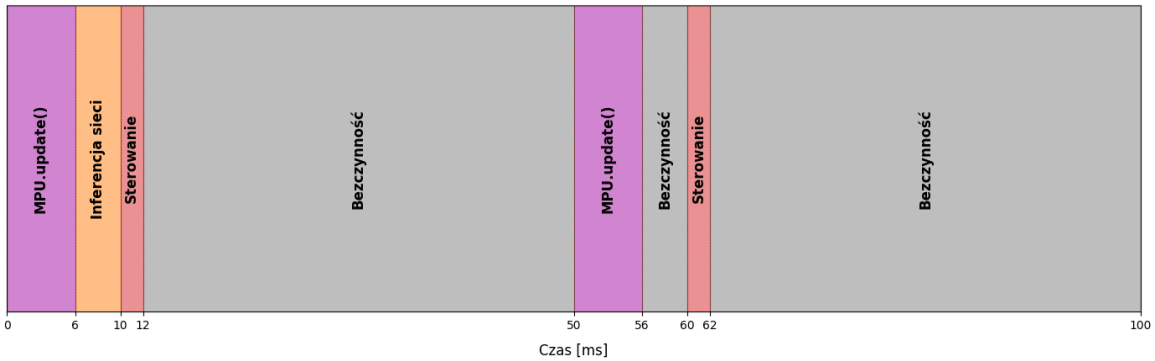
Należy podkreślić, że przeprowadzenie tej optymalizacji było bardzo ważne dla powodzenia całego projektu. Z wyników eksperymentów przedstawionych w podrozdziale 6.9.2 wynika, że docelowa częstotliwość głównej pętli sterowania powinna wynosić 10 Hz. Czas inferencji na poziomie ponad 54 ms zajmowałby większość tego okna, pozostawiając niewiele marginesu na obsługę czujników i wykonywanie innych zadań związanych ze

sterowaniem, natomiast czas rzędu 113 ms (w przypadku Pico 1) całkowicie przekraczałby dopuszczalny limit. Bez zastosowania biblioteki *ulab* utrzymanie założonego reżimu czasu byłoby niemożliwe. Konieczne stałoby się wówczas pójście na niepożądane kompromisy, takie jak uproszczenie architektury sieci neuronowej, co mogłoby obniżyć jakość wyuczonej polityki, lub zmniejszenie częstotliwości sterowania, co miałoby negatywny wpływ na stabilność chodu robota.

Na podstawie przeprowadzonych pomiarów ostatecznie zdecydowano się na wykorzystanie mikrokontrolera Raspberry Pi Pico w wersji 2 jako docelowej platformy sprzętowej robota. Decyzja ta została podjęta ze względu na znacznie wyższą wydajność obliczeniową względem Raspberry Pi Pico 1, co umożliwiło szybsze wykonywanie inferencji modelu sterowania w czasie rzeczywistym i zapewniło większy komfort przy planowaniu wszystkich zadań wykonywanych na mikrokontrolerze. Należy jednak zaznaczyć, że już na etapie projektowania zakładano wykorzystanie mikrokontrolera z rodziny Raspberry Pi Pico, a wybór konkretnej wersji został podjęty dopiero po przeprowadzeniu testów wydajnościowych. Ponieważ płytki Raspberry Pi Pico 1 i Raspberry Pi Pico 2 są ze sobą kompatybilne pod względem mechanicznym oraz programowym, możliwe było odłożenie decyzji o wyborze konkretnej wersji na późniejszy etap projektu.

7.2. Cykl działania Raspberry Pi Pico

Cykl pracy mikrokontrolera w oknie o długości 100 ms przedstawiono na rysunku 7.6. Cykl rozpoczyna się od wykonania metody `MPU.update()`, która odczytuje dane z żyroskopu i akcelerometru MPU6050, a następnie estymuje aktualne kąty przechyłu ψ_x i pochylenia ψ_y robota. Do wyznaczania tych wartości zastosowano filtr komplementarny, łączący szybkie, lecz podatne na dryf dane z żyroskopu z bardziej stabilnymi pomiarami akcelerometru. Testy wykazały, że dokładność estymacji poprawia się przy częstszej aktualizacji danych, dlatego metoda `MPU.update()` wykonywana jest dwukrotnie podczas jednego cyklu - na początku oraz po upływie 50 ms. Wykonanie metody `MPU.update()` trwa około 6 milisekund.



Rysunek 7.1. Cykl pracy mikrokontrolera

Po zakończeniu pracy czujnika MPU6050 wykonywana jest inferencja sieci neuronowej. Przed rozpoczęciem propagacji w przód dane wejściowe są normalizowane do założonego zakresu wartości. Następnie sieć neuronowa generuje akcje sterujące dla serwomechanizmów. Przygotowanie danych, ich normalizacja i propagacja sieci neuronowej trwają około 4 milisekund. Z przyczyn praktycznych zdecydowano się na przesunięcie w czasie aktualizacji sygnałów sterujących serwami kolanowymi i biodrowymi. W związku z tym, jedynie połowa wyznaczonych wartości zadanych przekazywana jest do serwomechanizmów natychmiast po inferencji. Czas trwania odczytów z MPU6050 oraz obliczeń sieci neuronowej nie jest stały, w związku z czym zapisywany jest dokładny moment cyklu w którym serwomechanizmy poruszające stawami kolanowymi otrzymały kolejne pozycje zadane.

Po pierwszym etapie sterowania mikrokontroler przechodzi w stan oczekiwania do momentu osiągnięcia 50 ms cyklu, kiedy ponownie wykonywana jest metoda `MPU.update()`. Następnie układ ponownie oczekuje, by dokładnie 50 ms po zadaniu pozycji stawów kolanowych zrobić to samo dla serwomechanizmów odpowiedzialnych za ruch stawów biodrowych. Po zakończeniu wszystkich operacji mikrokontroler oczekuje na rozpoczęcie kolejnego cyklu.

7.3. Problemy z przeniesieniem polityki

Przedstawione w poprzednich podrozdziałach wyniki dowiodły, że zaprojektowana architektura systemu uczącego, dobrana metryka jakości chodu oraz wybrane eksperymentalnie hiperparametry algorytmu PPO pozwalają na wygenerowanie skutecznej strategii chodu w środowisku symulacyjnym. Osiągnięcie tego etapu stanowiło niezbędny fundament całego projektu. Należy jednak pamiętać, że przeniesienie wyuczonej polityki bezpośrednio z idealnego środowiska symulacyjnego na fizyczną platformę sprzętową (tzw. transfer *Sim-to-Real*) jest praktycznie zawsze dużym wyzwaniem. Wynika to z istnienia tzw. luki rzeczywistości (ang. *reality gap*), czyli niemożliwych do idealnego zamodelowania zjawisk fizycznych i opóźnień.

Pierwsze uruchomienie wytrenowanej polityki na fizycznym robocie wykazało znaczące różnice pomiędzy zachowaniem układu w symulacji a jego działaniem w rzeczywistych warunkach. Robot wykonywał ruchy o dużych amplitudach i sprawiał wrażenie niestabilnego. Stopy podczas chodu silnie uderzały o podłoże, a następnie często się po nim ślizgały. Zaobserwowane ruchy różniły się również od tych uzyskiwanych w środowisku symulacyjnym. W związku z tym przeprowadzono dokładną analizę działania systemu sterowania rzeczywistego robota.

Analiza kolejnych wartości wyjściowych sieci neuronowej wykazała, że polityka nie nauczyła się płynnego sterowania pozycją poszczególnych stawów, lecz wykorzystywała specyficzną strategię polegającą na naprzemiennym kierowaniu stawem w przeciwne strony. W kolejnych krokach czasowych dla danego stawu mogła być generowana przykła-

dowo sekwencja wartości 1, 1, −1, −1, 1. W praktyce oznaczało to, że sieć w każdym kroku decydowała przede wszystkim o kierunku dalszego ruchu stawu, a nie o jego docelowym położeniu. Strategia ta była skuteczna w środowisku symulacyjnym, jednak jej poprawne odtworzenie na fizycznym robocie wymagałoby bardzo dokładnego odwzorowania dynamiki i charakterystyki zastosowanych serwomechanizmów. W rzeczywistym układzie występują natomiast między innymi opóźnienia, bezwładność oraz różnice pomiędzy zadanym a rzeczywiście osiąganym położeniem stawu. W rezultacie niewielkie różnice pomiędzy modelem symulacyjnym a rzeczywistym mechanizmem mogły prowadzić do znacznych zmian w zachowaniu całego robota.

Dodatkowym potencjalnym źródłem różnic pomiędzy symulacją a rzeczywistym robotem była informacja o orientacji korpusu przekazywana do sieci neuronowej. W środowisku symulacyjnym wartości pochylenia i przechylenia były wyznaczane bezpośrednio na podstawie stanu modelu, a więc stanowiły dokładną informację o orientacji robota. W rzeczywistym układzie wartości te są natomiast estymowane na podstawie pomiarów czujnika MPU-6050, przez co mogą być obarczone szumem pomiarowym oraz błędem. W konsekwencji polityka uczona na podstawie idealnych wartości orientacji mogła wykorzystywać informacje, których dokładność nie była możliwa do uzyskania na fizycznym robocie.

Warto w tym miejscu podkreślić, że zaobserwowane problemy nie podważały poprawności dotychczasowej metodologii. Nie wymagały one zmiany wybranego algorytmu uczenia przez wzmacnianie, środowiska symulacyjnego ani żadnego z wcześniej dobranych kluczowych hiperparametrów. Aby wypełnić lukę rzeczywistości, zdecydowano się na zastosowanie klasycznych w dziedzinie uczenia przez wzmacnianie technik: regularyzacji funkcji jakości (ang. *regularization*), randomizacji środowiska (ang. *domain randomization*) oraz fizycznych poprawek mechanicznych.

Pierwszą z wprowadzonych zmian było dodanie składowej funkcji nagrody ograniczającej gwałtowne zmiany wartości akcji. Zastosowano składnik postaci:

$$p_{\text{regularyzacja}} = -\left|\Lambda(k) - \Lambda(k-1)\right|, \tag{18}$$

gdzie $\Lambda(k)$ oraz $\Lambda(k-1)$ oznaczają wektory akcji wygenerowane w bieżącym i poprzednim kroku czasowym. Składnik ten zachęca politykę do generowania bardziej płynnych zmian sterowania, ograniczając możliwość powstawania gwałtownych skoków sterowania i niejako wymusza sterowanie polegające na zadawaniu dokładnych pozycji stawów w kolejnym kroku czasowym. Wprowadzenie tej regularyzacji miało na celu uzyskanie strategii łatwiejszej do odtworzenia przez rzeczywiste serwomechanizmy.

Drugą zmianą było zmniejszenie udziału składowej odpowiadającej za prędkość poruszania się robota w funkcji nagrody. W pierwotnym procesie uczenia polityka miała silną motywację do maksymalizowania osiąganej prędkości, nawet kosztem jakości i stabilności chodu. W przypadku środowiska symulacyjnego takie zachowanie mogło być korzystne z

punktu widzenia przyjętej funkcji celu, jednak podczas przeniesienia na rzeczywistą konstrukcję prowadziło do powstawania gwałtownych ruchów oraz zwiększało ryzyko utraty stabilności. Zmniejszenie wagi tej składowej pozwoliło nadać większe znaczenie jakości i stabilności ruchu. W przypadku rzeczywistego robota uznano bowiem, że uzyskanie nieco mniejszej prędkości chodu jest akceptowalnym kompromisem, jeżeli jednocześnie prowadzi do bardziej stabilnego i bezpiecznego zachowania.

W celu zwiększenia odporności polityki na niedokładności estymacji orientacji wprowadzono do środowiska symulacyjnego zakłócenia odwzorowujące charakterystykę rzeczywistych pomiarów. Podczas uczenia do wartości pochylenia i przechylenia zaczęto dodawać losowy szum pomiarowy oraz losowy błąd, którego wartość pozostawała stała w obrębie pojedynczego epizodu. Dzięki temu sieć podczas treningu otrzymywała nieidealną informację o orientacji i była zmuszana do podejmowania decyzji mimo występujących błędów estymacji. Zabieg ten miał ograniczyć zależność wyuczonej polityki od dokładnych wartości orientacji i zwiększyć jej odporność na różnice pomiędzy symulacją a rzeczywistym układem.

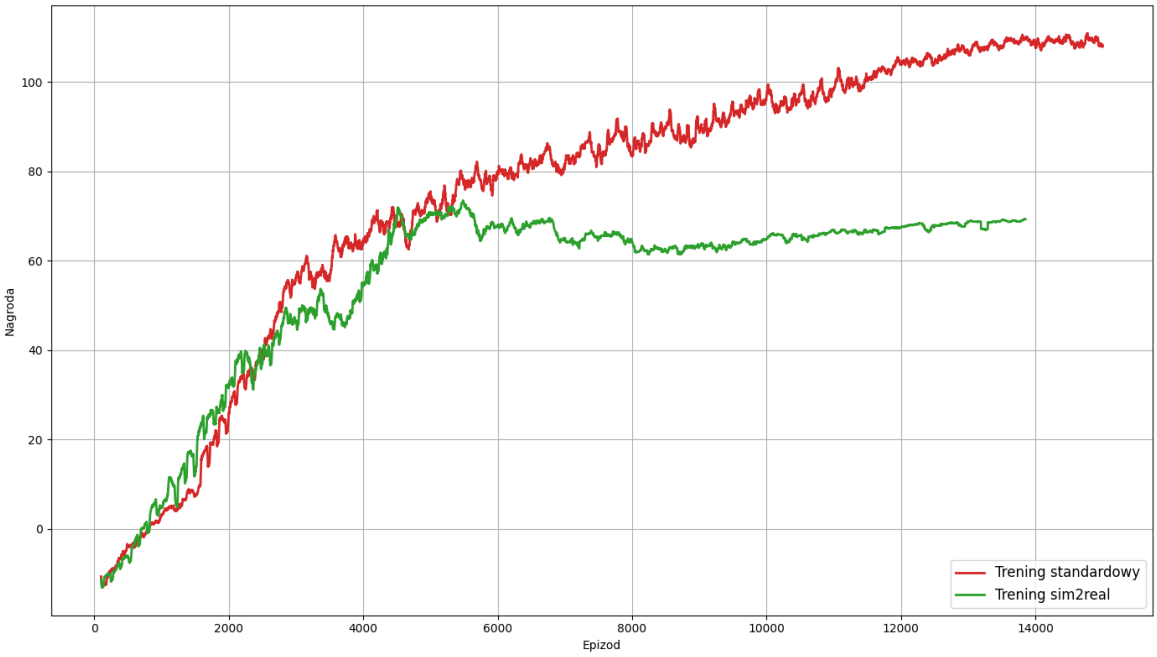
Ostatnią wprowadzoną zmianą była modyfikacja konstrukcji stóp robota. Pierwotnie zastosowano kuliste końcówki wykonane z twardej gumy. Podczas pierwszych prób wykazały one jednak niewystarczającą przyczepność do podłoża, a dodatkowo charakteryzowały się małą zdolnością do tłumienia energii podczas kontaktu stopy z podłożem. W celu poprawy właściwości kontaktu zaprojektowano dedykowaną konstrukcję stopy, a następnie wykonano ją techniką druku 3D z materiału TPU. W dolnej części stopy zastosowano specjalnie zaprojektowane wypustki zwiększające tarcie pomiędzy stopą a podłożem. Dodatkowo w stopie stworzono otwór umożliwiający amortyzowanie kontaktu z podłożem, dzięki czemu możliwe było częściowe ograniczenie gwałtowności kontaktu stopy z podłożem.

Rysunek 7.2. Model 3D (po lewej) oraz zdjęcie (po prawej) gotowej, dedykowanej stopy robota

Po wprowadzeniu opisanych zmian, proces uczenia powtórzono z zachowaniem identycznej metodologii i parametrów treningowych jak w poprzednich rozdziałach. Wyniki

uzyskane w środowisku symulacyjnym dla nowej polityki potwierdziły słuszność dotychczasowych założeń – agent osiągnął niemal identyczną zbieżność i stabilność procesu nauki. Zgodnie z oczekiwaniami, ze względu na zwiększone kary za gwałtowne ruchy oraz dodatkowy składnik odpowiadający za regularyzację metryki jakości, ostateczna polityka w symulacji cechowała się jedynie nieznacznie niższą maksymalną prędkością chodu. Strata ta była jednak marginalna w porównaniu do ogromnego zysku w postaci poprawy jakości transferu polityki do rzeczywistego robota.

Wizualne potwierdzenie tych wniosków przedstawiono na rysunku 7.3. Wykres ten zestawia krzywą uczenia, wyrażoną w postaci średniej kroczącej nagrody z kolejnych epizodów, dla zmodyfikowanego środowiska oraz dla polityki bazowej (której przebieg zaprezentowano wcześniej na rysunku 6.11). Kształt obydwu krzywych jest w pierwszej części wykresu bardzo zbliżony. Główną zauważalną różnicą jest to, że w końcowej fazie treningu krzywa dla nowego modelu stabilizuje się na wyraźnie niższym poziomie. Zjawisko to jest w pełni oczekiwane i wynika bezpośrednio ze zmniejszenia wagi składowej premiującej wysoką prędkość chodu oraz z wprowadzenia dodatkowej kary regularyzującej, co obniża maksymalną sumę nagród możliwą do uzyskania w trakcie epizodu. Dodatkowo, na nowym przebiegu uwydatnia się wpływ zastosowanej techniki *curriculum learning* – wyraźnie widoczne nagłe spadki nagrody oraz następujące po nich ponowne wzrosty stanowią dowód na to, że agent z powodzeniem adaptował się do rosnącego poziomu trudności.



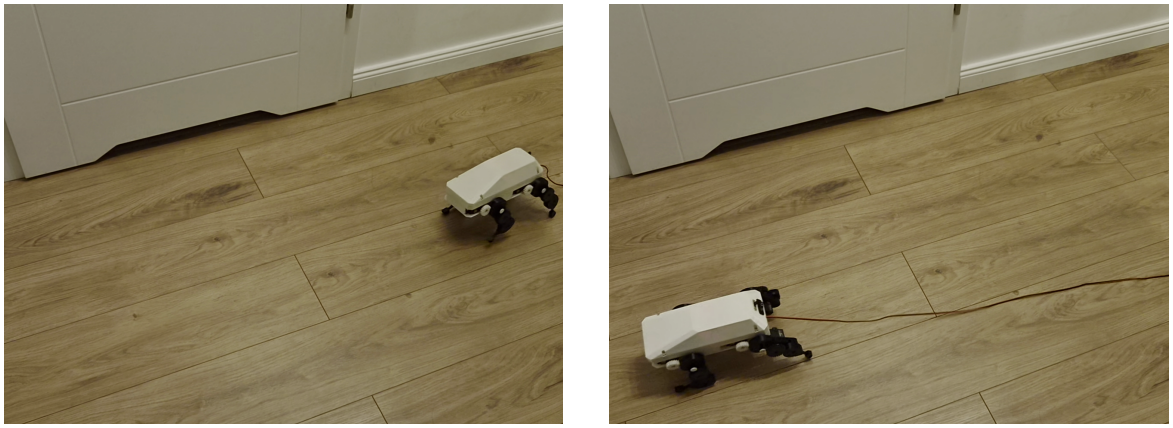
Rysunek 7.3. Porównanie przebiegu nauki modelu bazowego z modelem uczonym po wprowadzeniu modyfikacji mających na celu ograniczenie luki rzeczywistości

Co jednak najważniejsze, zaobserwowano drastyczną poprawę zachowania na rzeczywistym robocie. Ruchy stawów stały się płynne, a nowa konstrukcja stóp zniwelowała

problem poślizgu. Przeprowadzone modyfikacje udowodniły, że stabilne i rygorystyczne przygotowanie modelu w symulacji (opisane w poprzednich rozdziałach) było poprawne, a kluczem do sukcesu na sprzęcie fizycznym było jedynie odpowiednie dostrojenie modelu do ograniczeń sprzętowych i zjawisk świata rzeczywistego.

7.4. Wyniki

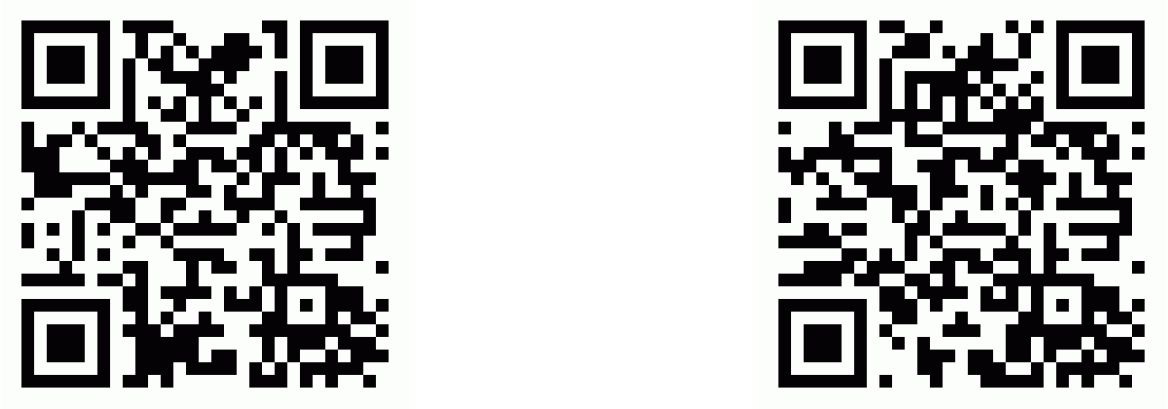
Po wprowadzeniu zmian opisanych w poprzednim podrozdziale, których celem było ograniczenie różnic pomiędzy środowiskiem symulacyjnym a rzeczywistym układem, przeprowadzono testy wytrenowanej ponownie polityki na fizycznym robocie. Uzyskane rezultaty wykazały znaczącą poprawę jakości generowanego chodu. Robot był w stanie stabilnie poruszać się w jednym kierunku, a przeniesiony z symulacji chód charakteryzował się stosunkowo dużą prędkością, stabilnością oraz naturalnym wyglądem wykonywanych ruchów. Co najważniejsze z punktu widzenia celu przeprowadzonych badań, charakter chodu na rzeczywistym sprzęcie był spójny z zachowaniem zaobserwowanym wcześniej w środowisku symulacyjnym. Dowodzi to faktu, że zastosowane techniki transferu *Sim-to-Real* pozwoliły nie tylko wygenerować działającą strategię sterowania, ale umożliwiły również skuteczne przeniesienie konkretnej, wyuczonej strategii chodu z wirtualnego bliźniaka na konstrukcję fizyczną. Robot w trakcie chodu przedstawiony jest na rysunku 7.4.



Rysunek 7.4. Zdjęcia rzeczywistego robota w trakcie chodu

Dla porównania przeprowadzono również test chodu wykorzystującego trajektorię referencyjną opracowaną wcześniej z zastosowaniem podejścia białej skrzynki. W tym przypadku jakość ruchu była wyraźnie niższa. Chód był bardziej toporny, charakteryzował się mniejszą płynnością ruchów oraz znacząco niższą prędkością. Dodatkowo robot nie utrzymywał zadanego kierunku ruchu, lecz stopniowo odchylał się od linii, wzdłuż której powinien się poruszać.

Dodatkowym materiałem dokumentującym uzyskane rezultaty są nagrania przedstawiające działanie rzeczywistego robota. Pierwszy film, do którego link znajduje się pod adresem `https://youtu.be/LTJIqD_ElI8`, zawiera bezpośrednie porównanie obu analizowanych sposobów generowania chodu – sterowania za pomocą wyuczonej polityki oraz sterowania z wykorzystaniem trajektorii referencyjnej. Drugi materiał, do którego link znajduje się pod adresem `https://youtu.be/0cr2MrAnJHc`, stanowi kilkuminutowe podsumowanie całej pracy w języku angielskim. Odnośniki do obu nagrań zostały dodatkowo umieszczone w postaci kodów QR przedstawionych na rysunku 7.5.



Rysunek 7.5. Kod QR z linkiem do filmu demonstrującego chód rzeczywistego robota w dwóch trybach (po lewej) oraz kod QR z linkiem do dłuższego filmu podsumowującego całą pracę w języku angielskim

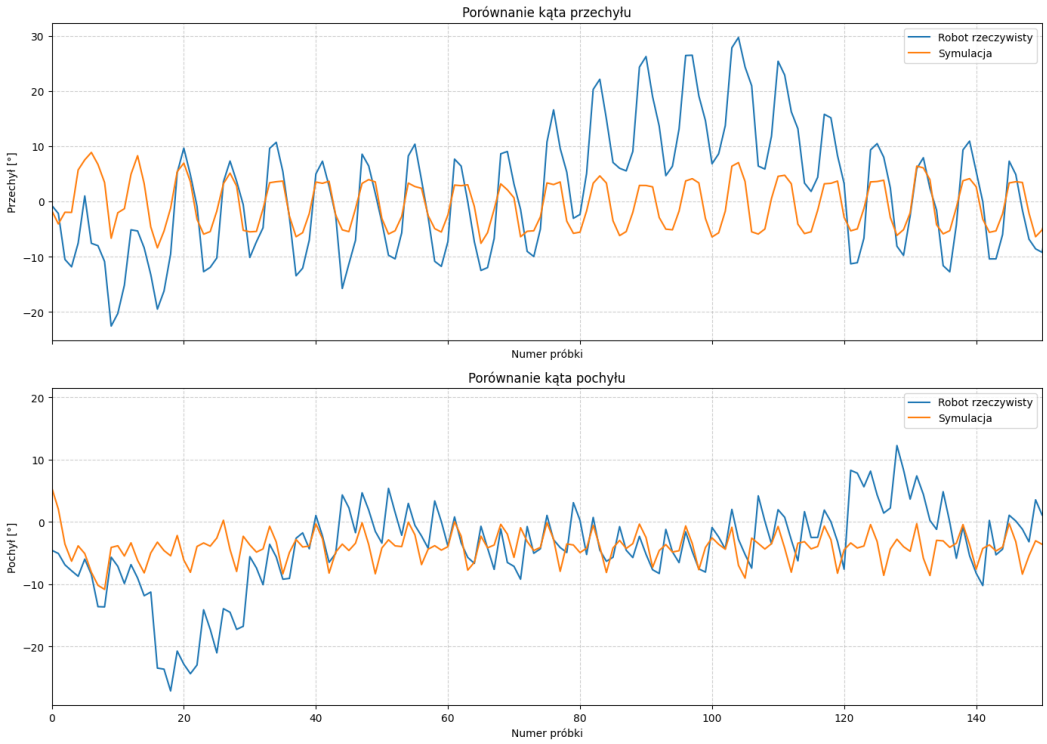
Wyniki implementacji obu podejść do generowania trajektorii chodu wyraźnie wskazują, że technika oparta na uczeniu przez wzmocnienie pozwoliła uzyskać lepszą jakość ruchu niż wcześniej opracowana metoda bazująca na ręcznie zdefiniowanej trajektorii referencyjnej. W przypadku analizowanej platformy podejście wykorzystujące uczenie maszynowe zapewniło przewagę przede wszystkim pod względem stabilności, płynności oraz prędkości chodu.

Jednym ze sposobów oceny uzyskanych rezultatów było porównanie prędkości chodu uzyskanej w symulacji z prędkością osiąganą przez rzeczywistego robota. W przypadku polityki stworzonej przy pomocy techniki uczenia przez wzmacnianie prędkość zmierzona w symulacji wyniosła 18,91 $\frac{cm}{s}$, natomiast estymowana prędkość rzeczywistego robota osiągnęła wartość 18,63 $\frac{cm}{s}$. Otrzymane wartości są bardzo zbliżone, co wskazuje na dobre odwzorowanie charakterystyki chodu podczas przenoszenia polityki z symulacji do rzeczywistego układu. Znacznie większą różnicę zaobserwowano dla trajektorii referencyjnej. Prędkość uzyskana w symulacji wyniosła w tym przypadku 5,46 $\frac{cm}{s}$, podczas gdy rzeczywisty robot osiągnął około 6,73 $\frac{cm}{s}$. Większa prędkość rzeczywistego robota może wynikać z zastosowania zmodyfikowanych stóp zwiększających współczynnik tarcia pomiędzy stopą a podłożem, co ograniczało poślizg podczas wykonywania ruchu. Opisane powyżej wyniki zestawiono w tabeli 7.2.

	Algorytm RL	**Trajektoria referencyjna**
Prędkości liczone w symulacji $\left(\frac{cm}{s}\right)$	18,91	5,46
Estymacja prędkości rzeczywistego robota $\left(\frac{cm}{s}\right)$	18,63	6,73

Tabela 7.2. Zestawienie prędkości chodu osiąganej w symulacji z estymowaną prędkością rzeczywistego robota dla dwóch trybów generowania trajektorii chodu

Drugim sposobem weryfikacji zgodności chodu uzyskanego w symulacji z jego rzeczywistym wykonaniem było porównanie zmian przechyłu i pochylenia korpusu robota. Wartości uzyskane podczas symulacji zestawiono z wartościami estymowanymi na podstawie pomiarów z czujnika MPU-6050 zamontowanego na rzeczywistym robocie. Jak przedstawiono na rysunku 7.6, przebiegi charakteryzują się bardzo dobrą zgodnością pod względem czasowym. Kolejne zmiany orientacji występują w obu przypadkach w tych samych momentach, a ich kolejność oraz czas trwania są niemalże identyczne. Oznacza to, że charakterystyczne ruchy korpusu związane z kolejnymi fazami chodu zostały zachowane podczas przeniesienia wyuczonej polityki na rzeczywisty układ.



Rysunek 7.6. Porównanie przechyłu i pochyłu robota w trakcie chodu – wartości z symulacji oraz estymacje z czujnika MPU-6050

Jednocześnie widoczne są różnice w amplitudzie zmian orientacji. W przypadku rzeczywistego robota przechyły i pochylenia osiągają większe wartości niż w symulacji, co

wskazuje na występowanie dodatkowych efektów dynamicznych nieuwzględnionych w pełni w modelu. Na przebiegach widoczne jest również przesunięcie wartości średniej, utrzymujące się przez kilka sekund. Zjawisko to jest związane prawdopodobnie z błędem estymacji orientacji i stanowi przykład różnicy pomiędzy idealną informacją dostępną w środowisku symulacyjnym a pomiarem uzyskiwanym z rzeczywistego czujnika. Pomimo tych różnic zgodność czasowa obu przebiegów jest zachowana, co potwierdza, że wyuczona polityka generuje na rzeczywistym robocie ruch o charakterystyce odpowiadającej ruchowi uzyskanemu w symulacji.

8. Podsumowanie

W ramach niniejszej pracy zaprojektowano oraz wykonano prototyp czworonożnego robota mobilnego, a następnie nauczono go poruszania się przy wykorzystaniu metod uczenia przez wzmacnianie przeprowadzanych na jego cyfrowym bliźniaku. Głównym celem pracy było przeniesienie polityki sterowania wytrenowanej w środowisku symulacyjnym na rzeczywistą platformę robotyczną wykonaną z tanich i łatwo dostępnych komponentów. Uzyskane wyniki potwierdziły zasadność takiego podejścia i pokazały słuszność wykorzystania technik uczenia przez wzmacnianie do nauki trajektorii ruchu względem podejść opartych o "białe skrzynki".

Do najważniejszych osiągnięć pracy należą:

- zaprojektowanie i wykonanie czworonożnego robota mobilnego z wykorzystaniem łatwo dostępnych i tanich elementów,
- dostosowanie założeń konstrukcyjnych robota w taki sposób, aby możliwe było uzyskanie stabilnego chodu,
- stworzenie cyfrowego bliźniaka odwzorowującego konstrukcję rzeczywistego robota,
- zastosowanie dwustronnego sprzężenia zwrotnego pomiędzy symulacją a konstrukcją fizyczną – wyniki uzyskane w symulacji wpływały na modyfikacje konstrukcji robota, natomiast ograniczenia konstrukcyjne definiowały sposób poruszania się modelu w środowisku symulacyjnym,
- przeprowadzenie procesu uczenia przez wzmacnianie na cyfrowym bliźniaku robota,
- analizę wpływu wybranych hiperparametrów procesu uczenia oraz porównanie różnych algorytmów uczenia przez wzmacnianie,
- eksperymentalne potwierdzenie skuteczności wybranych technik wspomagających proces uczenia, takich jak rozdzielenie obserwacji pomiędzy aktora i krytyka czy zastosowanie *curriculum learning*,
- zaproponowanie i potwierdzenie słuszności metod mających na celu zmniejszenie wpływu różnicy pomiędzy symulacją a rzeczywistym robotem na jakość chodu rzeczywistej konstrukcji,
- implementację wytrenowanej sieci polityki na rzeczywistym robocie oraz eksperymentalne potwierdzenie możliwości transferu polityki sterowania z symulacji do rzeczywistego układu.

Uzyskane rezultaty pokazują, że wykorzystanie cyfrowego bliźniaka może znacząco przyspieszyć proces projektowania i uczenia robotów kroczących, ograniczając liczbę kosztownych i czasochłonnych eksperymentów wykonywanych bezpośrednio na rzeczywistym urządzeniu.

Potencjalnym kierunkiem dalszego rozwoju projektu jest rozszerzenie możliwości ruchowych robota o zdolność skręcania. Obecna konstrukcja może częściowo umożliwiać

realizację takiego manewru, jednak prawdopodobnie wymagałoby to niewielkich modyfikacji mechanicznych, na przykład umożliwienia obrotu przednich kończyn w sposób analogiczny do układu kierowniczego pojazdu. Po wprowadzeniu takich zmian możliwe byłoby przeprowadzenie procesu uczenia w symulacji również dla bardziej złożonych trajektorii ruchu.

Innym potencjalnym kierunkiem rozwoju byłoby zwiększenie możliwości ruchowych robota poprzez dodanie do każdej z jego kończyn trzeciego stopnia swobody. Pozwoliłoby to na realizację bardziej złożonych wzorców chodu oraz zwiększenie możliwości poruszania się po nierównym i zróżnicowanym terenie. Podczas realizacji pracy zaobserwowano ograniczenia wynikające z zastosowania jedynie dwóch stopni swobody na każdą kończynę, w szczególności w zakresie możliwości dostosowania położenia stóp do zmiennego podłoża. Rozbudowanie konstrukcji o dodatkowe stopnie swobody, w połączeniu z wykorzystaniem metod uczenia przez wzmacnianie, mogłoby zatem pozwolić na opracowanie bardziej uniwersalnego systemu zdolnego do uczenia się różnorodnych sposobów poruszania. Przedstawione kierunki rozwoju wskazują, że opracowana platforma oraz przygotowane środowisko symulacyjne mogą stanowić podstawę do dalszych prac nad wykorzystaniem uczenia przez wzmacnianie w sterowaniu robotami kroczącymi.

Bibliografia

[1] R. Brandom, "Boston Dynamics' Spot is leaving the laboratory", Dostęp zdalny (30.05.2026): https://www.theverge.com/2019/9/24/20880511/boston-dynamics-spot-robot-mini-hands-on-lease-buy, 2019.

[2] G. Catavitello, Y. Ivanenko i F. Lacquaniti, "Planar Covariation of Hindlimb and Forelimb Elevation Angles during Terrestrial and Aquatic Locomotion of Dogs", *PLoS ONE*, t. 10, lip. 2015. DOI: 10.1371/journal.pone.0133936.

[3] Holler, Brazda, Dal-Bianco i in., "Kinematic motion analysis of the joints of the forelimbs and hind limbs of dogs during walking exercise regimens", *American Journal of Veterinary Research*, t. 71, nr. 7, s. 734–740, 2010. DOI: 10.2460/ajvr.71.7.734.

[4] H. Taheri i N. Mozayani, "A study on quadruped mobile robots", *Mechanism and Machine Theory*, t. 190, s. 105 448, 2023, ISSN: 0094-114X. DOI: https://doi.org/10.1016/j.mechmachtheory.2023.105448. adr.: https://www.sciencedirect.com/science/article/pii/S0094114X23002197.

[5] B. Dynamics, "Deploying Quadruped Robots to the Automotive Manufacturing Industry", Dostęp zdalny (30.12.2024): https://bostondynamics.com/whitepaper/deploying-quadruped-robots-in-automotive-manufacturing/, 2024.

[6] B. Dynamics, "Implementing Industrial Robots in the Food and Beverage Industry", Dostęp zdalny (30.12.2024): https://bostondynamics.com/whitepaper/implementing-industrial-robots-in-food-and-beverage, 2024.

[7] B. Dynamics, "The Benefits of Flexible Autonomy on Construction Sites", Dostęp zdalny (30.12.2024): https://bostondynamics.com/whitepaper/the-benefits-of-flexible-autonomy-on-construction-sites/, 2024.

[8] G. Times, "China's military robo-wolves unveiled at Airshow China 2024, possessing high mobility and adaptability in complex terrains", https://www.globaltimes.cn/page/202411/132
2024.

[9] B. Dynamics, "Robotics Role in Nuclear Decommissioning", Dostęp zdalny (30.12.2024): https://bostondynamics.com/whitepaper/robotics-role-in-nuclear-decommissioning/, 2024.

[10] D. Vogel, R. Baines, J. Church, J. Lotzer, K. Werner i M. Hutter, *Robust Ladder Climbing with a Quadrupedal Robot*, 2024. arXiv: 2409.17731 [cs.RO]. adr.: https://arxiv.org/abs/2409.17731.

[11] P. Biswal i P. K. Mohanty, "Development of quadruped walking robots: A review", *Ain Shams Engineering Journal*, t. 12, nr. 2, s. 2017–2031, 2021, ISSN: 2090-4479. DOI: https://doi.org/10.1016/j.asej.2020.11.005. adr.: https://www.sciencedirect.com/science/article/pii/S2090447920302501.

[12] H. Xie, Z. Zhang, J. Shang i Z. Luo, "Mechanical Design of A Modular Quadruped Robot - Xdog", *2014 International Conference on Mechatronics, Electronic, Industrial*

and Control Engineering, MEIC 2014, s. 1074–1078, sty. 2014. DOI: 10.2991/meic-14.2014.237.

[13] S. Fredriksson, "Design, Development and Control of a Quadruped Robot", prac. mag., Luleå University of Technology, 2021.

[14] B. G. Katz, "A Low Cost Modular Actuator for Dynamic Robots", prac. mag., Massachusetts Institute of Technology, 2018.

[15] V. Mnih, A. P. Badia, M. Mirza i in., "Asynchronous Methods for Deep Reinforcement Learning", *CoRR*, t. abs/1602.01783, 2016. arXiv: 1602.01783. adr.: http://arxiv.org/abs/1602.01783.

[16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford i O. Klimov, *Proximal Policy Optimization Algorithms*, 2017. arXiv: 1707.06347 [cs.LG]. adr.: https://arxiv.org/abs/1707.06347.

[17] T. Haarnoja, A. Zhou, P. Abbeel i S. Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor", *CoRR*, t. abs/1801.01290, 2018. arXiv: 1801.01290. adr.: http://arxiv.org/abs/1801.01290.

[18] T. Haarnoja, S. Ha, A. Zhou, J. Tan, G. Tucker i S. Levine, *Learning to Walk via Deep Reinforcement Learning*, 2019. arXiv: 1812.11103 [cs.LG]. adr.: https://arxiv.org/abs/1812.11103.

[19] R. Pocius, D. Isele, M. Roberts i D. Aha, "Comparing Reward Shaping, Visual Hints, and Curriculum Learning", *Proceedings of the AAAI Conference on Artificial Intelligence*, t. 32, nr. 1, kw. 2018. DOI: 10.1609/aaai.v32i1.12160. adr.: https://ojs.aaai.org/index.php/AAAI/article/view/12160.

[20] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba i P. Abbeel, "Asymmetric Actor Critic for Image-Based Robot Learning", *CoRR*, t. abs/1710.06542, 2017. arXiv: 1710.06542. adr.: http://arxiv.org/abs/1710.06542.

[21] W. Zhao, J. P. Queralta i T. Westerlund, *Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: a Survey*, 2021. adr.: https://arxiv.org/abs/2009.13303.

[22] M. Mozifian, J. C. G. Higuera, D. Meger i G. Dudek, *Learning Domain Randomization Distributions for Training Robust Locomotion Policies*, 2019. arXiv: 1906.00410 [cs.LG]. adr.: https://arxiv.org/abs/1906.00410.

[23] N. Kau i S. Bowers, "Stanford Pupper: A Low-Cost Agile Quadruped Robot for Benchmarking and Education", *CoRR*, t. abs/2110.00736, 2021. arXiv: 2110.00736. adr.: https://arxiv.org/abs/2110.00736.

[24] R. Kozlica, S. Wegenkittl i S. Hirlander, *Deep Q-Learning versus Proximal Policy Optimization: Performance Comparison in a Material Sorting Task*, czer. 2023. DOI: 10.48550/arXiv.2306.01451.

[25] V. Mnih, K. Kavukcuoglu, D. Silver i in., *Playing Atari with Deep Reinforcement Learning*, 2013. arXiv: 1312.5602 [cs.LG]. adr.: https://arxiv.org/abs/1312.5602.

[26] C. Zhang, O. Vinyals, R. Munos i S. Bengio, *A Study on Overfitting in Deep Reinforcement Learning*, 2018. arXiv: 1804.06893 [cs.LG]. adr.: https://arxiv.org/abs/1804.06893.

[27] K. Kielak, *Importance of using appropriate baselines for evaluation of data-efficiency in deep reinforcement learning for Atari*, 2020. arXiv: 2003.10181 [cs.LG]. adr.: https://arxiv.org/abs/2003.10181.

[28] J. Schulman, S. Levine, P. Moritz, M. I. Jordan i P. Abbeel, *Trust Region Policy Optimization*, 2017. arXiv: 1502.05477 [cs.LG]. adr.: https://arxiv.org/abs/1502.05477.

[29] Y. Duan, X. Chen, R. Houthooft, J. Schulman i P. Abbeel, *Benchmarking Deep Reinforcement Learning for Continuous Control*, 2016. arXiv: 1604.06778 [cs.LG]. adr.: https://arxiv.org/abs/1604.06778.

[30] OpenAI, : C. Berner i in., *Dota 2 with Large Scale Deep Reinforcement Learning*, 2019. arXiv: 1912.06680 [cs.LG]. adr.: https://arxiv.org/abs/1912.06680.

[31] E. Todorov, T. Erez i Y. Tassa, "MuJoCo: A physics engine for model-based control", *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, s. 5026–5033, 2012. adr.: https://api.semanticscholar.org/CorpusID:5230692.

[32] "Micropython ulab", https://github.com/v923z/micropython-ulab.

Wykaz symboli i skrótów

DQN – Deep Q-Learning
PPO – Proximal Policy Optimization
TRPO – Trust Region Policy Optimization
A2C – Advantage Actor-Critic
A3C – Asynchronous Advantage Actor Critic
SAC – Soft Actor-Critic

Spis rysunków

1.1 Rysunek ilustrujący schematycznie ułożenie głównych kości i stawów składających się na kończyny tylne i przednie psa 9
1.2 Przebiegi zmian kątów w stawach kończyn psa podczas różnych form lokomocji. Źródło: tłumaczenie własne na podstawie [2]. 10
1.3 Rysunek przedstawiający nazwy nadane poszczególnych segmentom kończyn robota 11
3.1 Porównanie wybranych komercyjnych konstrukcji robotów czworonożnych: (a) Spot, (b) Unitree Go2 14
3.2 Porównanie wybranych konstrukcji robotów czworonożnych: (a) ANYmal X, (b) Pupper v3 16
3.3 Porównanie wybranych otwartoźródłowych konstrukcji robotów czworonożnych: (a) The Dingo X, (b) Mjukost 17
4.1 Ogólny schemat działania algorytmów uczenia przez wzmacnianie 20
4.2 Poglądowe porównanie sposobu aktualizacji polityki w algorytmach DQN (lewa strona) i TRPO (prawa strona) 22
5.1 Wizualizacja jednej z wcześniejszych koncepcji napędu nogi robota, wykorzystująca siłownik liniowy do poruszania stawem kolanowym 29
5.2 Wizualizacja nogi robota w ostatecznej formie 30
5.3 Wizualizacja modelu robota bez górnej pokrywy 32
5.4 Zdjęcie układu sterowania złożonego z mikrokontrolera i dedykowanej płytki PCB 33
5.5 Porównanie serwomechanizmów Futaba S3003 oraz Feetech FT3325P 35
5.6 Zdjęcia złożonej konstrukcji 36
5.7 Schemat nogi robota 37
6.1 Cyfrowy bliźniak robota w programie CoppeliaSim 41
6.2 Zdjęcie robota w trakcie identyfikacji parametrów dynamicznych ruchu jego kończyn wraz z wyróżnionymi znacznikami ArUco 43

6.3 Wyniki identyfikacji parametrów dynamicznych konstrukcji. Górne wykresy przedstawiają zmiany pozycji, natomiast dolne wykresy przedstawiają prędkości stawów biodrowego (a) oraz kolanowego (b). 43
6.4 Cztery instancje środowiska symulacyjnego CoppeliaSim, na których równolegle prowadzony jest proces uczenia 45
6.5 Schamat przedstawiający przepływ informacji w środowisku w trakcie procesu nauki przez wzmacnianie z wyodrębnioną siecią polityki π oraz siecią krytyka (funkcją wartości) V_f 47
6.6 Przebieg zmian współczynnika wariancji wyjaśnionej dla dwóch wariantów architektury sieci aktora i krytyka w trakcie pierwszych 20 epok treningu . . . 48
6.7 Ilustracja przedstawiająca stan początkowy i możliwe stany kończące epizod: 1) stan początkowy 2) przerwanie eksperymentu spowodowane zbyt dużym przechyłem 3) przerwanie eksperymentu spowodowane zbyt dużym pochyłem 4) przerwanie eksperymentu spowodowane dotarciem do celu 5) przerwanie eksperymentu spowodowane zbyt dużym odchyleniem. 53
6.8 Wizualizacja przebiegu trajektorii referencyjnej kończyny robota 54
6.9 Wizualizacja realizacji sieci aktora π dla sekwencji wejściowej o długości $n = 2$ 55
6.10 Średnia nagroda wraz z przedziałem ufności w kolejnych epokach treningu. Wartości metryki przedstawione na wykresie nie są dokładnie porównywalne, przedstawiają jednak poglądowo postępy nauki. 56
6.11 Wizualizacja procesu nauki agenta przedstawiająca nagrodę zdobytą w kolejnych epizodach treningowych i walidacyjnych. Górna oś przedstawia godzinę liczoną od startu nauki. 61
6.12 Trajektoria końcówki przedniej i tylnej lewej stopy w cyklu chodu, wyznaczona z prostego zadania kinematyki na podstawie kątów w stawach zarejestrowanych w symulacji. 62
7.1 Cykl pracy mikrokontrolera 65
7.2 Model 3D (po lewej) oraz zdjęcie (po prawej) gotowej, dedykowanej stopy robota 68
7.3 Porównanie przebiegu nauki modelu bazowego z modelem uczonym po wprowadzeniu modyfikacji mających na celu ograniczenie luki rzeczywistości 69
7.4 Zdjęcia rzeczywistego robota w trakcie chodu 70
7.5 Kod QR z linkiem do filmu demonstrującego chód rzeczywistego robota w dwóch trybach (po lewej) oraz kod QR z linkiem do dłuższego filmu podsumowującego całą pracę w języku angielskim 71
7.6 Porównanie przechyłu i pochyłu robota w trakcie chodu – wartości z symulacji oraz estymacje z czujnika MPU-6050 72

Spis tabel

4.1 Interpretacja działania algorytmu PPO dla różnych relacji pomiędzy $r_t(\theta)$ a $\hat{A}_t$. Analizując tabelę należy pamiętać, że w metodach optymalizacji gradientowe zmiana wag sieci jest proporcjonalna do gradientu funkcji straty. Nasycenie się wartości straty blokuje niekontrolowaną zmianę wartości wag. 25
4.2 Porównanie kluczowych cech algorytmów A2C, PPO oraz SAC 27
5.1 Zestawienie wybranych parametrów serwomechanizmów Feetech FT3325P oraz Futaba S3003. 34
6.1 Wybór algorytmu uczenia przez wzmacnianie 57
6.2 Wybór częstotliwości sterowania algorytmu 58
6.3 Wybór długości sekwencji wejściowej n 59
6.4 Porównanie wyników algorytmu RL oraz trajektorii referencyjnej 61
7.1 Porównanie czasu inferencji modelu neuronowego 64
7.2 Zestawienie prędkości chodu osiąganej w symulacji z estymowaną prędkością rzeczywistego robota dla dwóch trybów generowania trajektorii chodu 72

Spis załączników

1. Zestawienie elementów robota kroczącego 83

Załącznik 1. Zestawienie elementów robota kroczącego

Zestawienie materiałowe (BOM) robota kroczącego

Lp.	Element	Liczba	Masa j. [g]	Masa całk. [g]	Cena j. [zł]	Cena całk. [zł]
1	Serwomechanizm Futaba S3003	4	44	176	17,89	71,56
2	Segment biodra (PET-G)	4	27	108	1,91	7,63
3	Serwomechanizm Feetech FT3325P	4	24	96	56,90	227,60
4	Płytka Raspberry Pi PICO 2 H	1			27,83	27,83
5	Żyroskop MPU-6050	1			10,70	10,70
6	Przycisk zasilania BULGIN C1500ABMBB	1			9,10	9,10
7	Płytka drukowana PCB JLCPCB	1			7,22	7,22
8	Kondensator elektrolityczny 4700 µF Panasonic	1			7,12	7,12
9	Złącze kołkowe 3×8 Samtec TSW-108-07-T-T	1			6,64	6,64
10	Przełącznik suwakowy NINIGI S22L	1	70	70	3,41	3,41
11	Styk żeński JST SPH-200BK26	2			2,72	5,44
12	Dioda Schottky'ego 1N5822	1			2,43	2,43
13	Terminal śrubowy AK100/2DS-5,08	2			2,38	4,76
14	Kondensator ceramiczny 100 nF	1			0,66	0,66
15	Złącze JST PHR-2	1			0,30	0,30
16	Złącze JST B2B-PH-K-S	1			0,27	0,27
17	Korpus robota – część dolna (PLA)	1	63	63	3,92	3,92
18	Korpus robota – pokrywa (PLA)	1	61	61	3,79	3,79
19	Łożysko biodrowe	4	13	52	11,34	45,36
20	Segment łydki (PET-G)	4	12,5	50	0,75	3,00
21	Korpus robota – część środkowa (PLA)	1	38	38	2,36	2,36
22	Zębatka napędzająca staw kolanowy (PET-G)	4	4,4	17,6	0,27	1,06
23	Łożysko kolanowe	4	2,8	11,2	1,41	5,64
24	Zębatka napędzająca staw biodrowy (PET-G)	4	2,5	10	0,15	0,60
25	Insert M3	15	0,5	7,5	0,56	8,37
26	Śruba imbusowa walcowana M3×12	11	0,6	6,6	0,29	3,22
27	Wał z włókna węglowego 6 mm	2	1,5	3	5,00	10,00
28	Stopa drukowana (TPU)	4	0,5	2	0,50	2,00
29	Śruba imbusowa kulista M3×8	4	0,5	2	0,06	0,24
			Suma	**773,9**		**482,23**